

Contents and provenance

This listing includes application code, matching/workflow logic, database schema, tests and authored build/evaluation scripts. Vendored UI primitives, build scaffolding and package lockfiles are available in the repository. The MIT license applies to original code; dependencies retain their own licenses.

```
.firebaserc
.github/workflows/ci.yml
app/api/workspace/route.ts
app/assets.d.ts
app/chatgpt-auth.ts
app/globals.css
app/layout.tsx
app/lotlight.tsx
app/page.tsx
db/index.ts
db/schema.ts
drizzle/0000_real_hellcat.sql
firebase/index.html
firebase/main.tsx
firebase.json
firestore.rules
lib/lotlight/commands.ts
lib/lotlight/engine.ts
lib/lotlight/files.ts
lib/lotlight/sample.ts
lib/lotlight/semantic-worker.ts
lib/lotlight/storage.ts
lib/lotlight/types.ts
package.json
playwright.config.ts
scripts/assemble-narration.py
scripts/build-ai.mjs
scripts/build-api.mjs
scripts/build-verified.sh
scripts/check-accessibility.mjs
scripts/check-api-isolation.mjs
scripts/create-narration.mjs
scripts/create-submission-pdfs.py
scripts/deploy-firebase.sh
scripts/evaluate-ai.mjs
scripts/execution-profile.mjs
scripts/install-ci.mjs
scripts/install-ci.sh
scripts/install-pnpm.sh
scripts/pnpm-install.mjs
scripts/prepare-public-artifacts.mjs
scripts/record-demo.mjs
scripts/render-captioned-demo.py
scripts/run-framework.mjs
scripts/sites-env.mjs
scripts/sites-env.sh
scripts/transcribe-demo.mjs
server/index.ts
tests/e2e/accessibility.spec.ts
tests/e2e/firebase.spec.ts
```

Contents (continued)

tests/e2e/workflow.spec.ts

tests/engine.test.ts

vite.config.ts

vite.firebase.config.ts

.firebaserc

```
1 {"projects":{"default":"lotlight-care"}}
```

.github/workflows/ci.yml

```

1 name: Lotlight checks
2 on:
3   push:
4   pull_request:
5 jobs:
6   validate:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - uses: actions/setup-node@v4
11        with:
12          node-version: 22
13          cache: npm
14      - run: npm ci
15      - run: npm run typecheck
16      - run: npm test
17      - run: npm run build:firebase
18      - run: npm run build:api
19      - run: npm run build
20      - run: npx playwright install --with-deps chromium
21      - name: Initialize isolated test database
22        run: node --import ./scripts/sites-env.mjs ./node_modules/wrangler/bin/wrangler.js d1 execute DB
--local --config dist/server/wrangler.json --persist-to .wrangler/state --file drizzle/0000_real_hellcat.sql
23      - name: Browser workflows
24        run: |
25          npm run dev -- --port 5173 > /tmp/lotlight-dev.log 2>&1 &
26          for i in $(seq 1 60); do
27            if curl --fail http://localhost:5173/ > /dev/null 2>&1; then break; fi
28            sleep 1
29          done
30          LOTLIGHT_TEST_URL=http://localhost:5173 npm run test:e2e
31      - uses: actions/upload-artifact@v4
32        if: failure()
33        with:
34          name: browser-failure-details
35          path: test-results/

```

app/api/workspace/route.ts

```

1 import { getChatGPTUser } from "@app/chatgpt-auth";
2 import { readWorkspace, writeWorkspace } from "@lib/lotlight/storage";
3 import { reduceWorkspace } from "@lib/lotlight/commands";
4 import { z } from "zod";
5 export const dynamic = "force-dynamic";
6 const reply = (body: unknown, status = 200) =>
7   Response.json(body, {
8     status,
9     headers: {
10       "Cache-Control": "no-store",
11       "X-Content-Type-Options": "nosniff",
12     },
13   });
14 export async function GET() {
15   const user = await getChatGPTUser();
16   if (!user)
17     return reply({ error: "Sign in to access a saved workspace." }, 401);
18   try {
19     return reply(await readWorkspace(user.userId));
20   } catch (error) {
21     console.error(
22       "workspace_read_failed",
23       error instanceof Error ? error.name : "unknown",
24     );
25     return reply(
26       {
27         error:
28           "Your saved workspace is unavailable. Retry without changing your inputs.",
29       },
30       503,
31     );
32   }
33 }
34 export async function POST(request: Request) {
35   const user = await getChatGPTUser();
36   if (!user) return reply({ error: "Sign in to save your workspace." }, 401);
37   const origin = request.headers.get("origin");
38   if (!origin || origin !== new URL(request.url).origin)
39     return reply({ error: "Request origin rejected." }, 403);
40   if (!request.headers.get("content-type")?.startsWith("application/json"))
41     return reply({ error: "Use a JSON request." }, 415);
42   if (Number(request.headers.get("content-length") ?? 0) > 600000)
43     return reply({ error: "Request exceeds the size limit." }, 413);
44   let payload: { revision: number; command?: unknown };
45   try {
46     const body = await request.text();
47     if (body.length > 600000)
48       return reply({ error: "Request exceeds the size limit." }, 413);
49     payload = z
50       .object({ revision: z.number().int().min(0), command: z.unknown() })
51       .strict()
52       .parse(JSON.parse(body));
53   } catch {
54     return reply({ error: "Invalid request." }, 400);
55   }
56   try {
57     const state = await readWorkspace(user.userId);
58     if (state.revision !== payload.revision)
59       return reply(
60         {
61           error:
62             "This workspace changed in another tab. Reload the latest version before saving.",
63         },
64         409,
65       );
66     let next;
67     try {
68       next = reduceWorkspace(state, payload.command, user.displayName);
69     } catch (error) {
70       return reply(
71         {
72           error:
73             error instanceof z.ZodError
74               ? error.issues.map((i) => i.message).join(" ")
75               : error instanceof Error
76                 ? error.message
77                 : "Invalid update.",
78         },
79         400,
80       );

```

app/api/workspace/route.ts (continued)

```
81     }
82     if (JSON.stringify(next).length > 3000000)
83         return reply(
84             {
85                 error:
86                     "Workspace storage limit reached. Export your audit record before starting a new workspace.",
87             },
88             413,
89         );
90     if (!(await writeWorkspace(user.userId, payload.revision, next)))
91         return reply(
92             { error: "Another update arrived first. Reload before retrying." },
93             409,
94         );
95     return reply(next);
96 } catch (error) {
97     console.error(
98         "workspace_save_failed",
99         error instanceof Error ? error.name : "unknown",
100     );
101     return reply(
102         {
103             error:
104                 "Could not save. Your previous saved version is intact. Please retry.",
105         },
106         503,
107     );
108 }
109 }
```

app/assets.d.ts

```
1 declare module '*?url' { const url: string; export default url; }  
2  
3 declare module '*?worker' { const WorkerConstructor: {new():Worker}; export default WorkerConstructor; }
```

app/chatgpt-auth.ts

```

1 import { headers } from "next/headers";
2 import { redirect } from "next/navigation";
3
4 export type ChatGPTUser = {
5   userId: string;
6   displayName: string;
7   email: string;
8   fullName: string | null;
9 };
10
11 const USER_ID_HEADER = "oai-authenticated-user-id";
12 const USER_EMAIL_HEADER = "oai-authenticated-user-email";
13 const USER_FULL_NAME_HEADER = "oai-authenticated-user-full-name";
14 const USER_FULL_NAME_ENCODING_HEADER =
15   "oai-authenticated-user-full-name-encoding";
16 const PERCENT_ENCODED_UTF8 = "percent-encoded-utf-8";
17 const SIGN_IN_PATH = "/signin-with-chatgpt";
18 const SIGN_OUT_PATH = "/signout-with-chatgpt";
19 const CALLBACK_PATH = "/callback";
20
21 export async function getChatGPTUser(): Promise<ChatGPTUser | null> {
22   const requestHeaders = await headers();
23   const userId = requestHeaders.get(USER_ID_HEADER);
24   const email = requestHeaders.get(USER_EMAIL_HEADER);
25   if (!userId || !email) return null;
26
27   const encodedFullName = requestHeaders.get(USER_FULL_NAME_HEADER);
28   const fullName =
29     encodedFullName &&
30     requestHeaders.get(USER_FULL_NAME_ENCODING_HEADER) === PERCENT_ENCODED_UTF8
31       ? safeDecodeURIComponent(encodedFullName)
32       : null;
33
34   return {
35     userId,
36     displayName: fullName ?? email,
37     email,
38     fullName,
39   };
40 }
41
42 export async function requireChatGPTUser(
43   returnTo: string,
44 ): Promise<ChatGPTUser> {
45   const user = await getChatGPTUser();
46   if (user) return user;
47
48   redirect(chatGPTSignInPath(returnTo));
49 }
50
51 export function chatGPTSignInPath(returnTo: string): string {
52   const safeReturnTo = safeRelativeReturnPath(returnTo);
53   return `${SIGN_IN_PATH}?return_to=${encodeURIComponent(safeReturnTo)}`;
54 }
55
56 export function chatGPTSignOutPath(returnTo = "/"): string {
57   const safeReturnTo = safeRelativeReturnPath(returnTo);
58   return `${SIGN_OUT_PATH}?return_to=${encodeURIComponent(safeReturnTo)}`;
59 }
60
61 function safeRelativeReturnPath(value: string): string {
62   if (!value.startsWith("/") || value.startsWith("//")) return "/";
63
64   let url: URL;
65   try {
66     url = new URL(value, "https://app.local");
67   } catch {
68     return "/";
69   }
70   if (url.origin !== "https://app.local") return "/";
71   if (isReservedAuthPath(url.pathname)) return "/";
72
73   return `${url.pathname}${url.search}${url.hash}`;
74 }
75
76 function isReservedAuthPath(pathname: string): boolean {
77   return (
78     pathname === SIGN_IN_PATH ||
79     pathname === SIGN_OUT_PATH ||
80     pathname === CALLBACK_PATH

```

app/chatgpt-auth.ts (continued)

```
81   );  
82 }  
83  
84 function safeDecodeURIComponent(value: string): string | null {  
85   try {  
86     return decodeURIComponent(value);  
87   } catch {  
88     return null;  
89   }  
90 }
```

app/globals.css

```
1 @import "tailwindcss";
2 @import "tw-animate-css";
3 @import "../vendor/shadcn-tailwind-4.13.0.css";
4
5 @utility scrollbar-thin {
6   scrollbar-width: thin;
7 }
8
9 @utility scrollbar-none {
10   scrollbar-width: none;
11 }
12 &::-webkit-scrollbar {
13   display: none;
14 }
15 }
16
17 @utility scrollbar-gutter-stable {
18   scrollbar-gutter: stable;
19 }
20
21 :root {
22   --background: #f8f9f5;
23   --foreground: #263329;
24   --card: #ffffff;
25   --card-foreground: #171717;
26   --popover: #ffffff;
27   --popover-foreground: #171717;
28   --primary: #253c2d;
29   --primary-foreground: #fafafa;
30   --secondary: #f5f5f5;
31   --secondary-foreground: #171717;
32   --muted: #f5f5f5;
33   --muted-foreground: #697469;
34   --accent: #f5f5f5;
35   --accent-foreground: #171717;
36   --destructive: #e7000b;
37   --border: #e0e4da;
38   --input: #e5e5e5;
39   --ring: #a1a1a1;
40   --chart-1: #f54900;
41   --chart-2: #009689;
42   --chart-3: #104e64;
43   --chart-4: #ffb900;
44   --chart-5: #fe9a00;
45   --radius: 0.625rem;
46   --sidebar: #f1f3ed;
47   --sidebar-foreground: #263329;
48   --sidebar-primary: #171717;
49   --sidebar-primary-foreground: #fafafa;
50   --sidebar-accent: #f5f5f5;
51   --sidebar-accent-foreground: #171717;
52   --sidebar-border: #e5e5e5;
53   --sidebar-ring: #a1a1a1;
54 }
55
56 @theme inline {
57   --color-background: var(--background);
58   --color-foreground: var(--foreground);
59   --color-card: var(--card);
60   --color-card-foreground: var(--card-foreground);
61   --color-popover: var(--popover);
62   --color-popover-foreground: var(--popover-foreground);
63   --color-primary: var(--primary);
64   --color-primary-foreground: var(--primary-foreground);
65   --color-secondary: var(--secondary);
66   --color-secondary-foreground: var(--secondary-foreground);
67   --color-muted: var(--muted);
68   --color-muted-foreground: var(--muted-foreground);
69   --color-accent: var(--accent);
70   --color-accent-foreground: var(--accent-foreground);
71   --color-destructive: var(--destructive);
72   --color-border: var(--border);
73   --color-input: var(--input);
74   --color-ring: var(--ring);
75   --color-chart-1: var(--chart-1);
76   --color-chart-2: var(--chart-2);
77   --color-chart-3: var(--chart-3);
78   --color-chart-4: var(--chart-4);
79   --color-chart-5: var(--chart-5);
80   --color-sidebar: var(--sidebar);
```

app/globals.css (continued)

```

81  --color-sidebar-foreground: var(--sidebar-foreground);
82  --color-sidebar-primary: var(--sidebar-primary);
83  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
84  --color-sidebar-accent: var(--sidebar-accent);
85  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
86  --color-sidebar-border: var(--sidebar-border);
87  --color-sidebar-ring: var(--sidebar-ring);
88  --radius-sm: calc(var(--radius) - 4px);
89  --radius-md: calc(var(--radius) - 2px);
90  --radius-lg: var(--radius);
91  --radius-xl: calc(var(--radius) + 4px);
92  --font-sans: Arial, Helvetica, sans-serif;
93  --font-mono:
94      ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono",
95      "Courier New", monospace;
96 }
97
98 body {
99     margin: 0;
100    background: var(--background);
101    color: var(--foreground);
102    font-family: Arial, Helvetica, sans-serif;
103    -webkit-font-smoothing: antialiased;
104 }
105 button,
106 a,
107 input,
108 textarea {
109     -webkit-tap-highlight-color: transparent;
110 }
111 button,
112 a {
113     touch-action: manipulation;
114 }
115 button {
116     cursor: pointer;
117 }
118 button:disabled {
119     cursor: not-allowed;
120     opacity: 0.5;
121 }
122 a {
123     text-decoration: none;
124 }
125 button:focus-visible,
126 a:focus-visible,
127 input:focus-visible,
128 textarea:focus-visible {
129     outline: 3px solid #94aa45;
130     outline-offset: 3px;
131 }
132 .lot-sidebar {
133     border-right: 1px solid #e0e4da;
134 }
135 .lot-sidebar [data-slot="sidebar-inner"] {
136     padding: 24px 14px;
137     background: #f1f3ed;
138 }
139 .brand {
140     display: flex;
141     align-items: center;
142     gap: 10px;
143     font-size: 29px;
144     font-weight: 700;
145     letter-spacing: -1.4px;
146     padding: 0 6px;
147 }
148 .brand-mark {
149     display: grid;
150     place-items: center;
151     width: 36px;
152     height: 38px;
153     background: #ddec8b;
154     border-radius: 11px;
155 }
156 .brand-dot {
157     font-size: 10px;
158     align-self: flex-start;
159     margin-top: 7px;
160     letter-spacing: 0;

```

app/globals.css (continued)

```
161 }
162 .workspace-label {
163   margin: 35px 0 27px;
164   display: flex;
165   align-items: center;
166   gap: 9px;
167   font-size: 11px;
168   font-weight: 600;
169   letter-spacing: -0.15px;
170 }
171 .workspace-avatar {
172   background: white;
173   border: 1px solid #e0e4da;
174   border-radius: 8px;
175   padding: 9px;
176   color: #58694b;
177 }
178 .workspace-label small,
179 .creator small {
180   display: block;
181   color: #58694b;
182   font-size: 10px;
183   font-weight: 400;
184   margin-top: 5px;
185   letter-spacing: 0;
186 }
187 .nav-caption {
188   font-size: 10px;
189   font-weight: 600;
190   letter-spacing: 1.8px;
191   color: #58694b;
192   padding: 0 12px 15px;
193 }
194 .lot-sidebar [data-slot="sidebar-menu-button"] {
195   height: 44px;
196   gap: 12px;
197   padding: 0 12px;
198   font-size: 12px;
199   color: #58694b;
200 }
201 .lot-sidebar .nav-active {
202   background: #e1e7d7 !important;
203   color: #2d3d2a !important;
204   font-weight: 600;
205 }
206 .nav-count {
207   margin-left: auto;
208   background: #fafbf7;
209   padding: 1px 7px;
210   border-radius: 4px;
211   font-size: 10px;
212 }
213 .side-note {
214   margin: 42px 12px 15px;
215   padding-top: 22px;
216   border-top: 1px solid #dce1d4;
217 }
218 .side-note svg {
219   color: #58694b;
220 }
221 .side-note strong {
222   display: block;
223   font-size: 13px;
224   line-height: 1.6;
225   margin-top: 12px;
226   font-weight: 500;
227 }
228 .side-note p {
229   font-size: 11px;
230   line-height: 1.8;
231   color: #58694b;
232   margin-top: 9px;
233 }
234 .creator {
235   display: flex;
236   align-items: center;
237   gap: 10px;
238   font-size: 11px;
239   border-top: 1px solid #dce1d4;
240   padding-top: 20px;
```

app/globals.css (continued)

```
241 }
242 .creator > span {
243   border-radius: 50%;
244   background: #e2e8d6;
245   padding: 10px;
246   font-size: 10px;
247 }
248 .app-shell {
249   width: calc(100% - 224px);
250   min-width: 0;
251   flex: 1;
252 }
253 .topbar {
254   height: 75px;
255   border-bottom: 1px solid #e4e7df;
256   display: flex;
257   align-items: center;
258   justify-content: space-between;
259   padding: 0 44px;
260   font-size: 11px;
261   background: #fcfdf9;
262 }
263 .topbar > div {
264   display: flex;
265   align-items: center;
266   gap: 15px;
267   color: #58694b;
268 }
269 .topbar b {
270   font-weight: 500;
271   color: #495443;
272 }
273 .top-account {
274   display: flex;
275   align-items: center;
276   gap: 8px;
277   color: #4f5e47;
278 }
279 .mobile-menu {
280   display: none;
281 }
282 .main-content {
283   max-width: 1390px;
284   padding: 45px 44px 22px;
285   margin: auto;
286 }
287 .page-heading {
288   display: flex;
289   justify-content: space-between;
290   align-items: flex-end;
291   margin-bottom: 32px;
292 }
293 .eyebrow {
294   font-size: 10px;
295   letter-spacing: 1.8px;
296   font-weight: 700;
297   display: flex;
298   align-items: center;
299   gap: 8px;
300 }
301 .eyebrow > span {
302   width: 6px;
303   height: 6px;
304   border-radius: 50%;
305   background: #7c9646;
306 }
307 h1 {
308   font-size: clamp(36px, 3.8vw, 53px);
309   line-height: 1.09;
310   letter-spacing: -2.1px;
311   font-weight: 500;
312   margin: 19px 0 16px;
313 }
314 .intro {
315   font-size: 13px;
316   line-height: 1.9;
317   color: #58694b;
318 }
319 .heading-actions {
320   display: flex;
```

app/globals.css (continued)

```
321 flex-direction: column;
322 align-items: flex-end;
323 gap: 17px;
324 padding-bottom: 4px;
325 }
326 .demo-label {
327   font-size: 10px;
328   letter-spacing: 1.4px;
329   color: #58694b;
330   border: 1px solid #dce3d2;
331   border-radius: 5px;
332   padding: 6px 8px;
333   background: #f2f5eb;
334 }
335 .button {
336   display: inline-flex;
337   align-items: center;
338   justify-content: center;
339   gap: 8px;
340   border: 1px solid #d9e0d1;
341   padding: 11px 15px;
342   font-size: 11px;
343   font-weight: 500;
344   border-radius: 7px;
345   min-height: 39px;
346   white-space: nowrap;
347   background: white;
348   transition: background 0.15s;
349 }
350 .button:hover {
351   background: #edf1e7;
352 }
353 .button.primary {
354   background: #2b432f;
355   color: #fff;
356   border-color: #2b432f;
357 }
358 .button.primary:hover {
359   background: #38543b;
360 }
361 .overview-strip {
362   display: grid;
363   grid-template-columns: repeat(4, 1fr);
364   border-top: 1px solid #e0e4da;
365   border-bottom: 1px solid #e0e4da;
366   padding: 23px 0;
367   margin: 32px 0 39px;
368 }
369 .overview-strip > div {
370   padding: 0 24px;
371   border-right: 1px solid #e0e4da;
372 }
373 .overview-strip > div:first-child {
374   padding-left: 0;
375 }
376 .overview-strip > div:last-child {
377   border: 0;
378 }
379 .overview-strip span {
380   font-size: 10px;
381   color: #58694b;
382   display: block;
383   margin-bottom: 12px;
384 }
385 .overview-strip strong {
386   font-size: 32px;
387   font-weight: 400;
388   display: flex;
389   align-items: baseline;
390   gap: 13px;
391   letter-spacing: -1px;
392 }
393 .overview-strip small {
394   font-size: 10px;
395   letter-spacing: 0;
396   color: #58694b;
397   max-width: 90px;
398   line-height: 1.5;
399 }
400 .orange-text {
```

app/globals.css (continued)

```
401   color: #58694b;
402 }
403 .section-heading {
404   display: flex;
405   justify-content: space-between;
406   align-items: center;
407   margin-bottom: 17px;
408 }
409 .section-heading h2 {
410   font-size: 14px;
411   font-weight: 500;
412   display: flex;
413   align-items: center;
414   gap: 12px;
415 }
416 .section-heading h2 span {
417   color: #58694b;
418   font-size: 10px;
419 }
420 .section-heading > span {
421   font-size: 10px;
422   color: #58694b;
423 }
424 .review-grid {
425   display: grid;
426   grid-template-columns: minmax(0, 1.9fr) minmax(250px, 1fr);
427   gap: 22px;
428 }
429 .review-card {
430   background: white;
431   border: 1px solid #dce2d4;
432   border-radius: 11px;
433   padding: 25px 26px 0;
434   box-shadow: 0 3px 5px #2b432f03;
435   overflow: hidden;
436 }
437 .card-topline {
438   display: flex;
439   align-items: center;
440   justify-content: space-between;
441   gap: 10px;
442 }
443 .status-tag {
444   font-size: 10px;
445   font-weight: 600;
446   letter-spacing: 0.9px;
447   color: #58694b;
448   display: flex;
449   align-items: center;
450   gap: 6px;
451   background: #fff2e8;
452   border-radius: 5px;
453   padding: 6px 8px;
454 }
455 .status-tag > span {
456   width: 4px;
457   height: 4px;
458   border-radius: 50%;
459   background: #b57343;
460 }
461 .mono,
462 code {
463   font-family: ui-monospace, SFMono-Regular, monospace;
464   font-size: 10px;
465   color: #58694b;
466 }
467 .review-card h2 {
468   font-size: 23px;
469   letter-spacing: -0.8px;
470   font-weight: 500;
471   margin: 25px 0 10px;
472 }
473 .review-card > p {
474   font-size: 11px;
475   color: #58694b;
476 }
477 .scope-line {
478   display: flex;
479   gap: 8px;
480   align-items: center;
```

app/globals.css (continued)

```
481   margin: 21px 0 24px;
482 }
483 .scope-line > span {
484   font-size: 10px;
485   color: #58694b;
486   margin-right: 10px;
487 }
488 .scope-line code {
489   color: #58694b;
490   border: 1px solid #e4e8de;
491   background: #fafbf8;
492   padding: 4px 7px;
493   border-radius: 4px;
494 }
495 .match-visual {
496   background: #f6f8f1;
497   padding: 17px;
498   border-radius: 6px;
499 }
500 .match-bars {
501   display: flex;
502   gap: 5px;
503   margin-bottom: 11px;
504 }
505 .match-bars > span {
506   height: 8px;
507   flex: 1;
508   background: #dfe5d4;
509   border-radius: 2px;
510 }
511 .match-bars > span:nth-child(-n + 2) {
512   background: #c9865a;
513 }
514 .match-bars > span:nth-child(n + 3):nth-child(-n + 4) {
515   background: #d9ca80;
516 }
517 .match-visual > span {
518   font-size: 10px;
519   color: #58694b;
520 }
521 .match-visual b {
522   color: #556249;
523   font-weight: 600;
524 }
525 .match-visual > span > span {
526   margin: 0 8px;
527 }
528 .card-bottom {
529   display: flex;
530   justify-content: space-between;
531   align-items: center;
532   margin-top: 24px;
533   padding: 17px 0;
534   border-top: 1px solid #edf0e7;
535 }
536 .card-bottom > span {
537   display: flex;
538   align-items: center;
539   gap: 7px;
540   font-size: 10px;
541   color: #58694b;
542 }
543 .text-button {
544   display: flex;
545   align-items: center;
546   gap: 8px;
547   font-size: 11px;
548   color: #455b35;
549   font-weight: 600;
550 }
551 .principle-card {
552   background: #e6eec7;
553   border: 1px solid #dee8b8;
554   border-radius: 11px;
555   padding: 22px 25px;
556   display: flex;
557   flex-direction: column;
558 }
559 .mini-orbit {
560   height: 49px;
```

app/globals.css (continued)

```
561 position: relative;
562 margin-bottom: 21px;
563 display: flex;
564 align-items: center;
565 }
566 .mini-orbit svg {
567   color: #58694b;
568 }
569 .mini-orbit span {
570   position: absolute;
571   border: 1px solid #b7c98b;
572   border-radius: 50%;
573   width: 50px;
574   height: 50px;
575   left: -12px;
576 }
577 .mini-orbit span:last-child {
578   left: 8px;
579   border-color: #c9d5a9;
580 }
581 .principle-card .eyebrow {
582   font-size: 9px;
583   letter-spacing: 1.5px;
584   color: #58694b;
585 }
586 .principle-card h3 {
587   font-size: 25px;
588   letter-spacing: -0.8px;
589   line-height: 1.18;
590   font-weight: 400;
591   margin: 13px 0;
592 }
593 .principle-card > p:not(.eyebrow) {
594   font-size: 11px;
595   color: #58694b;
596   line-height: 1.8;
597   max-width: 225px;
598 }
599 .principle-footer {
600   display: flex;
601   justify-content: space-between;
602   align-items: center;
603   font-size: 10px;
604   letter-spacing: 1.2px;
605   color: #58694b;
606   margin-top: auto;
607   padding-top: 23px;
608 }
609 .bottom-explainer {
610   display: grid;
611   grid-template-columns: repeat(3, 1fr);
612   gap: 25px;
613   border-top: 1px solid #e3e7db;
614   margin-top: 36px;
615   padding: 23px 0 30px;
616 }
617 .step-number {
618   font-size: 10px;
619   color: #58694b;
620 }
621 .bottom-explainer h3 {
622   font-size: 12px;
623   margin: 8px 0;
624   color: #58694b;
625   font-weight: 500;
626 }
627 .bottom-explainer p {
628   font-size: 10px;
629   color: #58694b;
630   line-height: 1.7;
631 }
632 .page-footer {
633   display: flex;
634   justify-content: space-between;
635   gap: 16px;
636   border-top: 1px solid #e4e8de;
637   padding-top: 20px;
638   font-size: 10px;
639   color: #58694b;
640   line-height: 1.8;
```

app/globals.css (continued)

```
641 }
642 .page-footer > span:first-child {
643   display: flex;
644   align-items: center;
645   gap: 6px;
646 }
647 @media (min-width: 1500px) {
648   .main-content {
649     padding-top: 65px;
650   }
651   .review-card {
652     padding-top: 32px;
653   }
654   .principle-card {
655     padding-top: 30px;
656   }
657   .review-card h2 {
658     margin-top: 31px;
659   }
660 }
661 @media (max-width: 1100px) {
662   .main-content {
663     padding: 30px 25px;
664   }
665   .topbar {
666     padding: 0 25px;
667   }
668   .overview-strip strong {
669     flex-direction: column;
670     gap: 3px;
671   }
672   .overview-strip small {
673     max-width: none;
674   }
675   .review-grid {
676     grid-template-columns: minmax(0, 1fr);
677   }
678   .principle-card {
679     display: none;
680   }
681   .overview-strip > div {
682     padding: 0 13px;
683   }
684 }
685 @media (max-width: 767px) {
686   .mobile-menu {
687     display: flex;
688   }
689   .app-shell {
690     width: 100%;
691   }
692   .topbar {
693     padding: 0 18px;
694     height: 60px;
695   }
696   .main-content {
697     padding: 28px 19px;
698   }
699   .page-heading {
700     display: block;
701   }
702   .heading-actions {
703     align-items: flex-start;
704     flex-direction: row;
705     align-items: center;
706     margin-top: 22px;
707   }
708   .demo-label {
709     order: 2;
710   }
711   h1 {
712     font-size: 43px;
713   }
714   .overview-strip {
715     grid-template-columns: 1fr 1fr;
716     gap: 20px;
717     margin: 25px 0;
718   }
719   .overview-strip > div {
720     padding: 0 !important;
```

app/globals.css (continued)

```
721     border: 0;
722   }
723   .overview-strip strong {
724     flex-direction: row;
725     align-items: baseline;
726     font-size: 29px;
727     gap: 9px;
728   }
729   .overview-strip small {
730     max-width: 85px;
731   }
732   .section-heading {
733     align-items: flex-start;
734     gap: 15px;
735   }
736   .section-heading > span {
737     max-width: 115px;
738     font-size: 10px;
739     text-align: right;
740   }
741   .review-card {
742     padding: 20px 18px 0;
743   }
744   .review-card h2 {
745     font-size: 22px;
746   }
747   .bottom-explainer {
748     gap: 15px;
749   }
750   .bottom-explainer h3 {
751     line-height: 1.5;
752     font-size: 11px;
753   }
754   .page-footer {
755     flex-direction: column;
756   }
757   .card-bottom > span {
758     font-size: 10px;
759   }
760   .scope-line {
761     flex-wrap: wrap;
762   }
763   .topbar > div {
764     gap: 9px;
765   }
766 }
767 .notice-stack {
768   display: grid;
769   gap: 20px;
770 }
771 .workspace-tools {
772   display: flex;
773   gap: 25px;
774   margin-bottom: 25px;
775 }
776 .muted {
777   color: #58694b;
778   font-size: 10px;
779 }
780 .account-area {
781   gap: 20px !important;
782 }
783 .save-status {
784   font-size: 10px;
785   color: #58694b;
786 }
787 .compact {
788   margin-bottom: 28px;
789   align-items: center;
790 }
791 .compact h1 {
792   font-size: 36px;
793   letter-spacing: -1.4px;
794   margin: 12px 0;
795 }
796 .back-button {
797   display: flex;
798   align-items: center;
799   gap: 7px;
800   font-size: 10px;
```

app/globals.css (continued)

```
801   color: #58694b;
802   margin-bottom: 23px;
803 }
804 .review-disclaimer,
805 .info-banner,
806 .error-banner {
807   display: flex;
808   gap: 10px;
809   align-items: flex-start;
810   background: #f0f3e7;
811   border: 1px solid #e0e7d0;
812   border-radius: 7px;
813   padding: 13px 16px;
814   font-size: 11px;
815   line-height: 1.8;
816   color: #58694b;
817   margin-bottom: 25px;
818 }
819 .review-disclaimer svg,
820 .info-banner svg,
821 .error-banner svg {
822   flex-shrink: 0;
823   margin-top: 2px;
824 }
825 .error-banner {
826   background: #fff2ea;
827   color: #58694b;
828   border-color: #ebc9b7;
829 }
830 .error-banner .text-button {
831   margin-left: auto;
832   white-space: nowrap;
833 }
834 .review-tabs > [data-slot="tabs-list"] {
835   width: 100%;
836   justify-content: flex-start;
837   border-bottom: 1px solid #dfe5d5;
838   padding: 0;
839   height: 48px;
840   margin-bottom: 22px;
841   gap: 30px;
842 }
843 .review-tabs [data-slot="tabs-trigger"] {
844   flex: 0 1 auto;
845   font-size: 12px;
846   padding: 0 0 13px;
847   border-radius: 0;
848   gap: 10px;
849 }
850 .review-tabs [data-slot="tabs-trigger"]:after {
851   background: #65823c;
852   height: 2px;
853   bottom: 0;
854 }
855 .tab-count {
856   font-size: 10px;
857   padding: 2px 5px;
858   background: #e9eee0;
859   border-radius: 3px;
860   color: #58694b;
861 }
862 .ai-banner {
863   display: flex;
864   align-items: center;
865   gap: 15px;
866   padding: 20px;
867   border: 1px solid #dbe5c9;
868   background: #f0f5e7;
869   border-radius: 9px;
870   margin-bottom: 25px;
871 }
872 .ai-icon {
873   background: #e2ebcf;
874   border-radius: 8px;
875   padding: 11px;
876   color: #58694b;
877 }
878 .ai-banner > div:nth-child(2) {
879   flex: 1;
880 }
```

app/globals.css (continued)

```

881 .ai-banner strong {
882   font-size: 12px;
883   font-weight: 500;
884 }
885 .ai-banner p {
886   font-size: 10px;
887   line-height: 1.8;
888   color: #58694b;
889   margin-top: 5px;
890   max-width: 590px;
891 }
892 .table-toolbar {
893   display: flex;
894   align-items: center;
895   justify-content: space-between;
896   gap: 15px;
897   margin: 16px 0;
898 }
899 .legend {
900   display: flex;
901   gap: 15px;
902   align-items: center;
903   font-size: 10px;
904   color: #58694b;
905 }
906 .legend > span {
907   display: flex;
908   gap: 5px;
909   align-items: center;
910 }
911 .legend i {
912   width: 5px;
913   height: 5px;
914   border-radius: 50%;
915   display: block;
916 }
917 .dot-exact {
918   background: #bd805a;
919 }
920 .dot-missing {
921   background: #bdab5b;
922 }
923 .dot-unlisted {
924   background: #afb00;
925 }
926 .search-box {
927   display: flex;
928   align-items: center;
929   gap: 7px;
930   border: 1px solid #dce4d2;
931   border-radius: 6px;
932   padding: 8px 10px;
933   background: white;
934   color: #58694b;
935 }
936 .search-box input {
937   outline: none;
938   border: none;
939   width: 140px;
940   font-size: 10px;
941   color: #536447;
942   background: none;
943 }
944 .match-row {
945   background: #fff;
946   border: 1px solid #dfe5d6;
947   border-radius: 8px;
948   overflow: hidden;
949   margin-bottom: 10px;
950 }
951 .match-row-main {
952   display: grid;
953   grid-template-columns: minmax(200px, 1fr) 135px 135px 30px;
954   gap: 15px;
955   align-items: center;
956   padding: 19px 18px;
957 }
958 .match-product {
959   display: flex;
960   gap: 13px;

```

app/globals.css (continued)

```
961 align-items: center;
962 }
963 .match-symbol {
964   padding: 10px;
965   border-radius: 7px;
966   background: #f3f5ee;
967   color: #58694b;
968 }
969 .match-symbol.exact {
970   background: #fcf1e9;
971   color: #58694b;
972 }
973 .match-symbol.missing,
974 .match-symbol.possible {
975   background: #faf5e1;
976   color: #58694b;
977 }
978 .match-product h3 {
979   font-size: 12px;
980   font-weight: 500;
981   line-height: 1.5;
982 }
983 .match-product p {
984   font-size: 10px;
985   color: #58694b;
986   margin-top: 6px;
987   line-height: 1.7;
988 }
989 .match-product p > span {
990   margin: 0 4px;
991 }
992 .match-identifiers {
993   display: flex;
994   flex-direction: column;
995   gap: 5px;
996 }
997 .match-identifiers code {
998   font-size: 11px;
999   color: #58694b;
1000   letter-spacing: 0.2px;
1001 }
1002 .match-identifiers > span {
1003   font-size: 10px;
1004   color: #58694b;
1005 }
1006 .match-identifiers b {
1007   font-family: ui-monospace, monospace;
1008   font-weight: 400;
1009   color: #58694b;
1010   font-size: 10px;
1011   margin-left: 6px;
1012 }
1013 .match-state {
1014   display: flex;
1015   flex-direction: column;
1016   align-items: flex-start;
1017   gap: 5px;
1018 }
1019 .pill {
1020   font-size: 10px;
1021   padding: 5px 7px;
1022   border-radius: 4px;
1023   line-height: 1.4;
1024   font-weight: 500;
1025   display: inline-flex;
1026   width: fit-content;
1027   background: #f0f3ea;
1028   color: #58694b;
1029 }
1030 .pill.exact {
1031   background: #fcf0e7;
1032   color: #58694b;
1033 }
1034 .pill.missing,
1035 .pill.possible {
1036   background: #f9f3db;
1037   color: #58694b;
1038 }
1039 .pill.complete {
1040   background: #eaf3e4;
```

app/globals.css (continued)

```

1041   color: #58694b;
1042 }
1043 .match-state small {
1044   font-size: 10px;
1045   color: #58694b;
1046 }
1047 .icon-button {
1048   padding: 6px;
1049   color: #58694b;
1050   border-radius: 5px;
1051 }
1052 .icon-button:hover {
1053   background: #eef2e5;
1054 }
1055 .match-expanded {
1056   display: flex;
1057   align-items: flex-start;
1058   justify-content: space-between;
1059   gap: 20px;
1060   border-top: 1px solid #ecf0e5;
1061   padding: 20px 22px;
1062   background: #fcfdf9;
1063 }
1064 .match-expanded > div:first-child {
1065   flex: 1;
1066 }
1067 .match-expanded strong {
1068   font-size: 10px;
1069   font-weight: 600;
1070   color: #58694b;
1071 }
1072 .match-expanded p {
1073   font-size: 11px;
1074   line-height: 1.8;
1075   margin-top: 7px;
1076   color: #58694b;
1077 }
1078 .match-expanded blockquote {
1079   font-size: 10px;
1080   font-family: ui-monospace, monospace;
1081   border-left: 2px solid #cad7b7;
1082   padding: 7px 12px;
1083   margin: 11px 0 0;
1084   color: #58694b;
1085   white-space: pre-wrap;
1086 }
1087 .gate-hint {
1088   font-size: 10px;
1089   display: block;
1090   color: #58694b;
1091   max-width: 150px;
1092   margin-top: 7px;
1093   line-height: 1.6;
1094 }
1095 .fine-print {
1096   font-size: 10px;
1097   line-height: 1.8;
1098   color: #58694b;
1099   margin: 15px 0 22px;
1100 }
1101 .source-grid {
1102   display: grid;
1103   grid-template-columns: 1.3fr 1fr;
1104   gap: 24px;
1105 }
1106 .source-document {
1107   background: #fff;
1108   border: 1px solid #dfe5d6;
1109   border-radius: 8px;
1110   padding: 23px;
1111 }
1112 .source-document-heading {
1113   display: flex;
1114   align-items: center;
1115   gap: 8px;
1116   border-bottom: 1px solid #edf1e5;
1117   padding-bottom: 17px;
1118   color: #58694b;
1119   font-size: 11px;
1120 }

```

app/globals.css (continued)

```
1121 .source-document-heading a {
1122   display: flex;
1123   align-items: center;
1124   gap: 5px;
1125   margin-left: auto;
1126   font-size: 10px;
1127   color: #58694b;
1128 }
1129 .source-document pre {
1130   white-space: pre-wrap;
1131   overflow-wrap: anywhere;
1132   font:
1133     11px/2 ui-monospace,
1134     monospace;
1135   color: #58694b;
1136   margin: 20px 0;
1137 }
1138 .source-document > .text-button {
1139   font-size: 10px;
1140   margin-top: 25px;
1141 }
1142 .source-review-panel {
1143   padding: 20px 23px;
1144   background: #f1f4e9;
1145   border: 1px solid #e0e6d4;
1146   border-radius: 8px;
1147 }
1148 .source-review-panel h3 {
1149   font-size: 21px;
1150   letter-spacing: -0.5px;
1151   margin: 16px 0 10px;
1152 }
1153 .source-review-panel > p {
1154   font-size: 11px;
1155   line-height: 1.8;
1156   color: #58694b;
1157 }
1158 .scope-review-row {
1159   display: grid;
1160   grid-template-columns: 1fr 1fr;
1161   gap: 8px;
1162   border-bottom: 1px solid #dee5d1;
1163   padding: 18px 0;
1164 }
1165 .scope-review-row strong {
1166   font:
1167     12px ui-monospace,
1168     monospace;
1169   color: #58694b;
1170 }
1171 .scope-review-row > span {
1172   font:
1173     10px ui-monospace,
1174     monospace;
1175   color: #58694b;
1176   text-align: right;
1177 }
1178 .scope-review-row small {
1179   grid-column: 1/-1;
1180   font-size: 10px;
1181   line-height: 1.7;
1182   color: #58694b;
1183 }
1184 .source-action {
1185   padding: 20px 0;
1186 }
1187 .source-action strong {
1188   font-size: 10px;
1189 }
1190 .source-action p {
1191   font-size: 11px;
1192   line-height: 1.8;
1193   color: #58694b;
1194   margin: 8px 0;
1195 }
1196 .source-action small {
1197   font-size: 10px;
1198   color: #58694b;
1199 }
1200 .check-label {
```

app/globals.css (continued)

```
1201 display: flex;
1202 align-items: flex-start;
1203 gap: 9px;
1204 font-size: 10px;
1205 line-height: 1.8;
1206 color: #58694b;
1207 margin: 15px 0;
1208 }
1209 .check-label [data-slot="checkbox"] {
1210   margin-top: 2px;
1211   flex-shrink: 0;
1212 }
1213 .full-width {
1214   width: 100%;
1215 }
1216 .approved-box {
1217   display: flex;
1218   gap: 10px;
1219   padding: 15px;
1220   background: #e4edda;
1221   border: 1px solid #d5e1c5;
1222   border-radius: 7px;
1223   color: #58694b;
1224 }
1225 .approved-box strong {
1226   font-size: 11px;
1227   display: block;
1228 }
1229 .approved-box small {
1230   font-size: 10px;
1231   line-height: 1.8;
1232   display: block;
1233   margin-top: 6px;
1234 }
1235 .status-tag.approved {
1236   color: #58694b;
1237   background: #eaf2df;
1238 }
1239 .status-tag.approved > span {
1240   background: #8d9f6f;
1241 }
1242 .data-table {
1243   border: 1px solid #dfe5d6;
1244   border-radius: 8px;
1245   background: white;
1246   overflow: hidden;
1247 }
1248 .data-table th {
1249   background: #f2f5ed;
1250   font-size: 10px;
1251   color: #58694b;
1252   letter-spacing: 0.4px;
1253   height: 43px;
1254   padding: 0 15px;
1255 }
1256 .data-table td {
1257   padding: 17px 15px;
1258   font-size: 10px;
1259   color: #58694b;
1260   border-color: #e9efdf;
1261 }
1262 .data-table .product-cell {
1263   color: #58694b;
1264   font-weight: 500;
1265   font-size: 11px;
1266 }
1267 .data-table code {
1268   font-size: 10px;
1269   color: #58694b;
1270 }
1271 .missing-text {
1272   color: #58694b !important;
1273 }
1274 .download-template {
1275   margin: 20px 0 50px;
1276   font-size: 10px;
1277 }
1278 .empty-state {
1279   text-align: center;
1280   padding: 55px 25px;
```

app/globals.css (continued)

```
1281 background: #f4f6ee;
1282 border: 1px dashed #d4ddc7;
1283 border-radius: 9px;
1284 margin-bottom: 25px;
1285 color: #58694b;
1286 }
1287 .empty-state > svg {
1288   margin: 0 auto 18px;
1289 }
1290 .empty-state h3 {
1291   font-size: 19px;
1292   color: #58694b;
1293   font-weight: 400;
1294   margin-bottom: 10px;
1295 }
1296 .empty-state p {
1297   font-size: 11px;
1298   line-height: 1.8;
1299   max-width: 420px;
1300   margin: auto;
1301 }
1302 .empty-state .button {
1303   margin-top: 20px;
1304 }
1305 .action-cards {
1306   display: grid;
1307   gap: 13px;
1308 }
1309 .action-card {
1310   padding: 20px;
1311   border: 1px solid #dfe6d3;
1312   background: white;
1313   border-radius: 9px;
1314   display: flex;
1315   justify-content: space-between;
1316   gap: 20px;
1317 }
1318 .action-card h3 {
1319   font-size: 12px;
1320   line-height: 1.7;
1321   margin-top: 10px;
1322   color: #58694b;
1323 }
1324 .action-card p {
1325   font-size: 10px;
1326   line-height: 1.8;
1327   color: #58694b;
1328   margin-top: 5px;
1329 }
1330 .action-card > .mono {
1331   white-space: nowrap;
1332 }
1333 .activity-title {
1334   font-size: 17px;
1335   margin: 35px 0 25px;
1336   font-weight: 400;
1337 }
1338 .timeline {
1339   margin: 0 0 40px 6px;
1340 }
1341 .timeline-item {
1342   display: flex;
1343   gap: 18px;
1344   position: relative;
1345   padding-bottom: 26px;
1346   border-left: 1px solid #d7e1cb;
1347   padding-left: 25px;
1348 }
1349 .timeline-dot {
1350   position: absolute;
1351   width: 7px;
1352   height: 7px;
1353   background: #a9b994;
1354   left: -4px;
1355   top: 6px;
1356   border-radius: 50%;
1357   box-shadow: 0 0 4px #f8f9f5;
1358 }
1359 .timeline-item strong {
1360   font-size: 11px;
```

app/globals.css (continued)

```
1361 font-weight: 500;
1362 text-transform: capitalize;
1363 color: #58694b;
1364 }
1365 .timeline-item p {
1366 font-size: 11px;
1367 line-height: 1.8;
1368 color: #58694b;
1369 max-width: 850px;
1370 margin: 7px 0;
1371 }
1372 .timeline-item small {
1373 font-size: 10px;
1374 color: #58694b;
1375 }
1376 .about-page h1 {
1377 font-size: 43px;
1378 }
1379 .intro.large {
1380 max-width: 700px;
1381 font-size: 15px;
1382 }
1383 .about-story {
1384 margin: 35px 0;
1385 padding: 30px;
1386 border-radius: 9px;
1387 background: #eaf0d8;
1388 }
1389 .about-story h2 {
1390 font-size: 29px;
1391 letter-spacing: -1px;
1392 font-weight: 400;
1393 margin: 20px 0;
1394 }
1395 .about-story p {
1396 max-width: 700px;
1397 font-size: 12px;
1398 line-height: 1.9;
1399 color: #58694b;
1400 margin: 15px 0 20px;
1401 }
1402 .about-columns {
1403 display: grid;
1404 grid-template-columns: 1fr 1fr;
1405 gap: 35px;
1406 margin: 40px 0;
1407 }
1408 .about-columns h2 {
1409 font-size: 19px;
1410 font-weight: 400;
1411 margin-bottom: 16px;
1412 letter-spacing: -0.4px;
1413 }
1414 .about-columns p {
1415 font-size: 12px;
1416 line-height: 1.9;
1417 color: #58694b;
1418 margin: 12px 0;
1419 }
1420 .about-columns strong {
1421 font-weight: 500;
1422 color: #58694b;
1423 }
1424 .about-columns a {
1425 display: block;
1426 font-size: 10px;
1427 color: #58694b;
1428 text-decoration: underline;
1429 text-underline-offset: 3px;
1430 margin-top: 12px;
1431 }
1432 .creator-credit {
1433 display: flex;
1434 align-items: center;
1435 gap: 16px;
1436 border-top: 1px solid #e0e6d6;
1437 padding: 25px 0 40px;
1438 }
1439 .creator-credit strong {
1440 font-size: 12px;
```

app/globals.css (continued)

```
1441 font-weight: 500;
1442 }
1443 .creator-credit p {
1444   font-size: 10px;
1445   line-height: 1.7;
1446   color: #58694b;
1447   margin-top: 5px;
1448 }
1449 .creator-credit a {
1450   font-size: 10px;
1451   white-space: nowrap;
1452   margin-left: auto;
1453   color: #58694b;
1454 }
1455 .loading-state {
1456   padding: 100px 30px;
1457   text-align: center;
1458 }
1459 .loading-state > svg {
1460   margin: auto;
1461   color: #58694b;
1462 }
1463 .loading-state h1 {
1464   font-size: 30px;
1465 }
1466 .loading-state p {
1467   font-size: 12px;
1468   color: #58694b;
1469 }
1470 .spin {
1471   animation: spin 1s linear infinite;
1472 }
1473 @keyframes spin {
1474   to {
1475     transform: rotate(360deg);
1476   }
1477 }
1478 .wide-dialog {
1479   max-width: 680px !important;
1480 }
1481 .notice-dialog {
1482   max-width: 830px !important;
1483   max-height: 90vh;
1484   overflow: auto;
1485 }
1486 .wide-dialog {
1487   max-height: 90vh;
1488   overflow: auto;
1489 }
1490 [data-slot="dialog-content"],
1491 [data-slot="alert-dialog-content"] {
1492   background: #fcfdf9;
1493   border-color: #d7e0ca;
1494   color: #405331;
1495   border-radius: 12px;
1496   padding: 28px;
1497 }
1498 [data-slot="dialog-title"],
1499 [data-slot="alert-dialog-title"] {
1500   font-size: 25px !important;
1501   font-weight: 400;
1502   letter-spacing: -0.7px;
1503   line-height: 1.25 !important;
1504 }
1505 [data-slot="dialog-description"],
1506 [data-slot="alert-dialog-description"] {
1507   font-size: 11px !important;
1508   line-height: 1.8;
1509   color: #58694b;
1510 }
1511 [data-slot="dialog-footer"] {
1512   margin-top: 20px;
1513 }
1514 .field-label {
1515   display: flex;
1516   flex-direction: column;
1517   gap: 7px;
1518   font-size: 10px;
1519   color: #58694b;
1520   margin-bottom: 17px;
```

app/globals.css (continued)

```
1521 }
1522 .field-label input,
1523 .field-label textarea {
1524   background: white;
1525   border: 1px solid #d9e3cb;
1526   border-radius: 6px;
1527   padding: 10px 12px;
1528   font:
1529     12px/1.7 Arial,
1530     sans-serif;
1531   color: #4e663b;
1532   min-width: 0;
1533   width: 100%;
1534 }
1535 .field-label input::placeholder,
1536 .field-label textarea::placeholder {
1537   color: #58694b;
1538 }
1539 .field-label textarea {
1540   resize: vertical;
1541   min-height: 65px;
1542 }
1543 .form-two {
1544   display: grid;
1545   grid-template-columns: 1fr 1fr;
1546   gap: 16px;
1547 }
1548 .file-input-label {
1549   position: relative;
1550   overflow: hidden;
1551   display: flex;
1552   align-items: center;
1553   justify-content: center;
1554   gap: 9px;
1555   cursor: pointer;
1556   background: #eff4e5;
1557   border: 1px dashed #cbdab7;
1558   border-radius: 6px;
1559   padding: 14px;
1560   font-size: 11px;
1561   color: #58694b;
1562   margin: 5px 0 18px;
1563 }
1564 .file-input-label input {
1565   position: absolute;
1566   inset: 0;
1567   opacity: 0;
1568   cursor: pointer;
1569   width: 100%;
1570   height: 100%;
1571 }
1572 .form-meta {
1573   display: flex;
1574   justify-content: space-between;
1575   align-items: center;
1576   gap: 15px;
1577 }
1578 .form-meta .text-button {
1579   font-size: 10px;
1580 }
1581 .form-meta > span {
1582   font-size: 10px;
1583   color: #58694b;
1584 }
1585 .form-error {
1586   font-size: 11px;
1587   line-height: 1.7;
1588   padding: 12px;
1589   background: #fcf0e9;
1590   border: 1px solid #efd8c9;
1591   border-radius: 6px;
1592   color: #58694b;
1593   margin: 15px 0;
1594 }
1595 .source-upload-row {
1596   display: flex;
1597   gap: 25px;
1598   align-items: center;
1599 }
1600 .source-upload-row .file-input-label {
```

app/globals.css (continued)

```
1601   min-width: 185px;
1602 }
1603 .source-upload-row .text-button {
1604   font-size: 10px;
1605   margin-bottom: 14px;
1606 }
1607 .scope-editor-heading {
1608   margin: 25px 0 8px;
1609 }
1610 .scope-editor-heading h2 {
1611   font-size: 13px;
1612 }
1613 .scope-editor-heading .text-button {
1614   font-size: 10px;
1615 }
1616 .scope-editor {
1617   background: #f2f6eb;
1618   border: 1px solid #dce6ce;
1619   border-radius: 7px;
1620   padding: 17px;
1621   margin: 13px 0;
1622 }
1623 .scope-editor-fields {
1624   display: grid;
1625   grid-template-columns: 1fr 1.5fr 30px;
1626   gap: 15px;
1627   align-items: center;
1628 }
1629 .scope-editor-fields .field-label {
1630   margin: 0;
1631 }
1632 .scope-editor > .field-label {
1633   margin-bottom: 0;
1634 }
1635 .scope-editor .check-label {
1636   font-size: 10px;
1637 }
1638 .add-scope {
1639   margin: 17px 0 25px;
1640   font-size: 10px;
1641 }
1642 .quantity-note {
1643   align-self: center;
1644   font-size: 12px;
1645   color: #58694b;
1646   padding-bottom: 12px;
1647 }
1648 .quantity-note small {
1649   display: block;
1650   margin-top: 5px;
1651   font-size: 10px;
1652   color: #58694b;
1653 }
1654 .export-options {
1655   display: grid;
1656   gap: 12px;
1657   margin-top: 12px;
1658 }
1659 .export-options button {
1660   display: flex;
1661   align-items: center;
1662   gap: 15px;
1663   text-align: left;
1664   border: 1px solid #dce5d0;
1665   padding: 18px;
1666   border-radius: 8px;
1667 }
1668 .export-options button:hover {
1669   background: #f2f6e9;
1670 }
1671 .export-options button > svg {
1672   color: #58694b;
1673   width: 21px;
1674 }
1675 .export-options button > svg:last-child {
1676   margin-left: auto;
1677 }
1678 .export-options strong {
1679   font-size: 12px;
1680   font-weight: 500;
```

app/globals.css (continued)

```
1681 display: block;
1682 }
1683 .export-options small {
1684   display: block;
1685   font-size: 10px;
1686   line-height: 1.7;
1687   color: #58694b;
1688   margin-top: 5px;
1689 }
1690 .report-toolbar {
1691   display: flex;
1692   justify-content: space-between;
1693   margin-bottom: 25px;
1694 }
1695 .audit-report {
1696   background: white;
1697   padding: 40px;
1698   border: 1px solid #dfe6d5;
1699   margin-bottom: 35px;
1700 }
1701 .report-head {
1702   display: flex;
1703   justify-content: space-between;
1704   align-items: center;
1705 }
1706 .report-head > span {
1707   font-size: 10px;
1708   letter-spacing: 1px;
1709   color: #58694b;
1710 }
1711 .audit-report h1 {
1712   font-size: 30px;
1713   letter-spacing: -1px;
1714   margin: 30px 0 15px;
1715 }
1716 .audit-report p {
1717   font-size: 11px;
1718   line-height: 1.8;
1719   color: #58694b;
1720   margin: 10px 0;
1721 }
1722 .audit-report h2 {
1723   font-size: 18px;
1724   font-weight: 500;
1725   margin: 28px 0 15px;
1726 }
1727 .audit-report h3 {
1728   font-size: 12px;
1729   margin: 20px 0 10px;
1730 }
1731 .audit-report a {
1732   font-size: 10px;
1733   overflow-wrap: anywhere;
1734   color: #58694b;
1735 }
1736 .audit-report th {
1737   font-size: 10px;
1738   white-space: normal;
1739 }
1740 .audit-report td {
1741   font-size: 10px;
1742   white-space: normal;
1743   vertical-align: top;
1744   color: #58694b;
1745   line-height: 1.7;
1746 }
1747 .audit-report td small {
1748   font-size: 10px;
1749 }
1750 .report-notice {
1751   padding: 15px;
1752   background: #f7f2df;
1753 }
1754 .report-event {
1755   padding: 12px 0;
1756   border-bottom: 1px solid #edf0e6;
1757   break-inside: avoid;
1758 }
1759 .report-event strong {
1760   font-size: 10px;
```

app/globals.css (continued)

```
1761 }
1762 .report-event small {
1763   font-size: 10px;
1764   color: #58694b;
1765   float: right;
1766 }
1767 .report-event p {
1768   font-size: 10px;
1769 }
1770 .response-heading {
1771   margin: 10px 0 25px;
1772 }
1773 .response-heading .fine-print {
1774   margin: 8px 0;
1775 }
1776 .page-footer {
1777   margin-top: 30px;
1778 }
1779 @media (max-width: 1100px) {
1780   .source-grid {
1781     grid-template-columns: 1fr;
1782   }
1783   .match-row-main {
1784     grid-template-columns: minmax(160px, 1fr) 105px 120px 24px;
1785     gap: 9px;
1786     padding: 15px 12px;
1787   }
1788   .match-symbol {
1789     display: none;
1790   }
1791   .legend {
1792     gap: 8px;
1793   }
1794   .ai-banner {
1795     padding: 15px;
1796   }
1797   .table-toolbar {
1798     flex-wrap: wrap;
1799   }
1800   .search-box {
1801     margin-left: auto;
1802   }
1803 }
1804 @media (max-width: 767px) {
1805   .save-status {
1806     display: none;
1807   }
1808   .compact {
1809     display: block;
1810   }
1811   .compact > .button {
1812     margin-top: 12px;
1813   }
1814   .compact h1 {
1815     font-size: 32px;
1816   }
1817   .review-tabs > [data-slot="tabs-list"] {
1818     gap: 16px;
1819   }
1820   .review-tabs [data-slot="tabs-trigger"] {
1821     font-size: 11px;
1822     gap: 5px;
1823   }
1824   .ai-banner {
1825     flex-wrap: wrap;
1826     gap: 11px;
1827   }
1828   .ai-icon {
1829     padding: 8px;
1830   }
1831   .ai-banner > div:nth-child(2) {
1832     flex-basis: calc(100% - 55px);
1833   }
1834   .ai-banner > .button {
1835     margin-left: 43px;
1836   }
1837   .match-row-main {
1838     grid-template-columns: 1fr 26px;
1839     position: relative;
1840     gap: 13px;
```

app/globals.css (continued)

```
1841 }
1842 .match-product {
1843   grid-column: 1;
1844 }
1845 .match-row-main > .icon-button {
1846   grid-column: 2;
1847   grid-row: 1;
1848 }
1849 .match-identifiers {
1850   grid-column: 1;
1851   flex-direction: row;
1852   gap: 12px;
1853 }
1854 .match-state {
1855   grid-column: 1/-1;
1856   flex-direction: row;
1857   align-items: center;
1858 }
1859 .match-expanded {
1860   flex-direction: column;
1861   padding: 16px;
1862 }
1863 .match-expanded > div:last-child {
1864   width: 100%;
1865 }
1866 .legend {
1867   font-size: 10px;
1868   flex-wrap: wrap;
1869 }
1870 .search-box {
1871   width: 100%;
1872 }
1873 .search-box input {
1874   width: 100%;
1875 }
1876 .source-review-panel,
1877 .source-document {
1878   padding: 19px;
1879 }
1880 .form-two {
1881   grid-template-columns: 1fr;
1882   gap: 0;
1883 }
1884 .source-upload-row {
1885   flex-direction: column;
1886   align-items: stretch;
1887   gap: 0;
1888 }
1889 .scope-editor-fields {
1890   grid-template-columns: 1fr 1fr 22px;
1891   gap: 8px;
1892 }
1893 .source-upload-row .text-button {
1894   font-size: 10px;
1895 }
1896 .scope-editor-heading {
1897   gap: 15px;
1898   align-items: center;
1899 }
1900 .scope-editor-heading .text-button {
1901   text-align: right;
1902 }
1903 .about-columns {
1904   grid-template-columns: 1fr;
1905   gap: 25px;
1906 }
1907 .about-page h1 {
1908   font-size: 35px;
1909 }
1910 .creator-credit {
1911   flex-wrap: wrap;
1912 }
1913 .creator-credit > div {
1914   flex: 1;
1915 }
1916 .creator-credit a {
1917   width: 100%;
1918   margin-left: 51px;
1919 }
1920 .audit-report {
```

app/globals.css (continued)

```

1921     padding: 20px;
1922   }
1923   .report-head > span {
1924     max-width: 100px;
1925     line-height: 1.7;
1926     text-align: right;
1927   }
1928   .report-event small {
1929     display: block;
1930     float: none;
1931     margin-top: 5px;
1932   }
1933   .action-card {
1934     flex-direction: column;
1935     gap: 10px;
1936   }
1937   .workspace-tools {
1938     gap: 20px;
1939     flex-wrap: wrap;
1940   }
1941   .form-meta {
1942     align-items: flex-start;
1943   }
1944   .form-meta > span {
1945     text-align: right;
1946   }
1947   .gate-hint {
1948     max-width: none;
1949   }
1950   .export-options button {
1951     padding: 12px;
1952     gap: 10px;
1953   }
1954   .export-options small {
1955     font-size: 10px;
1956   }
1957 }
1958 @media (prefers-reduced-motion: reduce) {
1959   *,
1960   *:before,
1961   *:after {
1962     animation: none !important;
1963     transition: none !important;
1964   }
1965 }
1966 @media print {
1967   [data-slot="sidebar"],
1968   [data-slot="sidebar-wrapper"] > [data-slot="sidebar"],
1969   .topbar,
1970   .page-footer,
1971   .report-toolbar,
1972   [data-sonner-toaster] {
1973     display: none !important;
1974   }
1975   [data-slot="sidebar-container"],
1976   [data-slot="sidebar-gap"] {
1977     display: none !important;
1978   }
1979   .app-shell {
1980     width: 100% !important;
1981   }
1982   .main-content {
1983     padding: 0 !important;
1984     max-width: none;
1985   }
1986   .audit-report {
1987     border: 0;
1988     padding: 0;
1989     margin: 0;
1990   }
1991   .audit-report th,
1992   .audit-report td {
1993     font-size: 10px;
1994     color: #333;
1995   }
1996   .audit-report p {
1997     color: #333;
1998   }
1999   .audit-report tr {
2000     break-inside: avoid;

```

app/globals.css (continued)

```
2001  }
2002  .audit-report section {
2003    break-before: auto;
2004  }
2005  .audit-report h2 {
2006    break-after: avoid;
2007  }
2008  .report-notice {
2009    border: 1px solid #999;
2010    background: none;
2011  }
2012  @page {
2013    size: A4;
2014    margin: 14mm;
2015  }
2016 }
2017
2018 .orange-text{color:#ad5a32}.status-
tag{color:#994a2d}.pill.exact{color:#944f2c}.pill.missing,.pill.possible{color:#806b24}
2019 .review-tabs [data-slot='tabs-trigger']{color:#58694b}
2020
2021 .auth-form { display: grid; gap: 1rem; }
2022 .auth-form label { display: grid; gap: .45rem; font-size: .9rem; }
2023 .auth-form input { border: 1px solid #c8d1c2; border-radius: .5rem; padding: .8rem; background: white; font-
size: 1rem; width: 100%; }
2024 .auth-links { display: flex; justify-content: space-between; gap: 1rem; flex-wrap: wrap; }
2025 .auth-error { color: #913c24; font-size: .9rem; }
2026 .auth-form .btn-primary { padding: .8rem; border-radius: .5rem; background: #294430; color: white; }
```

app/layout.tsx

```
1 import type { Metadata } from "next";
2 import "../globals.css";
3
4 export const metadata: Metadata = {
5   title: "Lotlight | Recall to resolution",
6   description:
7     "Evidence-backed medical recall review. Connect the notice to your inventory and document the response.
      A project by Shivam Gupta.",
8   icons: {
9     icon: "/favicon.svg",
10    shortcut: "/favicon.svg",
11  },
12 };
13
14 export default function RootLayout({
15   children,
16 }: Readonly<{
17   children: React.ReactNode;
18 }>) {
19   return (
20     <html lang="en">
21       <body className="antialiased">{children}</body>
22     </html>
23   );
24 }
```

app/lotlight.tsx

```
1 "use client";
2 import { useCallback, useEffect, useMemo, useRef, useState } from "react";
3 import {
4   ArrowUpRight,
5   ArrowRight,
6   ScanLine,
7   Layers,
8   ClipboardCheck,
9   BookOpen,
10  Plus,
11  ShieldCheck,
12  Package,
13  FileText,
14  ChevronRight,
15  LogIn,
16  Download,
17  Check,
18  AlertTriangle,
19  Search,
20  RotateCcw,
21  Loader2,
22  Sparkles,
23  X,
24  ExternalLink,
25  ArrowLeft,
26  CheckCircle2,
27  Clock,
28  MoreHorizontal,
29  Upload,
30  Trash2,
31 } from "lucide-react";
32 import {
33   Sidebar,
34   SidebarProvider,
35   SidebarContent,
36   SidebarHeader,
37   SidebarFooter,
38   SidebarMenu,
39   SidebarMenuItem,
40   SidebarMenuButton,
41   SidebarTrigger,
42 } from "@/components/ui/sidebar";
43 import {
44   Dialog,
45   DialogContent,
46   DialogTitle,
47   DialogDescription,
48   DialogHeader,
49   DialogFooter,
50 } from "@/components/ui/dialog";
51 import { Tabs, TabsList, TabsTrigger, TabsContent } from "@/components/ui/tabs";
52 import {
53   Table,
54   TableBody,
55   TableCell,
56   TableHead,
57   TableHeader,
58   TableRow,
59 } from "@/components/ui/table";
60 import {
61   Select,
62   SelectContent,
63   SelectItem,
64   SelectTrigger,
65   SelectValue,
66 } from "@/components/ui/select";
67 import { Checkbox } from "@/components/ui/checkbox";
68 import {
69   AlertDialog,
70   AlertDialogContent,
71   AlertDialogTitle,
72   AlertDialogDescription,
73   AlertDialogFooter,
74   AlertDialogCancel,
75   AlertDialogAction,
76 } from "@/components/ui/alert-dialog";
77 import { Toaster, toast } from "sonner";
78 import {
79   initialWorkspace,
80   SAMPLE_CSV,
```

app/lotlight.tsx (continued)

```

81  SAMPLE_NOTICE_TEXT,
82  BAXTER_URL,
83 } from "@lib/lotlight/sample";
84 import {
85   activeAction,
86   reduceWorkspace,
87   type Command,
88 } from "@lib/lotlight/commands";
89 import {
90   reconcile,
91   extractScope,
92   parseCsv,
93   validateScope,
94 } from "@lib/lotlight/engine";
95 import {
96   download,
97   exportAudit,
98   exportMatches,
99   readNoticeFile,
100 } from "@lib/lotlight/files";
101 import type {
102   Workspace,
103   Notice,
104   Match,
105   Scope,
106   UserInfo,
107   ResponseAction,
108   InventoryItem,
109 } from "@lib/lotlight/types";
110
111 type View =
112   | "overview"
113   | "inventory"
114   | "actions"
115   | "about"
116   | "review"
117   | "report";
118 const NAV = [
119   { icon: Layers, label: "Recall workspace", view: "overview" },
120   { icon: Package, label: "Inventory", view: "inventory" },
121   { icon: ClipboardCheck, label: "Action log", view: "actions" },
122   { icon: BookOpen, label: "Evidence & approach", view: "about" },
123 ] as const;
124 const LABELS = {
125   exact: "Identifier match",
126   missing: "Needs information",
127   possible: "AI candidate",
128   unlisted: "No listed match",
129 };
130 const date = (s: string) =>
131   new Date(s).toLocaleDateString("en-GB", {
132     day: "2-digit",
133     month: "short",
134     year: "numeric",
135     timeZone: "UTC",
136   });
137 type WorkspaceRequest = (
138   input: string,
139   init?: RequestInit,
140 ) => Promise<Response>;
141 const defaultRequest: WorkspaceRequest = (input, init) => fetch(input, init);
142 export default function Lotlight({
143   user,
144   request = defaultRequest,
145   onAuth,
146 }: {
147   user: UserInfo;
148   request?: WorkspaceRequest;
149   onAuth?: () => void;
150 }) {
151   const [hydrated, setHydrated] = useState(false);
152   useEffect(() => {
153     setHydrated(true);
154   }, []);
155   const [noticeNonce, setNoticeNonce] = useState(0);
156   const [correcting, setCorrecting] = useState<InventoryItem | null>(null);
157   const [state, setState] = useState<Workspace>(initialWorkspace);
158   const [view, setView] = useState<View>("overview");
159   const [loading, setLoading] = useState(!user);
160   const [hasLoaded, setHasLoaded] = useState(!user);

```

app/lotlight.tsx (continued)

```

161 const [busy, setBusy] = useState(false);
162 const busyRef = useRef(false);
163 const [saveError, setSaveError] = useState("");
164 const [noticeId, setNoticeId] = useState("baxter-demo");
165 const [noticeDialog, setNoticeDialog] = useState(false);
166 const [editing, setEditing] = useState<Notice | undefined>();
167 const [inventoryDialog, setInventoryDialog] = useState(false);
168 const [actionMatch, setActionMatch] = useState<Match | null>(null);
169 const [exportOpen, setExportOpen] = useState(false);
170 const [reset, setReset] = useState<"reset_demo" | "clear_workspace" | null>(
171   null,
172 );
173 const [scores, setScores] = useState<Record<string, number>>({});
174 const [aiStatus, setAiStatus] = useState("idle");
175 const [aiMessage, setAiMessage] = useState("");
176 const worker = useRef<Worker | null>(null);
177 const [query, setQuery] = useState("");
178 const notice =
179   state.notices.find((n) => n.id === noticeId) ?? state.notices[0];
180 const matches = useMemo(
181   () => (notice ? reconcile(state.inventory, notice, scores) : []),
182   [state.inventory, notice, scores],
183 );
184 const attention = matches.filter((m) => m.kind !== "unlisted");
185 const exact = matches.filter((m) => m.kind === "exact");
186 const uncertain = matches.filter(
187   (m) => m.kind === "missing" || m.kind === "possible",
188 );
189 const completed = notice
190   ? attention.filter(
191     (m) =>
192       activeAction(state, notice.id, m.item.id)?.status === "completed",
193   ).length
194   : 0;
195 const load = useCallback(async () => {
196   if (!user) return;
197   setLoading(true);
198   try {
199     const res = await request("/api/workspace", { cache: "no-store" });
200     const data = (await res.json()) as Workspace & { error: string };
201     if (!res.ok) throw new Error(data.error);
202     setState(data);
203     setHasLoaded(true);
204     setSaveError("");
205   } catch (error) {
206     setSaveError(
207       error instanceof Error ? error.message : "Could not load workspace.",
208     );
209   } finally {
210     setLoading(false);
211   }
212 }, [user, request]);
213 useEffect(() => {
214   void load();
215 }, [load]);
216 useEffect(() => {
217   setScores({});
218   setAiStatus("idle");
219   worker.current?.terminate();
220   worker.current = null;
221   return () => {
222     worker.current?.terminate();
223   };
224 }, [notice?.id, notice?.version, state.inventoryVersion]);
225 const send = async (command: Command): Promise<boolean> => {
226   if (busyRef.current || loading) return false;
227   busyRef.current = true;
228   setBusy(true);
229   try {
230     if (user) {
231       if (saveError)
232         throw new Error("Reload your saved workspace before making changes.");
233       const res = await request("/api/workspace", {
234         method: "POST",
235         headers: { "Content-Type": "application/json" },
236         body: JSON.stringify({ revision: state.revision, command }),
237       });
238       const data = (await res.json()) as Workspace & { error: string };
239       if (!res.ok) {
240         if (res.status === 409) setSaveError(data.error);

```

app/lotlight.tsx (continued)

```

241         throw new Error(data.error);
242     }
243     setState(data);
244 } else
245     setState(reduceWorkspace(state, command, "Demo operator (unsaved)"));
246 toast.success(
247     user ? "Saved to your workspace" : "Updated this unsaved demo",
248 );
249 return true;
250 } catch (error) {
251     toast.error(
252         error instanceof Error
253         ? error.message
254         : "Could not save. Your inputs are preserved.",
255     );
256     return false;
257 } finally {
258     busyRef.current = false;
259     setBusy(false);
260 }
261 };
262 function openReview(n: Notice) {
263     setNoticeId(n.id);
264     setView("review");
265     setQuery("");
266 }
267 function runAI() {
268     if (!notice || !state.inventory.length || aiStatus === "loading") return;
269     setAiStatus("loading");
270     setAiMessage(
271         "Loading the local model. First use downloads about 23 MB of model weights.",
272     );
273     try {
274         worker.current?.terminate();
275         const current = new Worker("/ai/semantic-worker.js", { type: "module" });
276         worker.current = current;
277         current.onmessage = (e) => {
278             if (e.data.type === "progress") setAiMessage(e.data.message);
279             if (e.data.type === "result") {
280                 setScores(e.data.scores);
281                 setAiStatus("ready");
282                 setAiMessage(
283                     "Semantic comparison complete. Similarity ranks candidates; it cannot establish a recall
match.",
284                 );
285             }
286             if (e.data.type === "error") {
287                 setAiStatus("error");
288                 setAiMessage(
289                     "Local AI could not load. Identifier checks still work. Check your connection and retry.",
290                 );
291                 current.terminate();
292             }
293         };
294         current.onerror = (event) => {
295             console.error(
296                 "AI worker error",
297                 event.message,
298                 event.filename,
299                 event.lineno,
300             );
301             setAiStatus("error");
302             setAiMessage(
303                 "Local AI is unavailable in this browser. Identifier checks still work.",
304             );
305             current.terminate();
306         };
307         current.postMessage({
308             query: `${notice.manufacturer} ${notice.text.match(/^Product:\s*(.+)$\/m)?.[1] ?? notice.title}`,
309             items: state.inventory.map((i) => ({ id: i.id, product: i.product })),
310         });
311     } catch (error) {
312         console.error("AI worker initialization", error);
313         setAiStatus("error");
314         setAiMessage(
315             "This browser could not start the local AI worker. Identifier checks remain available.",
316         );
317     }
318 }
319 useEffect(() => {

```

app/lotlight.tsx (continued)

```

320     const context = (
321       document as Document & {
322         modelContext?: {
323           registerTool: (tool: unknown, options: unknown) => void;
324         };
325       }
326     ).modelContext;
327     if (!context) return;
328     const lifecycle = new AbortController();
329     try {
330       context.registerTool(
331         {
332           name: "lotlight_read_review",
333           title: "Read recall review",
334           description:
335             "Read the current inventory review, source approval, and unresolved lines. Does not change
records.",
336           inputSchema: {
337             type: "object",
338             properties: {},
339             additionalProperties: false,
340           },
341           annotations: { readOnlyHint: true, untrustedContentHint: true },
342           execute(input: unknown) {
343             if (
344               !input ||
345               typeof input !== "object" ||
346               Object.keys(input).length
347             )
348               throw new Error("Expected an empty object.");
349             return {
350               notice: notice?.title ?? null,
351               sourceApproved: !!notice?.approvedAt,
352               inventoryLines: state.inventory.length,
353               matches: matches.map((m) => ({
354                 product: m.item.product,
355                 catalog: m.item.catalog,
356                 lot: m.item.lot,
357                 kind: m.kind,
358                 reason: m.reason,
359               })),
360               unsavedDemo: !user,
361             };
362           },
363         },
364         { signal: lifecycle.signal },
365       );
366     } catch {
367       /* Optional browser capability. */
368     }
369     return () => lifecycle.abort();
370   }, [notice, state.inventory, matches, user]);
371   const isTraining = state.notices.some((n) => n.training);
372   const navTitle = {
373     overview: "Recall review",
374     inventory: "Inventory",
375     actions: "Action log",
376     about: "Evidence & approach",
377     review: "Recall review",
378     report: "Audit report",
379   }[view];
380   return (
381     <SidebarProvider
382       style={{ "--sidebar-width": "224px" } as React.CSSProperties}
383     >
384       <Toaster richColors position="bottom-right" />
385       <Sidebar className="lot-sidebar">
386         <SidebarHeader>
387           <a className="brand" href="/">
388             <span className="brand-mark">
389               <ScanLine size={23} />
390             </span>
391             lotlight<span className="brand-dot">\u2733</span>
392           </a>
393           <div className="workspace-label">
394             <span className="workspace-avatar">{user ? "P" : "W"}</span>
395             <div>
396               {user ? "Personal workspace" : "Willow Community Clinic"}
397             <small>
398               {user

```

app/lotlight.tsx (continued)

```

399         ? "Private account records"
400         : "Fictional demonstration clinic"}
401     </small>
402 </div>
403 </div>
404 </SidebarHeader>
405 <SidebarContent>
406   <span className="nav-caption">WORKSPACE</span>
407   <SidebarMenu>
408     {NAV.map(({ icon: Icon, label, view: target }) => (
409       <SidebarMenuItem key={target}>
410         <SidebarMenuButton
411           onClick={() => {
412             setView(target);
413             setQuery("");
414           }}
415           isActive={
416             view === target ||
417             (view === "review" && target === "overview")
418           }
419           className={
420             view === target ||
421             (view === "review" && target === "overview")
422             ? "nav-active"
423             : ""
424           }
425         >
426           <Icon size={18} />
427           <span>{label}</span>
428           {target === "overview" && (
429             <span className="nav-count">{state.notices.length}</span>
430           )}
431         </SidebarMenuButton>
432       </SidebarMenuItem>
433     ))}
434 </SidebarMenu>
435 <div className="side-note">
436   <ShieldCheck size={20} />
437   <strong>
438     Human judgment.
439     <br />A better paper trail.
440   </strong>
441   <p>Every match shows its evidence. Every action has an owner.</p>
442 </div>
443 </SidebarContent>
444 <SidebarFooter>
445   <div className="creator">
446     <span>SG</span>
447     <div>
448       Shivam Gupta<small>Creator \xb7 DSH Hacks V2</small>
449     </div>
450   </div>
451 </SidebarFooter>
452 </Sidebar>
453 <div className="app-shell">
454   <header className="topbar">
455     <div>
456       <SidebarTrigger className="mobile-menu" />
457       <span>Workspace</span>
458       <ChevronRight size={14} />
459       <b>{navTitle}</b>
460     </div>
461     <div className="account-area">
462       <span className="save-status">
463         {loading
464           ? "Loading\u2026"
465           : busy
466             ? "Saving\u2026"
467             : saveError
468               ? "Save unavailable"
469               : user
470                 ? "Account storage"
471                 : "Demo \xb7 not saved"}
472       </span>
473       <a
474         className="top-account"
475         onClick={
476           onAuth
477             ? (e) => {
478               e.preventDefault();

```

app/lotlight.tsx (continued)

```

479         onAuth();
480     }
481     : undefined
482 }
483 href={
484     onAuth
485     ? "#account"
486     : user
487     ? "/signout-with-chatgpt?return_to=%2F"
488     : "/signin-with-chatgpt?return_to=%2F"
489 }
490 title={
491     user ? user.email : "Demo changes will not transfer on sign-in"
492 }
493 >
494 <LogIn size={15} />
495 {user ? "Sign out" : "Sign in to save"}
496 </a>
497 </div>
498 </header>
499 <main className="main-content" data-ready={hydrated} inert={!hydrated}>
500     {saveError && (
501         <div className="error-banner" role="alert">
502             <AlertTriangle size={18} />
503             <span>{saveError}</span>
504             <button className="text-button" onClick={() => void load()}>
505                 Reload saved version
506             </button>
507         </div>
508     )}
509     {user && !loading && !hasLoaded ? (
510         <div className="loading-state">
511             <AlertTriangle />
512             <h1>Your saved workspace could not open</h1>
513             <p>
514                 Retry using the button above. Your account records have not been
515                 replaced.
516             </p>
517         </div>
518     ) : loading ? (
519         <div className="loading-state">
520             <Loader2 className="spin" />
521             <h1>Opening your workspace</h1>
522             <p>Loading your saved inventory and source reviews.</p>
523         </div>
524     ) : (
525         <>
526             {view === "overview" && (
527                 <>
528                     <div className="page-heading">
529                         <div>
530                             <p className="eyebrow">
531                                 <span />
532                                 THE LAST MILE OF RECALL RESPONSE
533                             </p>
534                             <h1>
535                                 A notice is only
536                                 <br />
537                                 the beginning.
538                             </h1>
539                             <p className="intro">
540                                 Find what\u2019s on your shelves. See why it matches.
541                                 <br />
542                                 Give every response a clear next step.
543                             </p>
544                         </div>
545                         <div className="heading-actions">
546                             <span className="demo-label">
547                                 {isTraining
548                                 ? "HISTORICAL NOTICE \xb7 SYNTHETIC STOCK"
549                                 : "PRIVATE REVIEW WORKSPACE"}
550                             </span>
551                             <button
552                                 className="button primary"
553                                 onClick={() => {
554                                     setEditing(undefined);
555                                     setNoticeDialog(true);
556                                 }}
557                             >
558                                 <Plus size={17} /> New recall review

```

app/lotlight.tsx (continued)

```

559         </button>
560     </div>
561 </div>
562 <div className="overview-strip">
563     <div>
564         <span>Inventory lines</span>
565         <strong>
566             {state.inventory.length}
567             <small>
568                 across{" "}
569                 {new Set(state.inventory.map((i) => i.location)).size}{" "}
570                 locations
571             </small>
572         </strong>
573     </div>
574     <div>
575         <span>Identifier matches</span>
576         <strong className="orange-text">
577             {exact.length}
578             <small>in the selected notice</small>
579         </strong>
580     </div>
581     <div>
582         <span>Needs more evidence</span>
583         <strong>
584             {uncertain.length}
585             <small>never silently cleared</small>
586         </strong>
587     </div>
588     <div>
589         <span>Responses completed</span>
590         <strong>
591             {completed}
592             <small>with staff-recorded evidence</small>
593         </strong>
594     </div>
595 </div>
596 <div className="section-heading">
597     <h2>
598         Recall workspace{" "}
599         <span>
600             {String(state.notices.length).padStart(2, "0")}
601         </span>
602     </h2>
603     <span>
604         {notice
605             ? "Summary for " + notice.title
606             : "Ready for your first notice"}
607     </span>
608 </div>
609 <div className="review-grid">
610     <div className="notice-stack">
611         {state.notices.length === 0 ? (
612             <Empty
613                 title="Start with the notice"
614                 text="Add the original text and paired identifiers, then import a stock
spreadsheet."
615                 action={
616                     <button
617                         className="button primary"
618                         onClick={() => {
619                             setEditing(undefined);
620                             setNoticeDialog(true);
621                         }}
622                     >
623                         Create a review <Plus size={15} />
624                     </button>
625                 >
626             </div>
627         ) : (
628             state.notices.map((n) => {
629                 const m = reconcile(state.inventory, n);
630                 const count = m.filter(
631                     (x) => x.kind !== "unlisted",
632                 ).length;
633                 return (
634                     <section className="review-card" key={n.id}>
635                         <div className="card-topline">
636                             <span
637                                 className={

```

app/lotlight.tsx (continued)

```

638         "status-tag " +
639         (n.approvedAt ? "approved" : "")
640     }
641 >
642     <span />
643     {n.approvedAt
644     ? "SOURCE REVIEWED"
645     : "SOURCE REVIEW REQUIRED"}
646 </span>
647 <span className="mono">
648     {n.training
649     ? "FA-2025-039 \xb7 TRAINING"
650     : `VERSION ${n.version}`}
651 </span>
652 </div>
653 <h2>{n.title}</h2>
654 <p>
655     {n.manufacturer} \xb7 Notice dated {date(n.date)}
656     {n.training
657     ? " \xb7 Two-pair training extract"
658     : ""}
659 </p>
660 <div className="scope-line">
661     <span>Reviewed scope</span>
662     {n.scope.slice(0, 3).map((s, i) => (
663         <code key={i}>
664             {s.catalog} /{" "}
665             {s.allLots ? "all lots" : s.lots.join(", ")}
666         </code>
667     ))}
668 </div>
669 <div className="match-visual">
670     <div className="match-bars">
671         {m.slice(0, 24).map((x) => (
672             <span
673                 key={x.item.id}
674                 style={{
675                     background:
676                         x.kind === "exact"
677                         ? "#c9865a"
678                         : x.kind === "unlisted"
679                         ? "#dfe5d4"
680                         : "#d9ca80",
681                 }}
682             />
683         ))}
684     </div>
685     <span>
686         <b>{m.length}</b> inventory lines checked{" "}
687         <span>\xb7</span> <b>{count}</b> need attention
688     </span>
689 </div>
690 <div className="card-bottom">
691     <span>
692         <FileText size={16} />{" "}
693         {n.approvedAt
694         ? "Staff-attested source review"
695         : "Source-linked decisions"}
696     </span>
697     <button
698         className="text-button"
699         onClick={() => openReview(n)}
700     >
701         Open review <ArrowRight size={16} />
702     </button>
703 </div>
704 </section>
705 );
706 })
707 })
708 </div>
709 <aside className="principle-card">
710     <div className="mini-orbit">
711         <ScanLine size={25} />
712         <span />
713         <span />
714     </div>
715     <p className="eyebrow">BUILT FOR THE IN-BETWEEN</p>
716     <h3>
717         Published doesn\u2019t

```

app/lotlight.tsx (continued)

```

718         <br />
719         mean resolved.
720     </h3>
721     <p>
722         Connect the notice to the shelf, and the shelf to a
723         documented response.
724     </p>
725     <button
726         className="principle-footer"
727         onClick={() => setView("about")}
728     >
729         EVIDENCE BEFORE ACTION <ArrowUpRight size={17} />
730     </button>
731 </aside>
732 </div>
733 <div className="bottom-explainer">
734     <div>
735         <span className="step-number">01</span>
736         <h3>Bring the notice</h3>
737         <p>Preserve the original wording and scope.</p>
738     </div>
739     <div>
740         <span className="step-number">02</span>
741         <h3>Reconcile the shelf</h3>
742         <p>Exact identifiers first. AI helps find aliases.</p>
743     </div>
744     <div>
745         <span className="step-number">03</span>
746         <h3>Document the response</h3>
747         <p>Assign, verify, and record completion.</p>
748     </div>
749 </div>
750 <div className="workspace-tools">
751     <button
752         className="text-button muted"
753         onClick={() => setReset("reset_demo")}
754     >
755         <RotateCcw size={13} /> Restart demo
756     </button>
757     <button
758         className="text-button muted"
759         onClick={() => setReset("clear_workspace")}
760     >
761         <Plus size={13} /> Start empty workspace
762     </button>
763 </div>
764 </>
765 })
766 {view === "review" && notice && (
767     <>
768         <button
769             className="back-button"
770             onClick={() => setView("overview")}
771         >
772             <ArrowLeft size={14} /> Recall workspace
773         </button>
774         <div className="page-heading compact">
775             <div>
776                 <p className="eyebrow">
777                     {notice.training
778                     ? "HISTORICAL TRAINING EXTRACT"
779                     : "SOURCE REVIEW"}{" "}
780                     \xb7 VERSION {notice.version}
781                 </p>
782                 <h1>{notice.title}</h1>
783                 <p className="intro">
784                     {notice.manufacturer} \xb7 Issued {date(notice.date)} \xb7{" "}
785                     {notice.scope.length} catalog/lot groups
786                 </p>
787             </div>
788             <button
789                 className="button"
790                 onClick={() => setExportOpen(true)}
791             >
792                 <Download size={15} /> Export record
793             </button>
794         </div>
795         <div className="review-disclaimer">
796             <ShieldCheck size={16} />
797             <span>

```

app/lotlight.tsx (continued)

```

798         {notice.training
799           ? "This historical example contains only two catalog/lot pairs. Verify the full,
current notice before any real-world action."
800           : "This review covers only the source and identifiers entered here. Verify the
complete current manufacturer instructions before acting."}{ " "}
801         Inventory: {state.inventoryProvenance ?? "synthetic"}.
802       </span>
803     </div>
804     <Tabs defaultValue="inventory" className="review-tabs">
805       <TabsList variant="line">
806         <TabsTrigger value="inventory">
807           Inventory review{" "}
808           <span className="tab-count">{matches.length}</span>
809         </TabsTrigger>
810         <TabsTrigger value="source">Source & scope</TabsTrigger>
811         <TabsTrigger value="response">
812           Response plan{" "}
813           <span className="tab-count">{attention.length}</span>
814         </TabsTrigger>
815       </TabsList>
816       <TabsContent value="inventory">
817         <div className="ai-banner">
818           <div className="ai-icon">
819             <Sparkles size={20} />
820           </div>
821           <div>
822             <strong>Find the product behind the wording.</strong>
823             <p>
824               {aiStatus === "idle"
825                 ? "Local AI compares descriptions like \u201cIV line extender\u201d and
\u201cextension set.\u201d Identifiers always decide the match."
826                 : aiMessage}
827             </p>
828           </div>
829           <button
830             className="button"
831             disabled={
832               aiStatus === "loading" || !state.inventory.length
833             }
834             onClick={runAI}
835           >
836             {aiStatus === "loading" ? (
837               <Loader2 className="spin" size={15} />
838             ) : (
839               <Sparkles size={15} />
840             )}{ " "}
841             {aiStatus === "ready"
842               ? "Run again"
843               : aiStatus === "error"
844                 ? "Retry AI"
845                 : "Run local AI"}
846           </button>
847           {aiStatus === "loading" && (
848             <button
849               aria-label="Cancel AI"
850               className="icon-button"
851               onClick={() => {
852                 worker.current?.terminate();
853                 setAiStatus("idle");
854                 setAiMessage("");
855               }}
856             >
857               <X size={17} />
858             </button>
859           )}
860         </div>
861         <div className="table-toolbar">
862           <div className="legend">
863             <span>
864               <i className="dot-exact" />
865               {exact.length} identifier matches
866             </span>
867             <span>
868               <i className="dot-missing" />
869               {uncertain.length} need review
870             </span>
871             <span>
872               <i className="dot-unlisted" />
873               {matches.length - attention.length} not listed
874             </span>

```

app/lotlight.tsx (continued)

```

875         </div>
876         <label className="search-box">
877           <Search size={15} />
878           <input
879             placeholder="Search inventory\u2026"
880             aria-label="Search inventory"
881             value={query}
882             onChange={(e) => setQuery(e.target.value)}
883           />
884         </label>
885       </div>
886       <div className="inventory-review-list">
887         {matches
888           .filter((m) =>
889             [
890               m.item.product,
891               m.item.catalog,
892               m.item.lot,
893               m.item.location,
894             ]
895               .join(" ")
896               .toLowerCase()
897               .includes(query.toLowerCase()),
898           )
899           .map((m) => (
900             <MatchRow
901               key={m.item.id}
902               match={m}
903               action={activeAction(state, notice.id, m.item.id)}
904               onAction={() => setActionMatch(m)}
905               sourceApproved={!notice.approvedAt}
906             />
907           ))}
908         {!matches.length && (
909           <Empty
910             title="Your shelves are next"
911             text="Import a CSV to reconcile stock against this source."
912             action={
913               <button
914                 className="button"
915                 onClick={() => setInventoryDialog(true)}
916               >
917                 Import inventory
918               </button>
919             }
920           />
921         )}
922       </div>
923       <p className="fine-print">
924         AI similarity is not a probability of recall. A missing
925         or unlisted identifier never establishes safety. Model:
926         MiniLM-L6-v2, quantized, running on this device.
927       </p>
928     </TabsContent>
929     <TabsContent value="source">
930       <SourceReview
931         notice={notice}
932         busy={busy}
933         onApprove={() =>
934           send({
935             type: "approve_notice",
936             noticeId: notice.id,
937             sourceConfirmed: true,
938           })
939         }
940         onEdit={() => {
941           setEditing(notice);
942           setNoticeDialog(true);
943         }}
944       />
945     </TabsContent>
946     <TabsContent value="response">
947       <div className="section-heading response-heading">
948         <div>
949           <h2>Every open line gets a next step</h2>
950           <p className="fine-print">
951             {completed} of {attention.length} responses
952             complete. Missing identifiers stay open until
953             corrected in inventory.
954           </p>

```

app/lotlight.tsx (continued)

```

955         </div>
956         {!notice.approvedAt && (
957           <span className="status-tag">
958             Review the source first
959           </span>
960         )}
961       </div>
962       {attention.map((m) => (
963         <MatchRow
964           key={m.item.id}
965           match={m}
966           action={activeAction(state, notice.id, m.item.id)}
967           onAction={() => setActionMatch(m)}
968           sourceApproved={!notice.approvedAt}
969         />
970       ))}
971       {!attention.length && (
972         <Empty
973           title="No listed identifier matches"
974           text="This result covers only this reviewed scope. It is not a safety
certification."
975         />
976       )}
977     </TabsContent>
978   </Tabs>
979 </>
980 )}
981 {view === "inventory" && (
982   <>
983     <div className="page-heading compact">
984       <div>
985         <p className="eyebrow">
986           THE STOCKROOM, WITHOUT THE GUESSWORK
987         </p>
988         <h1>Your inventory.</h1>
989         <p className="intro">
990           {state.inventoryProvenance ?? "synthetic"} \xb7{" "}
991           {state.inventory.length} lines \xb7{" "}
992           {state.inventory.reduce((n, i) => n + i.quantity, 0)}{" "}
993           units \xb7 Inventory revision {state.inventoryVersion}
994         </p>
995       </div>
996       <button
997         className="button primary"
998         onClick={() => setInventoryDialog(true)}
999       >
1000         <Upload size={16} /> Import CSV
1001       </button>
1002     </div>
1003     <div className="info-banner">
1004       <Package size={17} />
1005       <span>
1006         Missing catalog or lot numbers are preserved and flagged.
1007         Replacing inventory opens a new review version and
1008         archives previous response actions.
1009       </span>
1010     </div>
1011     <div className="data-table">
1012       <Table>
1013         <TableHeader>
1014           <TableRow>
1015             {[
1016               "Product",
1017               "Manufacturer",
1018               "Catalog",
1019               "Lot",
1020               "Quantity",
1021               "Location",
1022               "Review",
1023             ].map((h) => (
1024               <TableHead key={h}>{h}</TableHead>
1025             ))}
1026           </TableRow>
1027         </TableHeader>
1028         <TableBody>
1029           {state.inventory.map((i) => (
1030             <TableRow key={i.id}>
1031               <TableCell className="product-cell">
1032                 {i.product}
1033               </TableCell>

```

app/lotlight.tsx (continued)

```

1034         <TableCell>
1035             {i.manufacturer || (
1036                 <span className="missing-text">Missing</span>
1037             )}
1038         </TableCell>
1039         <TableCell>
1040             <code>{i.catalog || "Missing"}</code>
1041         </TableCell>
1042         <TableCell>
1043             <code className={!i.lot ? "missing-text" : ""}>
1044                 {i.lot || "Missing"}
1045             </code>
1046         </TableCell>
1047         <TableCell>{i.quantity}</TableCell>
1048         <TableCell>{i.location}</TableCell>
1049         <TableCell>
1050             <button
1051                 className="text-button"
1052                 onClick={() => setCorrecting(i)}
1053                 aria-label={`Correct ${i.product} in ${i.location}`}
1054             >
1055                 Correct <ArrowRight size={12} />
1056             </button>
1057         </TableCell>
1058     </TableRow>
1059 ))}
1060 </TableBody>
1061 </Table>
1062 </div>
1063 {!state.inventory.length && (
1064     <Empty
1065         title="Nothing on the shelf yet"
1066         text="Download the template, add your inventory, and import it."
1067     />
1068 )}
1069 <button
1070     className="text-button download-template"
1071     onClick={() =>
1072         download(
1073             "lotlight-inventory-template.csv",
1074             SAMPLE_CSV,
1075             "text/csv",
1076         )
1077     }
1078 >
1079     <Download size={14} /> Download synthetic example CSV
1080 </button>
1081 </>
1082 )}
1083 {view === "actions" && (
1084     <>
1085         <div className="page-heading compact">
1086             <div>
1087                 <p className="eyebrow">ACCOUNTABILITY YOU CAN FOLLOW</p>
1088                 <h1>The response record.</h1>
1089                 <p className="intro">
1090                     Staff-recorded activity, with source and inventory
1091                     versions.
1092                 </p>
1093             </div>
1094             <button
1095                 className="button"
1096                 onClick={() => setExportOpen(true)}
1097             >
1098                 <Download size={15} /> Export record
1099             </button>
1100         </div>
1101         <div className="action-cards">
1102             {state.actions.map((a) => {
1103                 const n = state.notices.find((n) => n.id === a.noticeId);
1104                 const historic =
1105                     activeAction(state, a.noticeId, a.itemId)?.id !== a.id;
1106                 return (
1107                     <div key={a.id} className="action-card">
1108                         <div>
1109                             <span
1110                                 className={
1111                                     "pill " +
1112                                     (historic
1113                                         ? "unlisted"

```

app/lotlight.tsx (continued)

```

1114             : a.status === "completed"
1115             ? "complete"
1116             : "missing")
1117         }
1118     >
1119     {historic ? "Historical version" : a.status}
1120 </span>
1121 <h3>{a.itemLabel}</h3>
1122 <p>
1123     {a.owner} \xb7 Due {date(a.dueDate)} \xb7{" "}
1124     {a.handledQuantity} units handled
1125 </p>
1126 <p>{a.evidence || "Response evidence pending"}</p>
1127 </div>
1128 <span className="mono">{date(a.updatedAt)}</span>
1129 </div>
1130 );
1131 }}}
1132 </div>
1133 {!state.actions.length && (
1134     <Empty
1135         title="The paper trail starts here"
1136         text="Review a notice\u2019s source, then assign a response to an inventory line."
1137         action={
1138             <button
1139                 className="button"
1140                 onClick={() => notice && openReview(notice)}
1141             >
1142                 Open recall review <ArrowRight size={15} />
1143             </button>
1144         }
1145     />
1146 )}
1147 <h2 className="activity-title">Activity timeline</h2>
1148 <div className="timeline">
1149     {[...state.audit].reverse().map((e) => (
1150         <div key={e.id} className="timeline-item">
1151             <span className="timeline-dot" />
1152             <div>
1153                 <strong>{e.type.replace(/[_\.]/g, " ")}</strong>
1154                 <p>{e.detail}</p>
1155                 <small>
1156                     {e.actor} \xb7 {new Date(e.at).toLocaleString()}
1157                 </small>
1158             </div>
1159         </div>
1160     )]}
1161 </div>
1162 </>
1163 )}
1164 {view === "about" && <About />}
1165 {view === "report" && (
1166     <>
1167         <div className="report-toolbar">
1168             <button
1169                 className="text-button"
1170                 onClick={() => setView("overview")}
1171             >
1172                 <ArrowLeft size={14} /> Workspace
1173             </button>
1174             <button
1175                 className="button primary"
1176                 onClick={() => window.print()}
1177             >
1178                 <Download size={15} /> Print / save PDF
1179             </button>
1180         </div>
1181         <AuditReport state={state} />
1182     </>
1183 )}
1184 <footer className="page-footer">
1185     <span>
1186         <ShieldCheck size={14} /> Decision support. Verify current
1187         manufacturer instructions before acting.
1188     </span>
1189     <span>Built with care, by Shivam Gupta</span>
1190 </footer>
1191 </>
1192 )}
1193 </main>

```

app/lotlight.tsx (continued)

```

1194     </div>
1195     {correcting && (
1196       <CorrectionDialog
1197         item={correcting}
1198         busy={busy}
1199         onClose={() => setCorrecting(null)}
1200         onSave={async (command) => {
1201           if (await send(command)) setCorrecting(null);
1202         }}
1203       />
1204     )}
1205     <NoticeDialog
1206       key={editing?.id ?? `new-${noticeNonce}`}
1207       open={noticeDialog}
1208       onOpenChange={setNoticeDialog}
1209       existing={editing}
1210       busy={busy}
1211       onSave={async (n) => {
1212         if (await send({ type: "save_notice", notice: n })) {
1213           setNoticeDialog(false);
1214           setNoticeNonce((x) => x + 1);
1215           setView("overview");
1216         }
1217       }}
1218     />
1219     <InventoryDialog
1220       open={inventoryDialog}
1221       onOpenChange={setInventoryDialog}
1222       busy={busy}
1223       onSave={async (csv, synthetic) => {
1224         if (await send({ type: "import_inventory", csv, synthetic })) {
1225           setInventoryDialog(false);
1226         }
1227       }}
1228     />
1229     {actionMatch && notice && (
1230       <ActionDialog
1231         match={actionMatch}
1232         notice={notice}
1233         action={activeAction(state, notice.id, actionMatch.item.id)}
1234         busy={busy}
1235         onClose={() => setActionMatch(null)}
1236         onSave={async (command) => {
1237           if (await send(command)) setActionMatch(null);
1238         }}
1239       />
1240     )}
1241     <Dialog open={exportOpen} onOpenChange={setExportOpen}>
1242       <DialogContent>
1243         <DialogHeader>
1244           <DialogTitle>Take the evidence with you.</DialogTitle>
1245           <DialogDescription>
1246             Exports include source scope, unresolved lines, and staff-recorded
1247             actions. They do not certify device safety.
1248           </DialogDescription>
1249         </DialogHeader>
1250         <div className="export-options">
1251           <button
1252             onClick={() => {
1253               void exportAudit(state, !!user);
1254               setExportOpen(false);
1255             }}
1256           >
1257             <FileText />
1258             <span>
1259               <strong>Complete audit record</strong>
1260               <small>
1261                 JSON with source text, versions, activity, and SHA-256 digest
1262               </small>
1263             </span>
1264             <Download size={17} />
1265           </button>
1266           <button
1267             onClick={() => {
1268               exportMatches(state, !!user);
1269               setExportOpen(false);
1270             }}
1271           >
1272             <Layers />
1273             <span>

```

app/lotlight.tsx (continued)

```

1274         <strong>Inventory review spreadsheet</strong>
1275         <small>
1276             CSV with identifiers, match rationale, and response status
1277         </small>
1278     </span>
1279     <Download size={17} />
1280 </button>
1281 <button
1282     onClick={() => {
1283         setView("report");
1284         setExportOpen(false);
1285     }}
1286 >
1287     <ClipboardCheck />
1288     <span>
1289         <strong>Readable audit report</strong>
1290         <small>Print or save as PDF from your browser</small>
1291     </span>
1292     <ArrowRight size={17} />
1293 </button>
1294 </div>
1295 </DialogContent>
1296 </Dialog>
1297 <AlertDialog
1298     open={!!reset}
1299     onOpenChange={(open) => {
1300         if (!open) setReset(null);
1301     }}
1302 >
1303     <AlertDialogContent>
1304         <AlertDialogTitle>
1305             {reset === "clear_workspace"
1306             ? "Start an empty workspace?"
1307             : "Restart the demonstration?"}
1308         </AlertDialogTitle>
1309         <AlertDialogDescription>
1310             This replaces the current inventory, notices, and response records.
1311             Export your audit record first if you need to keep it.{ " "}
1312             {user
1313             ? "The change will update your saved account."
1314             : "Your demo is temporary and unsaved."}
1315         </AlertDialogDescription>
1316         <AlertDialogFooter>
1317             <AlertDialogCancel>Keep my workspace</AlertDialogCancel>
1318             <AlertDialogAction
1319                 onClick={() => {
1320                     if (reset)
1321                         void send({ type: reset }).then((ok) => {
1322                             if (ok) {
1323                                 setView("overview");
1324                                 setNoticeId("baxter-demo");
1325                             }
1326                         });
1327                     setReset(null);
1328                 }}
1329             >
1330                 Replace workspace
1331             </AlertDialogAction>
1332         </AlertDialogFooter>
1333     </AlertDialogContent>
1334 </AlertDialog>
1335 </SidebarProvider>
1336 );
1337 }
1338 function Empty({
1339     title,
1340     text,
1341     action,
1342 }: {
1343     title: string;
1344     text: string;
1345     action?: React.ReactNode;
1346 }) {
1347     return (
1348         <div className="empty-state">
1349             <ScanLine size={26} />
1350             <h3>{title}</h3>
1351             <p>{text}</p>
1352             {action}
1353         </div>

```

app/lotlight.tsx (continued)

```

1354 );
1355 }
1356 function MatchRow({
1357   match: m,
1358   action,
1359   onAction,
1360   sourceApproved,
1361 }): {
1362   match: Match;
1363   action?: ResponseAction;
1364   onAction: () => void;
1365   sourceApproved: boolean;
1366 }) {
1367   const [expanded, setExpanded] = useState(false);
1368   return (
1369     <article className={"match-row " + m.kind}>
1370       <div className="match-row-main">
1371         <div className="match-product">
1372           <span className={"match-symbol " + m.kind}>
1373             {m.kind === "exact" ? (
1374               <ScanLine size={18} />
1375             ) : m.kind === "unlisted" ? (
1376               <Package size={18} />
1377             ) : (
1378               <AlertTriangle size={18} />
1379             )}
1380           </span>
1381         </div>
1382         <h3>{m.item.product}</h3>
1383         <p>
1384           {m.item.location} <span>\xb7</span> {m.item.quantity} units{" "}
1385           <span>\xb7</span> {m.item.manufacturer || "Manufacturer missing"}
1386         </p>
1387       </div>
1388     </div>
1389     <div className="match-identifiers">
1390       <code>{m.item.catalog || "No catalog"}</code>
1391       <span>
1392         LOT <b>{m.item.lot || "Missing"}</b>
1393       </span>
1394     </div>
1395     <div className="match-state">
1396       <span
1397         className={
1398           "pill " + (action?.status === "completed" ? "complete" : m.kind)
1399         }
1400       >
1401         {action?.status === "completed"
1402           ? "Response complete"
1403           : LABELS[m.kind]}
1404       </span>
1405       {m.similarity !== undefined && (
1406         <small>AI similarity {m.similarity.toFixed(2)}</small>
1407       )}
1408     </div>
1409     <button
1410       className="icon-button"
1411       aria-label={`Show evidence for ${m.item.product} in ${m.item.location}`}
1412       aria-expanded={expanded}
1413       onClick={() => setExpanded(!expanded)}
1414     >
1415       <ChevronRight size={17} className={expanded ? "rotate-90" : ""} />
1416     </button>
1417   </div>
1418   {expanded && (
1419     <div className="match-expanded">
1420       <div>
1421         <strong>Why this result</strong>
1422         <p>{m.reason}</p>
1423         <blockquote>{m.evidence}</blockquote>
1424         {action && (
1425           <p className="action-summary">
1426             Owner: {action.owner} \xb7 {action.status} \xb7{" "}
1427             {action.handledQuantity}/{m.item.quantity} units handled
1428           </p>
1429         )}
1430       </div>
1431       {m.kind !== "unlisted" && (
1432         <div>
1433           <button

```

app/lotlight.tsx (continued)

```

1434         className="button"
1435         disabled={!sourceApproved}
1436         onClick={onAction}
1437     >
1438         {action ? "Update response" : "Assign response"}{" "}
1439         <ArrowRight size={14} />
1440     </button>
1441     {!sourceApproved && (
1442         <small className="gate-hint">
1443             Confirm the Source & scope tab first.
1444         </small>
1445     )}
1446 </div>
1447 )}
1448 </div>
1449 )}
1450 </article>
1451 );
1452 }
1453 function SourceReview({
1454     notice,
1455     busy,
1456     onApprove,
1457     onEdit,
1458 }): {
1459     notice: Notice;
1460     busy: boolean;
1461     onApprove: () => Promise<boolean>;
1462     onEdit: () => void;
1463 }) {
1464     const [checked, setChecked] = useState(false);
1465     return (
1466         <div className="source-grid">
1467             <section className="source-document">
1468                 <div className="source-document-heading">
1469                     <FileText size={18} />
1470                     <strong>Preserved source text</strong>
1471                     <a href={notice.sourceUrl} target="_blank" rel="noreferrer">
1472                         Open original <ExternalLink size={13} />
1473                     </a>
1474                 </div>
1475                 <pre>{notice.text}</pre>
1476                 <button className="text-button" onClick={onEdit}>
1477                     Edit source or paired scope <ArrowRight size={14} />
1478                 </button>
1479             </section>
1480             <aside className="source-review-panel">
1481                 <span className="eyebrow">HUMAN REVIEW GATE</span>
1482                 <h3>The source sets the scope.</h3>
1483                 <p>
1484                     Confirm catalog-to-lot pairs against the full current notice. Ranges,
1485                     exclusions, kit components, and packaging exceptions need specialist
1486                     review. Supported scope uses one labeled catalog/lot line at a time.
1487                 </p>
1488                 {notice.scope.map((s, i) => (
1489                     <div className="scope-review-row" key={i}>
1490                         <strong>{s.catalog}</strong>
1491                         <span>{s.allLots ? "All lots" : s.lots.join(", ")}</span>
1492                         <small>\u201c{s.evidence}\u201d</small>
1493                     </div>
1494                 ))}
1495                 <div className="source-action">
1496                     <strong>Response summary</strong>
1497                     <p>{notice.action}</p>
1498                     <small>
1499                         Staff-entered summary. The original instructions control.
1500                     </small>
1501                 </div>
1502                 {notice.approvedAt ? (
1503                     <div className="approved-box">
1504                         <CheckCircle2 size={20} />
1505                         <div>
1506                             <strong>Source review recorded</strong>
1507                             <small>
1508                                 {notice.approvedBy}
1509                                 <br />
1510                                 {new Date(notice.approvedAt).toLocaleString()}
1511                             </small>
1512                         </div>
1513                     </div>

```

app/lotlight.tsx (continued)

```

1514         ) : (
1515             <>
1516                 <label className="check-label">
1517                     <Checkbox
1518                         checked={checked}
1519                         onCheckedChange={value => setChecked(value === true)}
1520                         id="source-check"
1521                     />
1522                     <span>
1523                         {notice.training
1524                             ? "For this demonstration, I have reviewed the source link and confirmed these two
training pairs."
1525                             : "I checked the current original notice, including affected pairs, exceptions, and
response instructions."}
1526                     </span>
1527                 </label>
1528                 <button
1529                     className="button primary full-width"
1530                     disabled={!checked || busy}
1531                     onClick={() => void onApprove()}
1532                 >
1533                     {busy ? (
1534                         <Loader2 size={15} className="spin" />
1535                     ) : (
1536                         <Check size={15} />
1537                     )}{" "}
1538                     Confirm source review
1539                 </button>
1540             </>
1541         )}
1542     </aside>
1543 </div>
1544 );
1545 }
1546 function InventoryDialog({
1547     open,
1548     onOpenChange,
1549     busy,
1550     onSave,
1551 }): {
1552     open: boolean;
1553     onOpenChange: (b: boolean) => void;
1554     busy: boolean;
1555     onSave: (s: string, synthetic: boolean) => Promise<void>;
1556 }) {
1557     const [csv, setCsv] = useState("");
1558     const [synthetic, setSynthetic] = useState(false);
1559     const [error, setError] = useState("");
1560     const preview = useMemo(() => {
1561         try {
1562             return csv ? parseCsv(csv) : null;
1563         } catch {
1564             return null;
1565         }
1566     }, [csv]);
1567     return (
1568         <Dialog open={open} onOpenChange={onOpenChange}>
1569             <DialogContent className="wide-dialog">
1570                 <DialogHeader>
1571                     <DialogTitle>Bring your inventory.</DialogTitle>
1572                     <DialogDescription>
1573                         CSV columns: product, manufacturer, catalog, lot, quantity,
1574                         location. Up to 500 lines. Use equipment and supply records only,
1575                         without patient information.
1576                     </DialogDescription>
1577                 </DialogHeader>
1578                 <label className="file-input-label">
1579                     <Upload size={18} /> Choose a CSV
1580                     <input
1581                         type="file"
1582                         accept=".csv,text/csv"
1583                         aria-label="Upload inventory CSV"
1584                         onChange={async (e) => {
1585                             const file = e.target.files?.[0];
1586                             if (file) {
1587                                 if (file.size > 500000) {
1588                                     setError("Use a CSV smaller than 500 KB.");
1589                                 }
1590                             }
1591                             setCsv(await file.text());

```

app/lotlight.tsx (continued)

```

1592         setError("");
1593     }
1594 }
1595 />
1596 </label>
1597 <label className="field-label">
1598     Or paste CSV
1599     <textarea
1600         aria-label="Inventory CSV"
1601         rows={9}
1602         value={csv}
1603         onChange={(e) => {
1604             setCsv(e.target.value);
1605             setError("");
1606         }}
1607         placeholder="product,manufacturer,catalog,lot,quantity,location"
1608     />
1609 </label>
1610 <div className="form-meta">
1611     <button
1612         className="text-button"
1613         onClick={() => {
1614             setCsv(SAMPLE_CSV);
1615             setSynthetic(true);
1616         }}
1617     >
1618         Use synthetic example
1619     </button>
1620     <span>
1621         {preview
1622             ? `${preview.length} valid lines \xb7 ${preview.filter((i) => !i.lot || !i.catalog).length}
with missing identifiers`
1623             : "Paste or upload to validate"}
1624     </span>
1625 </div>
1626 {error && (
1627     <p className="form-error" role="alert">
1628         {error}
1629     </p>
1630 )}
1631 <label className="check-label">
1632     <Checkbox
1633         checked={synthetic}
1634         onCheckedChange={(v) => setSynthetic(v === true)}
1635     />
1636     <span>This inventory is synthetic demonstration data</span>
1637 </label>
1638 <p className="fine-print">
1639     Import replaces the inventory. Previous response actions become
1640     historical; they cannot close new inventory obligations.
1641 </p>
1642 <DialogFooter>
1643     <button className="button" onClick={() => onOpenChange(false)}>
1644         Cancel
1645     </button>
1646     <button
1647         className="button primary"
1648         disabled={busy || !csv}
1649         onClick={() => {
1650             try {
1651                 parseCsv(csv);
1652                 void onSave(csv, synthetic);
1653             } catch (e) {
1654                 setError(e instanceof Error ? e.message : "Invalid CSV");
1655             }
1656         }}
1657     >
1658         Validate & import <ArrowRight size={14} />
1659     </button>
1660 </DialogFooter>
1661 </DialogContent>
1662 </Dialog>
1663 );
1664 }
1665 function NoticeDialog({
1666     open,
1667     onOpenChange,
1668     existing,
1669     busy,
1670     onSave,

```

app/lotlight.tsx (continued)

```

1671 } : {
1672   open: boolean;
1673   onOpenChange: (b: boolean) => void;
1674   existing?: Notice;
1675   busy: boolean;
1676   onSave: (
1677     n: Extract<Command, { type: "save_notice" }>["notice"],
1678   ) => Promise<void>;
1679 }) {
1680   const [title, setTitle] = useState(existing?.title ?? "");
1681   const [manufacturer, setManufacturer] = useState(
1682     existing?.manufacturer ?? "",
1683   );
1684   const [noticeDate, setDate] = useState(
1685     existing?.date ?? new Date().toISOString().slice(0, 10),
1686   );
1687   const [url, setUrl] = useState(existing?.sourceUrl ?? "");
1688   const [text, setText] = useState(existing?.text ?? "");
1689   const [action, setAction] = useState(existing?.action ?? "");
1690   const [scope, setScope] = useState<Scope[]>(existing?.scope ?? []);
1691   const [training, setTraining] = useState(existing?.training ?? false);
1692   const [error, setError] = useState("");
1693   const [reading, setReading] = useState(false);
1694   const editScope = (i: number, patch: Partial<Scope>) =>
1695     setScope((rows) => rows.map((r, j) => (j === i ? { ...r, ...patch } : r)));
1696   return (
1697     <Dialog open={open} onOpenChange={onOpenChange}>
1698       <DialogContent className="notice-dialog">
1699         <DialogHeader>
1700           <DialogTitle>
1701             {existing
1702               ? "Revise the source. Reopen the review."
1703               : "Start with the source."}
1704           </DialogTitle>
1705           <DialogDescription>
1706             Import a text-based PDF or paste the notice. Preserve paired scope
1707             and supporting excerpts. OCR and automatic range interpretation are
1708             not included.
1709           </DialogDescription>
1710         </DialogHeader>
1711         <form
1712           onSubmit={(e) => {
1713             e.preventDefault();
1714             const n = {
1715               id: existing?.id,
1716               title,
1717               manufacturer,
1718               date: noticeDate,
1719               sourceUrl: url,
1720               text,
1721               action,
1722               scope,
1723               training,
1724             };
1725             const problem = validateScope(n);
1726             if (problem) {
1727               setError(problem);
1728               return;
1729             }
1730             setError("");
1731             void onSave(n);
1732           }}
1733         >
1734           <div className="form-two">
1735             <label className="field-label">
1736               Product / review title
1737             <input
1738               required
1739               value={title}
1740               onChange={(e) => setTitle(e.target.value)}
1741               placeholder="e.g. CLEARLINK IV sets"
1742             />
1743             </label>
1744             <label className="field-label">
1745               Manufacturer as listed in inventory
1746             <input
1747               required
1748               value={manufacturer}
1749               onChange={(e) => setManufacturer(e.target.value)}
1750               placeholder="e.g. Baxter"

```

app/lotlight.tsx (continued)

```

1751         />
1752     </label>
1753     <label className="field-label">
1754         Notice date
1755         <input
1756             required
1757             type="date"
1758             value={noticeDate}
1759             onChange={(e) => setDate(e.target.value)}
1760         />
1761     </label>
1762     <label className="field-label">
1763         Original source URL
1764         <input
1765             required
1766             type="url"
1767             value={url}
1768             onChange={(e) => setUrl(e.target.value)}
1769             placeholder="https://manufacturer.example/notice"
1770         />
1771     </label>
1772 </div>
1773 <div className="source-upload-row">
1774     <label className="file-input-label">
1775         <Upload size={16} />
1776         {reading ? "Reading PDF\u2026" : "Import PDF or text"}
1777         <input
1778             type="file"
1779             accept=".pdf,.txt"
1780             aria-label="Upload recall notice"
1781             disabled={reading}
1782             onChange={async (e) => {
1783                 const f = e.target.files?.[0];
1784                 if (!f) return;
1785                 setReading(true);
1786                 try {
1787                     const t = await readNoticeFile(f);
1788                     if (t.length > 50000)
1789                         throw new Error("Notice text exceeds 50,000 characters.");
1790                     setText(t);
1791                     setScope(extractScope(t));
1792                     setError("");
1793                 } catch (error) {
1794                     setError(
1795                         error instanceof Error
1796                             ? error.message
1797                             : "Could not read file.",
1798                     );
1799                 } finally {
1800                     setReading(false);
1801                 }
1802             }}
1803         />
1804     </label>
1805     <button
1806         type="button"
1807         className="text-button"
1808         onClick={() => {
1809             setTitle("CLEARLINK IV sets");
1810             setManufacturer("Baxter");
1811             setDate("2025-08-29");
1812             setUrl(BAXTER_URL);
1813             setText(SAMPLE_NOTICE_TEXT);
1814             setScope(extractScope(SAMPLE_NOTICE_TEXT));
1815             setTraining(true);
1816             setAction(
1817                 "Identify affected stock and follow the manufacturer\u2019s supplier-specific isolation,
return or replacement, and acknowledgement instructions. Verify the current full notice first.",
1818             );
1819         }}
1820     >
1821         Use historical training extract
1822     </button>
1823 </div>
1824 <label className="field-label">
1825     Original notice text / clearly labeled training extract
1826     <textarea
1827         required
1828         rows={6}
1829         value={text}

```

app/lotlight.tsx (continued)

```

1830         onChange={(e) => setText(e.target.value)}
1831         maxLength={50000}
1832     />
1833 </label>
1834 <div className="section-heading scope-editor-heading">
1835   <h2>Paired catalog / lot scope</h2>
1836   <button
1837     type="button"
1838     className="text-button"
1839     onClick={() => {
1840       const found = extractScope(text);
1841       setScope(found);
1842       if (!found.length)
1843         setError(
1844           "No explicitly labeled pairs found. Add rows manually and copy supporting excerpts from
the source.",
1845         );
1846     }}
1847   >
1848     Extract labeled pairs
1849   </button>
1850 </div>
1851 <p className="fine-print">
1852   Auto-extraction accepts \u201cCatalog: ABC; Lots: 123, 456\u201d. For tables,
1853   append a staff-checked labeled transcription to the preserved
1854   source. Each scope excerpt must contain one complete pair.
1855   Conditional ranges are unsupported.
1856 </p>
1857 {scope.map((s, i) => (
1858   <div className="scope-editor" key={i}>
1859     <div className="scope-editor-fields">
1860       <label className="field-label">
1861         Catalog
1862         <input
1863           required
1864           value={s.catalog}
1865           onChange={(e) => editScope(i, { catalog: e.target.value })}
1866         />
1867       </label>
1868       <label className="field-label">
1869         Exact lots, comma-separated
1870         <input
1871           value={s.lots.join(", ")}
1872           disabled={s.allLots}
1873           onChange={(e) =>
1874             editScope(i, {
1875               lots: e.target.value.split(",").map((l) => l.trim()),
1876             })
1877           }
1878         />
1879       </label>
1880       <button
1881         type="button"
1882         className="icon-button"
1883         aria-label={`Remove scope row ${i + 1}`}
1884         onClick={() =>
1885           setScope((rows) => rows.filter((_, j) => i !== j))
1886         }
1887       >
1888         <Trash2 size={15} />
1889       </button>
1890     </div>
1891     <label className="check-label">
1892       <Checkbox
1893         checked={s.allLots}
1894         onCheckedChange={(v) =>
1895           editScope(i, {
1896             allLots: v === true,
1897             lots: v === true ? [] : s.lots,
1898           })
1899       >
1900     />
1901     <span>
1902       Original source explicitly says \u201call lots\u201d without exceptions
1903     </span>
1904   </label>
1905   <label className="field-label">
1906     Verbatim excerpt supporting this specific pair
1907     <textarea
1908       required

```

app/lotlight.tsx (continued)

```

1909         rows={2}
1910         value={s.evidence}
1911         onChange={(e) => editScope(i, { evidence: e.target.value })}
1912     />
1913 </label>
1914 </div>
1915 )}}
1916 <button
1917     type="button"
1918     className="text-button add-scope"
1919     onClick={() =>
1920         setScope([
1921             ...scope,
1922             { catalog: "", lots: [], allLots: false, evidence: "" },
1923         ])
1924     }
1925 >
1926     <Plus size={14} /> Add paired scope row
1927 </button>
1928 <label className="field-label">
1929     Staff-entered response summary
1930     <textarea
1931         required
1932         minLength={15}
1933         rows={3}
1934         value={action}
1935         onChange={(e) => setAction(e.target.value)}
1936         placeholder="Summarize the actual manufacturer instructions. Do not infer stop-use from the
word recall."
1937     />
1938 </label>
1939 <label className="check-label">
1940     <Checkbox
1941         checked={training}
1942         onChange={(v) => setTraining(v === true)}
1943     />
1944     <span>This is a historical or synthetic training example</span>
1945 </label>
1946 {error && (
1947     <p className="form-error" role="alert">
1948         {error}
1949     </p>
1950 )}
1951 <DialogFooter>
1952     <button
1953         type="button"
1954         className="button"
1955         onClick={() => onOpenChange(false)}
1956     >
1957         Cancel
1958     </button>
1959     <button
1960         type="submit"
1961         className="button primary"
1962         disabled={busy || reading}
1963     >
1964         Save for source review <ArrowRight size={14} />
1965     </button>
1966 </DialogFooter>
1967 </form>
1968 </DialogContent>
1969 </Dialog>
1970 );
1971 }
1972 function AlertDialog({
1973     match,
1974     notice,
1975     action,
1976     busy,
1977     onClose,
1978     onSave,
1979 }): {
1980     match: Match;
1981     notice: Notice;
1982     action?: ResponseAction;
1983     busy: boolean;
1984     onClose: () => void;
1985     onSave: (c: Command) => Promise<void>;
1986 }) {
1987     const [owner, setOwner] = useState(action?.owner ?? "");

```

app/lotlight.tsx (continued)

```

1988 const [due, setDue] = useState(
1989   action?.dueDate ??
1990   new Date(Date.now() + 86400000).toISOString().slice(0, 10),
1991 );
1992 const [status, setStatus] = useState<ResponseAction["status"]>(
1993   action?.status ?? "assigned",
1994 );
1995 const [verification, setVerification] = useState(action?.verification ?? "");
1996 const [evidence, setEvidence] = useState(action?.evidence ?? "");
1997 const [handled, setHandled] = useState(action?.handledQuantity ?? 0);
1998 return (
1999   <Dialog
2000     open
2001     onOpenChange={(open) => {
2002       if (!open) onClose();
2003     }}
2004   >
2005     <DialogContent className="wide-dialog">
2006       <DialogHeader>
2007         <DialogTitle>A clear next step.</DialogTitle>
2008         <DialogDescription>
2009           {match.item.product} \xb7 {match.item.catalog || "Catalog missing"} \xb7{" "}
2010           {match.item.lot || "Lot missing"} \xb7 {match.item.quantity} units
2011         </DialogDescription>
2012       </DialogHeader>
2013       <form
2014         onSubmit={(e) => {
2015           e.preventDefault();
2016           void onSave({
2017             type: "save_action",
2018             noticeId: notice.id,
2019             itemId: match.item.id,
2020             owner,
2021             dueDate: due,
2022             status,
2023             verification,
2024             evidence,
2025             handledQuantity: handled,
2026           });
2027         }}
2028       >
2029         <div className="form-two">
2030           <label className="field-label">
2031             Responsible person
2032             <input
2033               required
2034               minLength={2}
2035               value={owner}
2036               onChange={(e) => setOwner(e.target.value)}
2037               placeholder="e.g. Maya Rao, stock coordinator"
2038             />
2039           </label>
2040           <label className="field-label">
2041             Response due date
2042             <input
2043               required
2044               type="date"
2045               value={due}
2046               onChange={(e) => setDue(e.target.value)}
2047             />
2048           </label>
2049         </div>
2050         <label className="field-label">
2051           Response stage
2052           <Select
2053             value={status}
2054             onChange={(v) => setStatus(v as ResponseAction["status"])}
2055           >
2056             <SelectTrigger className="full-width" aria-label="Response stage">
2057               <SelectValue />
2058             </SelectTrigger>
2059             <SelectContent>
2060               <SelectItem value="assigned">
2061                 Assigned \xb7 review pending
2062               </SelectItem>
2063               <SelectItem value="verified" disabled={match.kind !== "exact"}>
2064                 Verified \xb7 physical label checked
2065               </SelectItem>
2066               <SelectItem value="completed" disabled={match.kind !== "exact"}>
2067                 Completed \xb7 response evidence recorded

```

app/lotlight.tsx (continued)

```

2068         </SelectedItem>
2069     </SelectContent>
2070 </Select>
2071 </label>
2072 {match.kind !== "exact" && (
2073     <div className="info-banner">
2074         <AlertTriangle size={16} />
2075         <span>
2076             Collect missing identifiers, then use the Correct button in
2077             Inventory to update the affected line. This uncertain line
2078             cannot be marked complete.
2079         </span>
2080     </div>
2081 )}
2082 <label className="field-label">
2083     Physical-label and current-source verification
2084     <textarea
2085         rows={2}
2086         value={verification}
2087         onChange={(e) => setVerification(e.target.value)}
2088         placeholder="Record what you checked, by whom, and against which current instructions."
2089     />
2090 </label>
2091 <div className="form-two">
2092     <label className="field-label">
2093         Units handled
2094         <input
2095             type="number"
2096             min={0}
2097             max={match.item.quantity}
2098             required
2099             value={handled}
2100             onChange={(e) => setHandled(Number(e.target.value))}
2101         />
2102     </label>
2103     <div className="quantity-note">
2104         {handled} of {match.item.quantity} units
2105         <br />
2106         <small>Partial handling keeps the response open.</small>
2107     </div>
2108 </div>
2109 <label className="field-label">
2110     Response evidence / reference
2111     <textarea
2112         rows={3}
2113         value={evidence}
2114         onChange={(e) => setEvidence(e.target.value)}
2115         placeholder="e.g. Staff-recorded isolation log Q-104, supplier return reference RMA-210. No
patient data."
2116     />
2117 </label>
2118 <p className="fine-print">
2119     Completion records the staff response for this stock and source
2120     version. It does not independently verify the evidence or terminate
2121     the recall.
2122 </p>
2123 <DialogFooter>
2124     <button type="button" className="button" onClick={onClose}>
2125         Cancel
2126     </button>
2127     <button
2128         type="submit"
2129         className="button primary"
2130         disabled={busy || !notice.approvedAt}
2131     >
2132         Save response <Check size={14} />
2133     </button>
2134 </DialogFooter>
2135 </form>
2136 </DialogContent>
2137 </Dialog>
2138 );
2139 }
2140 function About() {
2141     return (
2142         <div className="about-page">
2143             <p className="eyebrow">THE REASON BEHIND THE REVIEW</p>
2144             <h1>
2145                 A recall is a handoff.
2146             <br />

```

app/lotlight.tsx (continued)

```

2147     Someone has to catch it.
2148   </h1>
2149   <p className="intro large">
2150     Lotlight helps independent clinics turn a manufacturer notice and a
2151     spreadsheet into a review someone can explain, own, and finish.
2152   </p>
2153   <div className="about-story">
2154     <span className="eyebrow">THE DEMONSTRATION</span>
2155     <h2>
2156       One letter. Four stock locations.
2157     <br />
2158       One missing lot number.
2159     </h2>
2160     <p>
2161       Willow is a fictional clinic. Its eight inventory lines include two
2162       exact pairs from a real Baxter notice, an incomplete label, and an \u201cIV
2163       line extender\u201d that AI can surface despite different wording. Every
2164       count is demonstration data.
2165     </p>
2166     <a
2167       className="text-button"
2168       href={BAXTER_URL}
2169       target="_blank"
2170       rel="noreferrer"
2171     >
2172       Read the original manufacturer notice <ExternalLink size={14} />
2173     </a>
2174   </div>
2175   <div className="about-columns">
2176     <section>
2177       <h2>What the AI actually does</h2>
2178       <p>
2179         A quantized MiniLM sentence transformer compares product
2180         descriptions locally in a browser worker. It can rank an alias for
2181         review. It never supplies a recalled lot, clears an unknown
2182         identifier, writes medical instructions, or approves a response.
2183       </p>
2184       <p>
2185         Exact matching uses paired catalog and lot scope plus manufacturer
2186         identity. Source extraction supports explicitly labeled lines; other
2187         formats require human pairing.
2188       </p>
2189       <a
2190         href="https://huggingface.co/Xenova/all-MiniLM-L6-v2"
2191         target="_blank"
2192         rel="noreferrer"
2193       >
2194         Model card and Apache 2.0 license \u2197
2195       </a>
2196     </section>
2197     <section>
2198       <h2>A credible first customer</h2>
2199       <p>
2200         Our proposed customer is an independent clinic using spreadsheet
2201         stock records. Enterprise products such as ECRI already manage
2202         recalls. Lotlight explores a smaller, transparent
2203         inventory-reconciliation workflow.
2204       </p>
2205       <p>
2206         <strong>Pricing hypothesis: $149 per site per month.</strong> At an
2207         assumed loaded staff cost of $30/hour, five hours saved monthly
2208         would cover that price. This is a testable business hypothesis, not
2209         customer traction.
2210       </p>
2211       <a
2212         href="https://home.ecri.org/solutions/healthcare-product-alerts"
2213         target="_blank"
2214         rel="noreferrer"
2215       >
2216         Explore the established category \u2197
2217       </a>
2218     </section>
2219     <section>
2220       <h2>Clear boundaries</h2>
2221       <p>
2222         This is a tested hackathon MVP, not a clinically validated or
2223         certified recall-management system. It has private per-account
2224         storage, not shared clinic roles. It does not monitor all recalls,
2225         contact suppliers, provide OCR, or decide patient treatment.
2226       </p>

```

app/lotlight.tsx (continued)

```

2227         <p>
2228             No patient information is needed. Public model files download on
2229             first AI use; inference happens on the device. Signed-in records are
2230             stored in the hosted database. Anonymous demo changes remain in
2231             memory and are lost on reload or sign-in.
2232         </p>
2233     </section>
2234     <section>
2235         <h2>Evidence, without overclaiming</h2>
2236         <p>
2237             The FDA explains that a recall can involve correction or removal.
2238             Follow the actual current instructions. openFDA is a weekly research
2239             source and explicitly does not support public alerts or recall
2240             lifecycle tracking. Lotlight therefore starts with an uploaded
2241             notice.
2242         </p>
2243         <a
2244             href="https://www.fda.gov/medical-devices/medical-device-recalls-and-early-alerts/what-medical-
device-recall"
2245             target="_blank"
2246             rel="noreferrer"
2247         >
2248             FDA: what a medical device recall means \u2197
2249         </a>
2250         <a
2251             href="https://open.fda.gov/apis/device/recall/"
2252             target="_blank"
2253             rel="noreferrer"
2254         >
2255             openFDA usage limitations \u2197
2256         </a>
2257     </section>
2258 </div>
2259 <div className="creator-credit">
2260     <span className="brand-mark">
2261         <ScanLine size={24} />
2262     </span>
2263     <div>
2264         <strong>A project by Shivam Gupta</strong>
2265         <p>
2266             Built for DSH Hacks V2 with AI-assisted research, implementation,
2267             testing, and presentation preparation.
2268         </p>
2269     </div>
2270     <a
2271         href="https://github.com/shi1720/DSH-Hacks-V2"
2272         target="_blank"
2273         rel="noreferrer"
2274     >
2275         View the code \u2197
2276     </a>
2277 </div>
2278 </div>
2279 );
2280 }
2281 function AuditReport({ state }: { state: Workspace }) {
2282     return (
2283         <article className="audit-report">
2284             <div className="report-head">
2285                 <div className="brand">
2286                     <span className="brand-mark">
2287                         <ScanLine size={23} />
2288                     </span>
2289                     lotlight
2290                 </div>
2291                 <span>AUDIT RECORD \xb7 {date(new Date().toISOString())}</span>
2292             </div>
2293             <h1>Recall review record</h1>
2294             <p>
2295                 Workspace revision {state.revision}. Inventory revision{" "}
2296                 {state.inventoryVersion}. {state.inventory.length} stock lines.
2297                 Staff-recorded evidence, not independent verification.
2298             </p>
2299             <p>
2300                 Inventory provenance: {state.inventoryProvenance ?? "synthetic"}. Last
2301                 record update: {state.savedAt ?? "No recorded update"}.
2302             </p>
2303             <p className="report-notice">
2304                 {state.notices.some((n) => n.training)
2305                 ? "HISTORICAL TRAINING SOURCE: this does not show current recall status. Inventory provenance is

```

app/lotlight.tsx (continued)

```

listed separately above."
2306       : "USER-SUPPLIED RECORDS: verify complete current manufacturer instructions before acting."}
2307   </p>
2308   {state.notices.map((n) => (
2309     <section key={n.id}>
2310       <h2>
2311         {n.title} \xb7 version {n.version}
2312       </h2>
2313       <p>
2314         {n.manufacturer} \xb7 Notice {n.date} \xb7{" "}
2315         {n.approvedAt
2316           ? `Source review recorded by ${n.approvedBy} on ${date(n.approvedAt)}`
2317           : "SOURCE NOT APPROVED"}
2318       </p>
2319       <a href={n.sourceUrl}>{n.sourceUrl}</a>
2320       <h3>Source scope</h3>
2321       {n.scope.map((s, i) => (
2322         <p key={i}>
2323           {s.catalog}: {s.allLots ? "All lots" : s.lots.join(", ")}
2324         <br />
2325         <em>{s.evidence}</em>
2326       </p>
2327     ))}
2328     <h3>Response summary</h3>
2329     <p>{n.action}</p>
2330     <Table>
2331       <TableHeader>
2332         <TableRow>
2333           <TableHead>Stock / location</TableHead>
2334           <TableHead>Identifiers</TableHead>
2335           <TableHead>Result</TableHead>
2336           <TableHead>Response</TableHead>
2337         </TableRow>
2338       </TableHeader>
2339       <TableBody>
2340         {reconcile(state.inventory, n).map((m) => {
2341           const a = activeAction(state, n.id, m.item.id);
2342           return (
2343             <TableRow key={m.item.id}>
2344               <TableCell>
2345                 {m.item.product}
2346               <br />
2347               {m.item.location} \xb7 {m.item.quantity} units
2348             </TableCell>
2349               <TableCell>
2350                 {m.item.catalog || "Missing catalog"}
2351               <br />
2352                 {m.item.lot || "Missing lot"}
2353             </TableCell>
2354               <TableCell>
2355                 {LABELS[m.kind]}
2356               <br />
2357                 <small>{m.reason}</small>
2358             </TableCell>
2359               <TableCell>
2360                 {a
2361                   ? `${a.status} \xb7 ${a.owner} \xb7 ${a.handledQuantity} units handled`
2362                   : "No response recorded"}
2363               <br />
2364                 {a?.evidence}
2365             </TableCell>
2366             </TableRow>
2367           );
2368         })}
2369       </TableBody>
2370     </Table>
2371   </section>
2372 ))}
2373 <h2>Activity history</h2>
2374 {state.audit.map((e) => (
2375   <div className="report-event" key={e.id}>
2376     <strong>
2377       {e.type} \xb7 {e.actor}
2378     </strong>
2379     <small>{e.at}</small>
2380     <p>{e.detail}</p>
2381   </div>
2382 ))}
2383 <p className="fine-print">
2384   No listed match is not a safety clearance. The JSON export includes full

```

app/lotlight.tsx (continued)

```

2385         source text and historical response records. No inference about patient
2386         safety or FDA recall termination is made.
2387     </p>
2388 </article>
2389 );
2390 }
2391 function CorrectionDialog({
2392     item,
2393     busy,
2394     onClose,
2395     onSave,
2396 }): {
2397     item: InventoryItem;
2398     busy: boolean;
2399     onClose: () => void;
2400     onSave: (c: Command) => Promise<void>;
2401 }) {
2402     const [form, setForm] = useState(item);
2403     return (
2404         <Dialog
2405             open
2406             onOpenChange={(v) => {
2407                 if (!v) onClose();
2408             }}
2409         >
2410             <DialogContent className="wide-dialog">
2411                 <DialogHeader>
2412                     <DialogTitle>Correct the physical record.</DialogTitle>
2413                     <DialogDescription>
2414                         Check the label before editing. This line gets a new revision and
2415                         fresh response review. Other unchanged lines keep their current
2416                         response state.
2417                     </DialogDescription>
2418                 </DialogHeader>
2419                 <form
2420                     onSubmit={(e) => {
2421                         e.preventDefault();
2422                         void onSave({
2423                             type: "correct_item",
2424                             id: item.id,
2425                             product: form.product,
2426                             manufacturer: form.manufacturer,
2427                             catalog: form.catalog,
2428                             lot: form.lot,
2429                             quantity: form.quantity,
2430                             location: form.location,
2431                         });
2432                     }}
2433                 >
2434                     <div className="form-two">
2435                         {(
2436                             ["product", "manufacturer", "catalog", "lot", "location"] as const
2437                         ).map((key) => (
2438                             <label className="field-label" key={key}>
2439                                 {key[0].toUpperCase() + key.slice(1)}
2440                                 <input
2441                                     value={form[key]}
2442                                     required={key === "product" || key === "location"}
2443                                     onChange={(e) => setForm({ ...form, [key]: e.target.value })}
2444                                 />
2445                             </label>
2446                         ))}
2447                     <label className="field-label">
2448                         Quantity
2449                         <input
2450                             type="number"
2451                             min={1}
2452                             max={1000000}
2453                             required
2454                             value={form.quantity}
2455                             onChange={(e) =>
2456                                 setForm({ ...form, quantity: Number(e.target.value) })
2457                             }
2458                         />
2459                     </label>
2460                 </div>
2461                 <DialogFooter>
2462                     <button type="button" className="button" onClick={onClose}>
2463                         Cancel
2464                     </button>

```

app/lotlight.tsx (continued)

```

2465         <button type="submit" className="button primary" disabled={busy}>
2466             Save correction <Check size={14} />
2467         </button>
2468     </DialogFooter>
2469 </form>
2470 </DialogContent>
2471 </Dialog>
2472 );
2473 }
```

app/page.tsx

```
1 import Lotlight from "./lotlight";
2 import { getChatGPTUser } from "./chatgpt-auth";
3 export const dynamic = "force-dynamic";
4 export default async function Home() {
5   const user = await getChatGPTUser();
6   return (
7     <Lotlight
8       user={user ? { name: user.displayName, email: user.email } : null}
9     />
10  );
11 }
```

db/index.ts

```
1 import { env } from "cloudflare:workers";
2 import { drizzle } from "drizzle-orm/d1";
3 import * as schema from "./schema";
4
5 export function getDb() {
6   if (!env.DB) {
7     throw new Error(
8       "Cloudflare D1 binding `DB` is unavailable. Set the `d1` field in .openai/hosting.json to `DB` or let
       your control plane inject the real binding values before using the database."
9     );
10  }
11
12  return drizzle(env.DB, { schema });
13 }
```

db/schema.ts

```
1 import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";
2 export const workspaces = sqliteTable("workspaces", {
3   ownerId: text("owner_id").primaryKey(),
4   state: text("state").notNull(),
5   revision: integer("revision").notNull().default(0),
6   updatedAt: text("updated_at").notNull(),
7 });
```

drizzle/0000_real_hellcat.sql

```
1 CREATE TABLE `workspaces` (  
2   `owner_id` text PRIMARY KEY NOT NULL,  
3   `state` text NOT NULL,  
4   `revision` integer DEFAULT 0 NOT NULL,  
5   `updated_at` text NOT NULL  
6 );
```

firebase/index.html

```
1 <!doctype html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-  
width, initial-scale=1.0"><meta name="theme-color" content="#263329"><meta name="description" content="Turn  
a medical recall notice into a documented response. Review inventory, resolve missing identifiers, and keep  
an evidence trail."><title>Lotlight | Recall to response</title><link rel="icon"  
href="/favicon.svg"></head><body><div id="root"></div><script type="module"  
src="/main.tsx"></script></body></html>
```

firebase/main.tsx

```

1 import React, { useEffect, useState, useCallback } from "react";
2 import { createRoot } from "react-dom/client";
3 import { initializeApp, type FirebaseOptions } from "firebase/app";
4 import {
5   getAuth,
6   onAuthStateChanged,
7   signInWithEmailAndPassword,
8   createUserWithEmailAndPassword,
9   sendPasswordResetEmail,
10  signOut,
11  type User,
12 } from "firebase/auth";
13 import Lotlight from "../app/lotlight";
14 import {
15   Dialog,
16   DialogContent,
17   DialogHeader,
18   DialogTitle,
19   DialogDescription,
20 } from "@components/ui/dialog";
21 import "../app/globals.css";
22 let config: FirebaseOptions & { apiBase: string };
23 let auth: ReturnType<typeof getAuth>;
24 function App() {
25   const [user, setUser] = useState<User | null>(null),
26     [ready, setReady] = useState(false),
27     [open, setOpen] = useState(false),
28     [mode, setMode] = useState<"signin" | "signup" | "reset">("signin");
29   const [email, setEmail] = useState(""),
30     [password, setPassword] = useState(""),
31     [error, setError] = useState(""),
32     [busy, setBusy] = useState(false),
33     [message, setMessage] = useState("");
34   useEffect(
35     () =>
36     onAuthStateChanged(auth, (u) => {
37       setUser(u);
38       setReady(true);
39     }),
40     [],
41   );
42   const request = useCallback(async (input: string, init?: RequestInit) => {
43     const current = auth.currentUser;
44     if (!current)
45       return Response.json(
46         { error: "Your session ended. Sign in again." },
47         { status: 401 },
48       );
49     const token = await current.getIdToken();
50     return fetch(config.apiBase + input, {
51       ...init,
52       headers: { ...init?.headers, Authorization: "Bearer " + token },
53       credentials: "omit",
54     });
55   }, []);
56   async function authenticate(e: React.FormEvent) {
57     e.preventDefault();
58     setBusy(true);
59     setError("");
60     setMessage("");
61     try {
62       if (mode === "reset") {
63         await sendPasswordResetEmail(auth, email);
64         setMessage(
65           "If an account exists for this email, password reset instructions will arrive shortly.",
66         );
67       } else {
68         if (mode === "signup")
69           await createUserWithEmailAndPassword(auth, email, password);
70         else await signInWithEmailAndPassword(auth, email, password);
71         setOpen(false);
72         setPassword("");
73       }
74     } catch (err) {
75       const code = (err as { code?: string }).code;
76       setError(
77         code === "auth/weak-password"
78           ? "Use at least 12 characters for your password."
79           : code === "auth/email-already-in-use"
80             ? "An account already uses this email. Sign in or reset your password."

```

firebase/main.tsx (continued)

```

81         : code === "auth/network-request-failed"
82         ? "Connection failed. Please retry."
83         : code === "auth/too-many-requests"
84         ? "Too many attempts. Wait a moment and try again."
85         : "Sign-in failed. Check your email and password, or create an account.",
86     );
87   } finally {
88     setBusy(false);
89   }
90 }
91 if (!ready)
92   return (
93     <div className="loading-state">
94       <h1>Opening Lotlight</h1>
95       <p>Checking your saved session.</p>
96     </div>
97   );
98 return (
99   <>
100     <Lotlight
101       key={user?.uid ?? "demo"}
102       user={
103         user
104         ? {
105             name:
106               user.displayName ||
107               user.email?.split("@")[0] ||
108               "Clinic reviewer",
109             email: user.email || "",
110           }
111         : null
112       }
113       request={request}
114       onAuth={() => {
115         if (user) {
116           void signOut(auth);
117         } else {
118           setOpen(true);
119           setError("");
120           setMessage("");
121         }
122       }}
123     />
124     <Dialog open={open} onOpenChange={setOpen}>
125       <DialogContent className="auth-dialog">
126         <DialogHeader>
127           <DialogTitle>
128             {mode === "signup"
129               ? "Create your workspace"
130               : mode === "reset"
131                 ? "Reset your password"
132                 : "Welcome back"}
133           </DialogTitle>
134           <DialogDescription>
135             Save inventory, source reviews and response history in your
136             private account. The current unsaved demo does not transfer when
137             you sign in. Do not enter patient data.
138           </DialogDescription>
139         </DialogHeader>
140         <form onSubmit={authenticate} className="auth-form">
141           <label>
142             Email address
143             <input
144               type="email"
145               autoComplete="email"
146               required
147               value={email}
148               onChange={(e) => setEmail(e.target.value)}
149             />
150           </label>
151           {mode !== "reset" && (
152             <label>
153               Password
154               <input
155                 type="password"
156                 autoComplete={
157                   mode === "signup" ? "new-password" : "current-password"
158                 }
159                 minLength={mode === "signup" ? 12 : undefined}
160                 required

```

firebase/main.tsx (continued)

```

161         value={password}
162         onChange={(e) => setPassword(e.target.value)}
163       />
164     </label>
165   )}
166   {mode === "signup" && (
167     <p className="field-help">
168       Use 12 or more characters. Your account is private to you.
169     </p>
170   )}
171   {error && (
172     <p role="alert" className="auth-error">
173       {error}
174     </p>
175   )}
176   {message && <p role="status">{message}</p>}
177   <button className="btn-primary" disabled={busy}>
178     {busy
179       ? "Please wait\u2026"
180       : mode === "signup"
181         ? "Create account"
182         : mode === "reset"
183           ? "Send reset instructions"
184           : "Sign in"}
185   </button>
186 </form>
187 <div className="auth-links">
188   <button
189     className="text-button"
190     onClick={() => {
191       setMode(mode === "signup" ? "signin" : "signup");
192       setError("");
193       setMessage("");
194     }}
195   >
196     {mode === "signup"
197       ? "Already have an account? Sign in"
198       : "New here? Create an account"}
199   </button>
200   <button
201     className="text-button"
202     onClick={() => {
203       setMode(mode === "reset" ? "signin" : "reset");
204       setError("");
205       setMessage("");
206     }}
207   >
208     {mode === "reset" ? "Back to sign in" : "Forgot password?"}
209   </button>
210 </div>
211 </DialogContent>
212 </Dialog>
213 </>
214 );
215 }
216 const root = createRoot(document.getElementById("root")!);
217 root.render(<div className="loading-state"><h1>Opening Lotlight</h1><p>Loading your workspace
  tools.</p></div>);
218 async function bootstrap() {
219   try {
220     const response = await fetch("/firebase-config.json", { cache: "no-store" });
221     if (!response.ok) throw new Error("Configuration unavailable");
222     config = await response.json();
223     if (!config.apiKey || !config.projectId || !config.apiBase?.startsWith("https://"))
224       throw new Error("Invalid configuration");
225     auth = getAuth(initializeApp(config));
226     root.render(<App />);
227   } catch {
228     root.render(<div className="loading-state" role="alert"><h1>Lotlight could not load</h1><p>Check your
      connection and try again. Your saved workspace has not changed.</p><button className="btn-primary"
      onClick={() => window.location.reload()}>Retry loading</button></div>);
229   }
230 }
231 void bootstrap();

```

firebase.json

```
1 {"hosting":{"site":"lotlight-care","public":"dist-firebase","ignore":["firebase.json","**/*.","**/node_modules/**"],"rewrites":[{"source":"**","destination":"/index.html"}],"headers":[{"source":"**","headers":[{"key":"X-Content-Type-Options","value":"nosniff"},{"key":"Referrer-Policy","value":"strict-origin-when-cross-origin"},{"key":"X-Frame-Options","value":"DENY"}]},{"source":"/firebase-config.json","headers":[{"key":"Cache-Control","value":"no-store"}]},{"source":"/index.html","headers":[{"key":"Cache-Control","value":"no-cache"}]},{"source":"/assets/**","headers":[{"key":"Cache-Control","value":"public,max-age=31536000,immutable"}]}]},"firestore":{"rules":"firestore.rules"}}
```

firestore.rules

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     // Clients use the authenticated, validated API. Direct database access is denied.
5     match /{document=**} { allow read, write: if false; }
6   }
7 }
```

lib/lotlight/commands.ts

```

1 import { z } from "zod";
2 import { parseCsv, validateScope, matchItem, inventoryKey } from "../engine";
3 import { initialWorkspace } from "../sample";
4 import type { Workspace, ResponseAction } from "../types";
5 const short = z.string().trim().min(1).max(200);
6 const isoDate = z
7   .string()
8   .regex(/^d{4}-d{2}-d{2}$/)
9   .refine(
10     (s) =>
11       !Number.isNaN(Date.parse(s)) &&
12       new Date(s).toISOString().slice(0, 10) === s,
13     "Use a valid calendar date.",
14   );
15 const scopeSchema = z
16   .object({
17     catalog: short,
18     lots: z.array(short).max(100),
19     allLots: z.boolean(),
20     evidence: z.string().trim().min(1).max(5000),
21   })
22   .strict();
23 const noticeInput = z
24   .object({
25     id: z.string().max(100).optional(),
26     title: short,
27     manufacturer: short,
28     date: isoDate,
29     sourceUrl: z
30       .string()
31       .url()
32       .max(1500)
33       .refine(
34         (s) => new URL(s).protocol === "https:",
35         "Use an HTTPS source URL.",
36       ),
37     text: z.string().trim().min(20).max(50000),
38     action: z.string().trim().min(15).max(2000),
39     scope: z.array(scopeSchema).min(1).max(40),
40     training: z.boolean(),
41   })
42   .strict();
43 export const commandSchema = z.discriminatedUnion("type", [
44   z
45     .object({
46       type: z.literal("import_inventory"),
47       csv: z.string().max(500000),
48       synthetic: z.boolean().optional(),
49     })
50     .strict(),
51   z.object({ type: z.literal("save_notice"), notice: noticeInput }).strict(),
52   z
53     .object({
54       type: z.literal("correct_item"),
55       id: short,
56       product: short,
57       manufacturer: z.string().trim().max(200),
58       catalog: z.string().trim().max(200),
59       lot: z.string().trim().max(200),
60       quantity: z.number().int().min(1).max(1000000),
61       location: short,
62     })
63     .strict(),
64   z
65     .object({
66       type: z.literal("approve_notice"),
67       noticeId: short,
68       sourceConfirmed: z.literal(true),
69     })
70     .strict(),
71   z
72     .object({
73       type: z.literal("save_action"),
74       noticeId: short,
75       itemId: short,
76       owner: z.string().trim().min(2).max(100),
77       dueDate: isoDate,
78       status: z.enum(["assigned", "verified", "completed"]),
79       verification: z.string().trim().max(2000),
80       evidence: z.string().trim().max(2000),

```

lib/lotlight/commands.ts (continued)

```

81     handledQuantity: z.number().int().min(0).max(1000000),
82   })
83   .strict(),
84   z.object({ type: z.literal("reset_demo") }).strict(),
85   z.object({ type: z.literal("clear_workspace") }).strict(),
86 });
87 export type Command = z.infer<typeof commandSchema>;
88 export function activeAction(
89   state: Workspace,
90   noticeId: string,
91   itemId: string,
92 ): ResponseAction | undefined {
93   const notice = state.notices.find((n) => n.id === noticeId);
94   const item = state.inventory.find((i) => i.id === itemId);
95   if (!notice || !item) return undefined;
96   return state.actions.find(
97     (a) =>
98       a.noticeId === noticeId &&
99       a.itemId === itemId &&
100       a.noticeVersion === notice?.version &&
101       (a.itemVersion ?? 1) === (item?.version ?? 1),
102   );
103 }
104 export function reduceWorkspace(
105   current: Workspace,
106   untrusted: unknown,
107   actor: string,
108   now = new Date().toISOString(),
109 ): Workspace {
110   const cmd = commandSchema.parse(untrusted);
111   let next = structuredClone(current);
112   next.noticeHistory ??= [];
113   next.inventoryHistory ??= [];
114   next.inventoryProvenance ??= "synthetic";
115   let detail = "";
116   if (current.audit.length >= 2000 && cmd.type !== "clear_workspace")
117     throw new Error(
118       "Workspace audit limit reached. Export your record before starting a fresh workspace.",
119     );
120   if (cmd.type === "reset_demo") {
121     next = initialWorkspace();
122     next.audit = current.audit;
123     next.inventoryVersion = current.inventoryVersion + 1;
124     detail =
125       "Started a new synthetic demonstration. Prior activity remains in the audit log; export detailed case
126     records before resetting.";
127   }
128   if (cmd.type === "clear_workspace") {
129     next = {
130       inventory: [],
131       inventoryProvenance: "user-supplied",
132       noticeHistory: [],
133       inventoryHistory: [],
134       inventoryVersion: current.inventoryVersion + 1,
135       notices: [],
136       actions: [],
137       audit: [],
138       revision: current.revision,
139       savedAt: null,
140     };
141     detail =
142       "Started an empty workspace. Earlier records were removed at the account holder\u2019s request.";
143   }
144   if (cmd.type === "import_inventory") {
145     next.inventoryHistory.push({
146       version: next.inventoryVersion,
147       items: structuredClone(next.inventory),
148       provenance: next.inventoryProvenance,
149     });
150     next.inventory = parseCsv(cmd.csv);
151     next.inventoryProvenance = cmd.synthetic ? "synthetic" : "user-supplied";
152     next.inventoryVersion++;
153     detail = `Imported ${next.inventory.length} inventory lines as revision ${next.inventoryVersion}. Prior
154     response actions are historical and do not close new inventory obligations.`;
155   }
156   if (cmd.type === "correct_item") {
157     const old = next.inventory.find((i) => i.id === cmd.id);
158     if (!old) throw new Error("Inventory line not found.");
159     if (

```

lib/lotlight/commands.ts (continued)

```

159         (i) => i.id !== cmd.id && inventoryKey(i) === inventoryKey(cmd),
160     )
161 )
162     throw new Error(
163         "Duplicate inventory line. Combine quantities before importing, or correct the existing record.",
164     );
165     next.inventoryHistory.push({
166         version: next.inventoryVersion,
167         items: structuredClone(next.inventory),
168         provenance: next.inventoryProvenance,
169     });
170     next.inventoryVersion++;
171     const { type, ...fields } = cmd;
172     next.inventory = next.inventory.map((i) =>
173         i.id === cmd.id ? { ...fields, version: (old.version ?? 1) + 1 } : i,
174     );
175     detail = `Corrected ${old.product} at ${old.location}. Item revision ${old.version ?? 1} + 1,
inventory revision ${next.inventoryVersion}. This line\u2019s responses require fresh review; other
unchanged lines retain their response state.`;
176 }
177 if (cmd.type === "save_notice") {
178     const error = validateScope(cmd.notice);
179     if (error) throw new Error(error);
180     const existing = cmd.notice.id
181         ? next.notices.find((n) => n.id === cmd.notice.id)
182         : undefined;
183     if (cmd.notice.id && !existing) throw new Error("Notice not found.");
184     if (!existing && next.notices.length >= 20)
185         throw new Error("This workspace supports up to 20 notices.");
186     if (existing) next.noticeHistory.push(structuredClone(existing));
187     const notice = {
188         ...cmd.notice,
189         id: existing?.id ?? crypto.randomUUID(),
190         version: (existing?.version ?? 0) + 1,
191         approvedBy: null,
192         approvedAt: null,
193     };
194     next.notices = existing
195         ? next.notices.map((n) => (n.id === existing.id ? notice : n))
196         : [notice, ...next.notices];
197     detail = `${existing ? "Updated" : "Added"} notice \u201c${notice.title}\u201d, version
${notice.version}. Source approval is required. Earlier version actions remain historical.`;
198 }
199 if (cmd.type === "approve_notice") {
200     const notice = next.notices.find((n) => n.id === cmd.noticeId);
201     if (!notice) throw new Error("Notice not found.");
202     const error = validateScope(notice);
203     if (error) throw new Error(error);
204     notice.approvedBy = actor;
205     notice.approvedAt = now;
206     detail = `Staff confirmed source and paired scope for \u201c${notice.title}\u201d version
${notice.version}${notice.training ? " in a training scenario" : ""}. This is staff attestation, not
independent verification.`;
207 }
208 if (cmd.type === "save_action") {
209     const notice = next.notices.find((n) => n.id === cmd.noticeId);
210     const item = next.inventory.find((i) => i.id === cmd.itemId);
211     if (!notice || !item)
212         throw new Error("Notice or inventory line not found.");
213     if (!notice.approvedAt)
214         throw new Error(
215             "Review and confirm the notice source before assigning a response.",
216         );
217     const match = matchItem(item, notice);
218     if (match.kind === "unlisted")
219         throw new Error(
220             "This line has no listed identifier match in this notice.",
221         );
222     if (cmd.handledQuantity > item.quantity)
223         throw new Error("Handled quantity cannot exceed recorded stock.");
224     if (cmd.status !== "assigned") {
225         if (match.kind !== "exact")
226             throw new Error(
227                 "Resolve missing identifiers in the inventory before recording verification or completion.",
228             );
229         if (cmd.verification.length < 12)
230             throw new Error(
231                 "Record how you checked the physical label and current manufacturer instructions (at least 12
characters).",
232             );

```

lib/lotlight/commands.ts (continued)

```

233     }
234     if (cmd.status === "assigned" && cmd.handledQuantity !== 0)
235         throw new Error(
236             "Verify the physical label before recording handled units.",
237         );
238     if (cmd.handledQuantity > 0 && cmd.evidence.length < 12)
239         throw new Error(
240             "Record response evidence and a supporting reference for handled units (at least 12 characters).",
241         );
242     if (
243         cmd.status === "completed" &&
244         (cmd.handledQuantity !== item.quantity || cmd.evidence.length < 12)
245     )
246         throw new Error(
247             "Completion requires all recorded units handled and a response evidence reference.",
248         );
249     const existing = activeAction(next, notice.id, item.id);
250     const action: ResponseAction = {
251         id: existing?.id ?? crypto.randomUUID(),
252         noticeId: notice.id,
253         noticeVersion: notice.version,
254         itemId: item.id,
255         inventoryVersion: next.inventoryVersion,
256         itemVersion: item.version ?? 1,
257         itemLabel: `${item.product} \xb7 ${item.catalog || "no catalog"} \xb7 ${item.lot || "no lot"} \xb7
258         ${item.location}`,
259         owner: cmd.owner,
260         dueDate: cmd.dueDate,
261         status: cmd.status,
262         verification: cmd.verification,
263         evidence: cmd.evidence,
264         handledQuantity: cmd.handledQuantity,
265         updatedAt: now,
266         updatedBy: actor,
267     };
268     next.actions = existing
269         ? next.actions.map((a) => (a.id === existing.id ? action : a))
270         : [action, ...next.actions];
271     detail = `${cmd.status === "completed" ? "Response recorded as complete" : cmd.status === "verified" ?
272     "Physical verification recorded" : "Response assigned"}: ${action.itemLabel}. Owner: ${action.owner}.
273     Handled ${cmd.handledQuantity}/${item.quantity}. Verification: ${cmd.verification || "pending"}. Evidence:
274     ${cmd.evidence || "pending"}`;
275 }
276 next.revision = current.revision + 1;
277 next.savedAt = now;
278 next.audit.push({
279     id: crypto.randomUUID(),
280     at: now,
281     actor,
282     type: cmd.type,
283     detail,
284 });
285 return next;
286 }

```

lib/lotlight/engine.ts

```

1 import type { InventoryItem, Match, Notice, Scope } from "./types";
2 /** Conservative normalization preserves separators and leading zeroes. */
3 export const identifier = (value: string) =>
4   value.normalize("NFKC").trim().toUpperCase();
5 export const company = (value: string) =>
6   value.normalize("NFKC").trim().toLowerCase().replace(/\s+/g, " ");
7 export function matchItem(
8   item: InventoryItem,
9   notice: Notice,
10  similarity?: number,
11 ): Match {
12   const manufacturerMatches =
13     company(item.manufacturer) === company(notice.manufacturer);
14   const productScope = notice.scope.filter(
15     (s) => identifier(s.catalog) === identifier(item.catalog),
16   );
17   const semanticallyRelated = similarity !== undefined && similarity >= 0.3;
18   const words = notice.title
19     .toLowerCase()
20     .split(/\W+/)
21     .filter((w) => w.length > 3);
22   const nameRelated = words.some((w) => item.product.toLowerCase().includes(w));
23   const base = { item, similarity };
24   if (item.manufacturer && !manufacturerMatches && productScope.length)
25     return {
26       ...base,
27       kind: "missing",
28       reason:
29         "Catalog agrees, but manufacturer identity differs. Verify whether this is an alias or a different
30 maker.",
31       evidence: `Inventory: ${item.manufacturer}. Source: ${notice.manufacturer}. Catalog ${item.catalog}.`,
32     };
33   if (item.manufacturer && !manufacturerMatches)
34     return {
35       ...base,
36       kind: "unlisted",
37       reason:
38         "Manufacturer differs. No listed match in this notice; this is not a safety clearance.",
39       evidence: `Inventory manufacturer: ${item.manufacturer}. Notice manufacturer:
40 ${notice.manufacturer}.`,
41     };
42   if (!item.catalog) {
43     if (manufacturerMatches || semanticallyRelated || nameRelated)
44       return {
45         ...base,
46         kind: semanticallyRelated ? "possible" : "missing",
47         reason: semanticallyRelated
48           ? "AI found a similar description, but catalog and lot evidence must be checked."
49           : "Catalog number is missing. Check the physical label before deciding.",
50         evidence:
51           "No catalog identifier available. Product-name similarity cannot prove recall scope.",
52       };
53     return {
54       ...base,
55       kind: "missing",
56       reason:
57         "Product identity is incomplete. Obtain manufacturer and catalog information.",
58       evidence: "Missing catalog and unverified manufacturer.",
59     };
60   }
61   if (!productScope.length)
62     return {
63       ...base,
64       kind: "unlisted",
65       reason:
66         "Catalog is not listed in this reviewed scope. This does not establish safety or cover other
67 notices.",
68       evidence: `Inventory catalog ${item.catalog}; listed: ${notice.scope.map((s) => s.catalog).join(",
69 ")}.`,
70     };
71   if (!item.manufacturer)
72     return {
73       ...base,
74       kind: "missing",
75       reason:
76         "Catalog matches but manufacturer is missing. Verify the physical label.",
77       evidence: productScope[0].evidence,
78     };
79   if (!item.lot && !productScope.some((s) => s.allLots))
80     return {

```

lib/lotlight/engine.ts (continued)

```

77     ...base,
78     kind: "missing",
79     reason:
80       "Catalog matches, but the lot is missing. The item cannot be cleared.",
81     evidence: productScope.map((s) => s.evidence).join("\n"),
82   };
83   const scope = productScope.find(
84     (s) =>
85       s.allLots || s.lots.some((l) => identifier(l) === identifier(item.lot)),
86   );
87   if (scope)
88     return {
89       ...base,
90       kind: "exact",
91       reason:
92         "Manufacturer, catalog, and listed lot scope agree. Verify the physical label and current notice
before acting.",
93       evidence: scope.evidence,
94     };
95   return {
96     ...base,
97     kind: "unlisted",
98     reason:
99       "Catalog matches, but this lot is not listed in the reviewed scope. No safety clearance is implied.",
100    evidence: productScope.map((s) => s.evidence).join("\n"),
101  };
102 }
103 export function reconcile(
104   inventory: InventoryItem[],
105   notice: Notice,
106   scores: Record<string, number> = {},
107 ) {
108   const order = { exact: 0, missing: 1, possible: 2, unlisted: 3 };
109   return inventory
110     .map((item) => matchItem(item, notice, scores[item.id]))
111     .sort(
112       (a, b) =>
113         order[a.kind] - order[b.kind] ||
114         (b.similarity ?? 0) - (a.similarity ?? 0),
115     );
116 }
117 /** Extract only explicitly paired labeled lines. Never cross-join catalogs and lots. */
118 export function extractScope(text: string): Scope[] {
119   const result: Scope[] = [];
120   for (const line of text.split(/\r?\n/)) {
121     const m = line.match(
122       /^\\s*(?:Catalog|REF|Model)\\s*:\\s*([A-Za-z0-9._/-]+)\\s*;\\s*Lots?\\s*:\\s*(.+)\\s*$/i,
123     );
124     if (!m) continue;
125     const raw = m[2].trim();
126     if (
127       /(?:\\bto\\b|\\bthrough\\b|\\bexcept\\b|\\bexcluding\\b|\\bbetween\\b|\\bonly if\\b|\\*|\\.\\.)/i.test(
128         raw,
129       )
130     )
131       continue;
132     const allLots = /^all lots$/i.test(raw);
133     const lots = allLots
134       ? []
135       : raw
136         .split(/[,;]/)
137         .map((l) => l.trim())
138         .filter(Boolean);
139     if (
140       !allLots &&
141       (!lots.length || lots.some((l) => !/^([A-Za-z0-9._/-]+)$/i.test(l)))
142     )
143       continue;
144     result.push({ catalog: m[1], lots, allLots, evidence: line.trim() });
145   }
146   return result;
147 }
148 export function validateScope(
149   notice: Pick<Notice, "text" | "scope" | "manufacturer">,
150 ): string | null {
151   if (!notice.scope.length)
152     return "Add at least one explicit catalog and lot pair. Ranges, exclusions, and serial-number scope need
specialist review.";
153   if (!notice.text.toLowerCase().includes(notice.manufacturer.toLowerCase()))
154     return "Manufacturer must appear in the source text.";

```

lib/lotlight/engine.ts (continued)

```

155   for (const s of notice.scope) {
156     if (!s.evidence.trim() || !notice.text.includes(s.evidence))
157       return "Every scope row needs a verbatim supporting excerpt from the source text.";
158     const lines = s.evidence.split(/\r?\n/).filter((l) => l.trim());
159     const parsed = extractScope(s.evidence);
160     if (lines.length !== 1 || parsed.length !== 1)
161       return "Use one unambiguous labeled source line per scope row: Catalog: ABC; Lots: 123, 456. For
tables, append a staff-checked labeled transcription to the preserved source first.";
162     const pair = parsed[0];
163     if (
164       identifier(pair.catalog) !== identifier(s.catalog) ||
165       pair.allLots !== s.allLots
166     )
167       return "Declared scope must exactly match its single labeled source pair.";
168     const expected = pair.lots.map(identifier).sort();
169     const actual = s.lots.map(identifier).sort();
170     if (JSON.stringify(expected) !== JSON.stringify(actual))
171       return "Declared lots must match the complete lot list in the supporting source pair exactly.";
172   }
173   return null;
174 }
175 export function parseCsv(text: string): InventoryItem[] {
176   if (text.length > 500_000) throw new Error("CSV exceeds the 500 KB limit.");
177   const rows: string[][] = [];
178   let row: string[] = [],
179       field = "",
180       quoted = false,
181       closedQuote = false;
182   const source = text.replace(/\uFEFF/, "");
183   for (let i = 0; i < source.length; i++) {
184     const c = source[i];
185     if (closedQuote && c !== "," && c !== "\n" && c !== "\r")
186       throw new Error("Unexpected text after a closing CSV quote.");
187     if (c === "'") {
188       if (quoted && source[i + 1] === "'") {
189         field += "'";
190         i++;
191       } else if (quoted) {
192         quoted = false;
193         closedQuote = true;
194       } else if (field.length === 0) quoted = true;
195       else throw new Error("Invalid quote in CSV.");
196     } else if (c === "," && !quoted) {
197       row.push(field);
198       field = "";
199       closedQuote = false;
200     } else if ((c === "\n" || c === "\r") && !quoted) {
201       if (c === "\r" && source[i + 1] === "\n") i++;
202       row.push(field);
203       if (row.some((v) => v.trim())) rows.push(row);
204       row = [];
205       field = "";
206       closedQuote = false;
207     } else field += c;
208   }
209   if (quoted) throw new Error("CSV has an unclosed quoted field.");
210   row.push(field);
211   if (row.some((v) => v.trim())) rows.push(row);
212   if (rows.length < 2)
213     throw new Error("CSV needs a header and at least one inventory line.");
214   if (rows.length > 501)
215     throw new Error("Import up to 500 inventory lines per workspace.");
216   const headers = rows.shift()!.map((h) => h.trim().toLowerCase());
217   const required = [
218     "product",
219     "manufacturer",
220     "catalog",
221     "lot",
222     "quantity",
223     "location",
224   ];
225   if (new Set(headers).size !== headers.length)
226     throw new Error("CSV contains duplicate column names.");
227   for (const name of required)
228     if (!headers.includes(name))
229       throw new Error(
230         `Missing column: ${name}. Use the downloadable template.`
231       );
232   const seen = new Set<string>();
233   return rows.map((values, i) => {

```

lib/lotlight/engine.ts (continued)

```

234     if (values.length !== headers.length)
235       throw new Error(`Row ${i + 2}: column count does not match the header.`);
236     const get = (key: string) => (values[headers.indexOf(key)] ?? "").trim();
237     const quantity = Number(get("quantity"));
238     if (
239       !get("quantity") ||
240       !Number.isInteger(quantity) ||
241       quantity < 1 ||
242       quantity > 1_000_000
243     )
244       throw new Error(
245         `Row ${i + 2}: quantity must be a whole number from 1 to 1,000,000.`,
246       );
247     if (!get("product") || !get("location"))
248       throw new Error(`Row ${i + 2}: product and location are required.`);
249     if (values.some((v) => v.length > 500))
250       throw new Error(`Row ${i + 2}: a field exceeds 500 characters.`);
251     const key = inventoryKey({
252       product: get("product"),
253       manufacturer: get("manufacturer"),
254       catalog: get("catalog"),
255       lot: get("lot"),
256       location: get("location"),
257     });
258     if (seen.has(key))
259       throw new Error(
260         `Row ${i + 2}: duplicate inventory line. Combine quantities before importing.`,
261       );
262     seen.add(key);
263     return {
264       id: crypto.randomUUID(),
265       product: get("product"),
266       manufacturer: get("manufacturer"),
267       catalog: get("catalog"),
268       lot: get("lot"),
269       quantity,
270       location: get("location"),
271     };
272   });
273 }
274 export function csvCell(value: unknown): string {
275   let s = String(value ?? "");
276   if (/^[^\s]*[=+@-]/.test(s)) s = "'" + s;
277   return "'" + s.replace(/"/g, '"') + '"';
278 }
279 export function exportCsv(rows: unknown[][]): string {
280   return "\uFEFF" + rows.map((r) => r.map(csvCell).join(",")).join("\r\n");
281 }
282
283 export function inventoryKey(
284   item: Pick<
285     InventoryItem,
286     "product" | "manufacturer" | "catalog" | "lot" | "location"
287   >,
288 ) {
289   return JSON.stringify(
290     [
291       item.product,
292       item.manufacturer,
293       item.catalog,
294       item.lot,
295       item.location,
296     ].map(identifier),
297   );
298 }

```

lib/lotlight/files.ts

```

1 import { exportCsv, reconcile } from "./engine";
2 import { activeAction } from "./commands";
3 import type { Workspace } from "./types";
4 export function download(name: string, content: string, type = "text/plain") {
5   const url = URL.createObjectURL(new Blob([content], { type }));
6   const a = document.createElement("a");
7   a.href = url;
8   a.download = name;
9   document.body.appendChild(a);
10  a.click();
11  a.remove();
12  setTimeout(() => URL.revokeObjectURL(url), 1000);
13 }
14 export async function exportAudit(state: Workspace, saved = false) {
15   const payload = {
16     product: "Lotlight",
17     formatVersion: 1,
18     storageMode: saved ? "account workspace" : "unsaved in-memory demo",
19     exportedAt: new Date().toISOString(),
20     scope:
21       "Staff-recorded decision-support record. Not independent verification, current recall status, or
safety certification.",
22     inventoryProvenance: state.inventoryProvenance,
23     sourceProvenance: state.notices.some((n) => n.training)
24       ? "Contains historical training extracts."
25       : "User-supplied sources; not independently verified.",
26     workspace: state,
27   };
28   const text = JSON.stringify(payload);
29   const hash = Array.from(
30     new Uint8Array(
31       await crypto.subtle.digest("SHA-256", new TextEncoder().encode(text)),
32     ),
33   )
34     .map((b) => b.toString(16).padStart(2, "0"))
35     .join("");
36   download(
37     "lotlight-audit-record.json",
38     JSON.stringify(
39       {
40         payload,
41         integrity: {
42           algorithm: "SHA-256",
43           digest: hash,
44           note: "Digest of JSON.stringify(payload). Detects accidental changes; not a digital signature or
tamper-proof guarantee.",
45         },
46       },
47       null,
48       2,
49     ),
50     "application/json",
51   );
52 }
53 export function exportMatches(state: Workspace, saved = false) {
54   const rows: unknown[][] = [
55     [
56       "storage_mode",
57       "inventory_provenance",
58       "source_provenance",
59       "notice_date",
60       "exported_at",
61       "notice",
62       "notice_version",
63       "source_url",
64       "source_approved_by",
65       "inventory_version",
66       "product",
67       "manufacturer",
68       "catalog",
69       "lot",
70       "quantity",
71       "location",
72       "match",
73       "rationale",
74       "evidence",
75       "owner",
76       "status",
77       "handled_quantity",
78       "response_evidence",

```

lib/lotlight/files.ts (continued)

```

79     ],
80   ];
81   for (const notice of state.notices)
82     for (const match of reconcile(state.inventory, notice)) {
83       const a = activeAction(state, notice.id, match.item.id);
84       rows.push([
85         saved ? "account workspace" : "unsaved in-memory demo",
86         state.inventoryProvenance,
87         notice.training
88           ? "historical training extract"
89           : "user-supplied source",
90         notice.date,
91         new Date().toISOString(),
92         notice.title,
93         notice.version,
94         notice.sourceUrl,
95         notice.approvedBy ?? "unreviewed",
96         state.inventoryVersion,
97         match.item.product,
98         match.item.manufacturer,
99         match.item.catalog,
100        match.item.lot,
101        match.item.quantity,
102        match.item.location,
103        match.kind,
104        match.reason,
105        match.evidence,
106        a?.owner,
107        a?.status ?? "open",
108        a?.handledQuantity ?? 0,
109        a?.evidence,
110      ]);
111    }
112    download(
113      "lotlight-inventory-review.csv",
114      exportCsv(rows),
115      "text/csv;charset=utf-8",
116    );
117  }
118  export async function readNoticeFile(file: File): Promise<string> {
119    if (file.size > 5_000_000) throw new Error("Use a notice smaller than 5 MB.");
120    if (file.name.toLowerCase().endsWith(".txt")) return file.text();
121    if (!file.name.toLowerCase().endsWith(".pdf"))
122      throw new Error("Upload a text-based PDF or a .txt notice.");
123    const pdfjs = await import("pdfjs-dist");
124    const worker = await import("pdfjs-dist/build/pdf.worker.min.mjs?url");
125    pdfjs.GlobalWorkerOptions.workerSrc = worker.default;
126    const loading = pdfjs.getDocument({
127      data: await file.arrayBuffer(),
128      isEvalSupported: false,
129    });
130    const pdf = await loading.promise;
131    try {
132      if (pdf.numPages > 20)
133        throw new Error("Use a notice with 20 pages or fewer.");
134      let text = "";
135      for (let i = 1; i <= pdf.numPages; i++) {
136        const page = await pdf.getPage(i);
137        const content = await page.getTextContent();
138        text +=
139          content.items
140            .map((item) =>
141              "str" in item
142                ? item.str + ("hasEOL" in item && item.hasEOL ? "\n" : " ")
143                : "",
144            )
145            .join("") + "\n";
146        if (text.length > 50000)
147          throw new Error(
148            "Extracted text exceeds 50,000 characters. Upload the relevant notice pages.",
149          );
150      }
151      if (text.trim().length < 20)
152        throw new Error(
153          "This PDF appears to be scanned. OCR is not included; paste a verified transcription instead.",
154        );
155      return text;
156    } finally {
157      await pdf.destroy();
158    }

```

lib/lotlight/files.ts (continued)

159 }

lib/lotlight/sample.ts

```

1 import type { Workspace, Notice } from "../types";
2 export const BAXTER_URL =
3   "https://www.medline.com/media/assets/pdf/vendor-list/FA-2025-039-Customer-Letter-Final-Combined.pdf";
4 export const SAMPLE_NOTICE_TEXT = `TRAINING EXTRACT \u2014 paraphrased from Baxter manufacturer notice
   FA-2025-039, dated 2025-08-29. This is a historical demonstration, not a current safety notice. Consult the
   full original notice for every affected product and instruction. Only two catalog/lot pairs are included
   here.
5 Manufacturer: Baxter
6 Product: CLEARLINK IV extension and solution sets
7 Catalog: 2C8632; Lots: R25C31031
8 Catalog: 2R8858; Lots: R25A13024
9 Reason (paraphrase): A potential leak can compromise delivery and introduce risks during use.
10 Manufacturer wording: \u201cImmediately locate, isolate, and cease all use of the affected product.\u201d
11 Response (paraphrase): Identify stock, follow the supplier-specific return or replacement route, and
   acknowledge the notice. Direct-purchase and distributor-purchase response routes differ.
12 Source: ${BAXTER_URL}`;
13 export const SAMPLE_CSV = `product,manufacturer,catalog,lot,quantity,location
14 CLEARLINK extension set,Baxter,2C8632,R25C31031,24,Infusion room
15 CLEARLINK solution set,Baxter,2R8858,R25A13024,12,Main stockroom
16 CLEARLINK extension set,Baxter,2C8632,,8,Procedure room
17 IV line extender,Baxter,,,6,Mobile cart
18 CLEARLINK extension set,Baxter,2C8632,R24Z99999,18,Main stockroom
19 CLEARLINK solution set,Baxter,2R8858,R24Z88888,10,Infusion room
20 Sterile gauze,Demo Supply Co,GAUZE-4,G202601,40,Procedure room
21 Examination gloves,Demo Supply Co,GLV-M,G202602,100,Main stockroom`;
22 export const sampleNotice: Notice = {
23   id: "baxter-demo",
24   title: "CLEARLINK IV sets",
25   manufacturer: "Baxter",
26   date: "2025-08-29",
27   sourceUrl: BAXTER_URL,
28   text: SAMPLE_NOTICE_TEXT,
29   action:
30     "Identify affected stock and follow the manufacturer\u2019s supplier-specific isolation, return or
   replacement, and acknowledgement instructions. Verify the current full notice first.",
31   scope: [
32     {
33       catalog: "2C8632",
34       lots: ["R25C31031"],
35       allLots: false,
36       evidence: "Catalog: 2C8632; Lots: R25C31031",
37     },
38     {
39       catalog: "2R8858",
40       lots: ["R25A13024"],
41       allLots: false,
42       evidence: "Catalog: 2R8858; Lots: R25A13024",
43     },
44   ],
45   version: 1,
46   training: true,
47   approvedBy: null,
48   approvedAt: null,
49 };
50 export function initialWorkspace(): Workspace {
51   const lines = SAMPLE_CSV.split("\n").slice(1);
52   return {
53     inventoryProvenance: "synthetic",
54     noticeHistory: [],
55     inventoryHistory: [],
56     inventory: lines.map((line, i) => {
57       const [product, manufacturer, catalog, lot, quantity, location] =
58         line.split(",");
59       return {
60         id: `sample-${i + 1}`,
61         product,
62         manufacturer,
63         catalog,
64         lot,
65         quantity: Number(quantity),
66         location,
67       };
68     }),
69     inventoryVersion: 1,
70     notices: [structuredClone(sampleNotice)],
71     actions: [],
72     audit: [
73       {
74         id: "initial",
75         at: "2026-09-16T00:00:00.000Z",

```

lib/lotlight/sample.ts (continued)

```
76         actor: "Lotlight demo",
77         type: "workspace.created",
78         detail:
79           "Historical Baxter training extract loaded with synthetic inventory. No current recall status is
            implied.",
80       },
81     ],
82     revision: 0,
83     savedAt: null,
84   };
85 }
```

lib/lotlight/semantic-worker.ts

```

1  /// <reference lib="webworker" />
2  import {
3    env,
4    pipeline,
5    type FeatureExtractionPipeline,
6  } from "@huggingface/transformers";
7  env.allowLocalModels = false;
8  if (env.backends.onnx.wasm) {
9    env.backends.onnx.wasm.numThreads = 1;
10   env.backends.onnx.wasm.wasmPaths = new URL("/ai/", self.location.origin).href;
11 }
12 type Progress = { status: string; progress?: number };
13 const createExtractor = pipeline as unknown as (
14   task: string,
15   model: string,
16   options: Record<string, unknown>,
17 ) => Promise<FeatureExtractionPipeline>;
18 let extractor: Promise<FeatureExtractionPipeline> | undefined;
19 self.onmessage = async (
20   event: MessageEvent<{
21     query: string;
22     items: { id: string; product: string }[];
23   }>,
24 ) => {
25   try {
26     extractor ??= createExtractor(
27       "feature-extraction",
28       "Xenova/all-MiniLM-L6-v2",
29       {
30         revision: "751bff37182d3f1213fa05d7196b954e230abad9",
31         dtype: "q8",
32         device: "wasm",
33         progress_callback: (p: Progress) => {
34           if ("status" in p)
35             self.postMessage({
36               type: "progress",
37               message:
38                 p.status === "progress" && "progress" in p
39                 ? `Loading local AI \xb7 ${Math.round(p.progress as number)}%`
40                 : "Preparing local semantic model\u2026",
41             });
42         },
43       },
44     );
45     const model = await extractor;
46     const { query, items } = event.data;
47     self.postMessage({
48       type: "progress",
49       message: "Comparing product descriptions on your device\u2026",
50     });
51     const q = await model(query.slice(0, 500), {
52       pooling: "mean",
53       normalize: true,
54     });
55     const vector = Array.from(q.data as Float32Array);
56     const scores: Record<string, number> = {};
57     for (let i = 0; i < items.length; i += 16) {
58       const batch = items.slice(i, i + 16);
59       const embeddings = await model(
60         batch.map((item) => item.product.slice(0, 500)),
61         { pooling: "mean", normalize: true },
62       );
63       const vectors = embeddings.tolist() as number[][];
64       batch.forEach((item, j) => {
65         scores[item.id] = vectors[j].reduce(
66           (sum, value, k) => sum + value * vector[k],
67           0,
68         );
69       });
70       self.postMessage({
71         type: "progress",
72         message: `Compared ${Math.min(i + 16, items.length)} of ${items.length} inventory descriptions`,
73       });
74     }
75     self.postMessage({ type: "result", scores });
76   } catch (error) {
77     extractor = undefined;
78     self.postMessage({
79       type: "error",
80       message:

```

lib/lotlight/semantic-worker.ts (continued)

```
81         error instanceof Error ? error.message : "Semantic model unavailable",
82     });
83 }
84 };
```

lib/lotlight/storage.ts

```
1 import { env } from "cloudflare:workers";
2 import { initialWorkspace } from "./sample";
3 import type { Workspace } from "./types";
4 export function database() {
5   if (!env.DB)
6     throw new Error(
7       "Persistent storage is temporarily unavailable. Please retry.",
8     );
9   return env.DB;
10 }
11 export async function readWorkspace(ownerId: string): Promise<Workspace> {
12   const db = database();
13   const row = await db
14     .prepare("SELECT state FROM workspaces WHERE owner_id = ?")
15     .bind(ownerId)
16     .first<{ state: string }>();
17   if (row) return JSON.parse(row.state) as Workspace;
18   const state = initialWorkspace();
19   await db
20     .prepare(
21       "INSERT OR IGNORE INTO workspaces (owner_id, state, revision, updated_at) VALUES (?, ?, ?, ?)",
22     )
23     .bind(ownerId, JSON.stringify(state), 0, new Date().toISOString())
24     .run();
25   const saved = await db
26     .prepare("SELECT state FROM workspaces WHERE owner_id = ?")
27     .bind(ownerId)
28     .first<{ state: string }>();
29   if (!saved)
30     throw new Error("Could not initialize your workspace. Please retry.");
31   return JSON.parse(saved.state);
32 }
33 export async function writeWorkspace(
34   ownerId: string,
35   expectedRevision: number,
36   state: Workspace,
37 ) {
38   const result = await database()
39     .prepare(
40       "UPDATE workspaces SET state = ?, revision = ?, updated_at = ? WHERE owner_id = ? AND revision = ?",
41     )
42     .bind(
43       JSON.stringify(state),
44       state.revision,
45       state.savedAt,
46       ownerId,
47       expectedRevision,
48     )
49     .run();
50   return result.meta.changes === 1;
51 }
```

lib/lotlight/types.ts

```
1 export type InventoryItem = {
2   id: string;
3   version?: number;
4   product: string;
5   manufacturer: string;
6   catalog: string;
7   lot: string;
8   quantity: number;
9   location: string;
10 };
11 export type Scope = {
12   catalog: string;
13   lots: string[];
14   allLots: boolean;
15   evidence: string;
16 };
17 export type Notice = {
18   id: string;
19   title: string;
20   manufacturer: string;
21   date: string;
22   sourceUrl: string;
23   text: string;
24   action: string;
25   scope: Scope[];
26   version: number;
27   training: boolean;
28   approvedBy: string | null;
29   approvedAt: string | null;
30 };
31 export type MatchKind = "exact" | "missing" | "possible" | "unlisted";
32 export type Match = {
33   item: InventoryItem;
34   kind: MatchKind;
35   reason: string;
36   evidence: string;
37   similarity?: number;
38 };
39 export type ResponseAction = {
40   id: string;
41   noticeId: string;
42   noticeVersion: number;
43   itemId: string;
44   inventoryVersion: number;
45   itemVersion: number;
46   itemLabel: string;
47   owner: string;
48   dueDate: string;
49   status: "assigned" | "verified" | "completed";
50   verification: string;
51   evidence: string;
52   handledQuantity: number;
53   updatedAt: string;
54   updatedBy: string;
55 };
56 export type AuditEvent = {
57   id: string;
58   at: string;
59   actor: string;
60   type: string;
61   detail: string;
62 };
63 export type Workspace = {
64   inventory: InventoryItem[];
65   inventoryProvenance: "synthetic" | "user-supplied";
66   noticeHistory: Notice[];
67   inventoryHistory: {
68     version: number;
69     items: InventoryItem[];
70     provenance: string;
71   }[];
72   inventoryVersion: number;
73   notices: Notice[];
74   actions: ResponseAction[];
75   audit: AuditEvent[];
76   revision: number;
77   savedAt: string | null;
78 };
79 export type UserInfo = { name: string; email: string } | null;
```

package.json

```

1 {
2   "name": "lotlight",
3   "version": "1.0.0",
4   "private": true,
5   "engines": {
6     "node": ">=22.13.0"
7   },
8   "scripts": {
9     "install:ci": "node scripts/install-ci.mjs",
10    "dev": "node scripts/run-framework.mjs dev",
11    "build": "node scripts/run-framework.mjs build",
12    "start": "node --import ./scripts/sites-env.mjs ./node_modules/wrangler/bin/wrangler.js dev --config
dist/server/wrangler.json --local --persist-to .wrangler/state --ip 127.0.0.1 --inspector-port 0",
13    "lint": "eslint . --ignore-pattern dist --ignore-pattern .next",
14    "db:generate": "drizzle-kit generate",
15    "test": "tsx --test tests/engine.test.ts",
16    "typecheck": "tsc --noEmit",
17    "test:e2e": "playwright test",
18    "predev": "node scripts/build-ai.mjs",
19    "prebuild": "node scripts/build-ai.mjs",
20    "build:firebase": "node scripts/build-ai.mjs && vite build --config vite.firebase.config.ts",
21    "dev:firebase": "vite --config vite.firebase.config.ts",
22    "build:api": "node scripts/build-api.mjs"
23  },
24  "dependencies": {
25    "@base-ui/react": "^1.7.0",
26    "@hookform/resolvers": "^5.7.1",
27    "@huggingface/transformers": "3.8.1",
28    "@shadcn/react": "^0.3.0",
29    "class-variance-authority": "0.7.1",
30    "clsx": "2.1.1",
31    "cmdk": "1.1.1",
32    "date-fns": "^4.4.0",
33    "drizzle-orm": "0.45.2",
34    "embla-carousel-react": "^8.6.0",
35    "express": "15.2.1",
36    "firebase": "12.9.0",
37    "firebase-admin": "14.4.0",
38    "input-otp": "1.4.2",
39    "lucide-react": "1.31.0",
40    "next": "16.3.4",
41    "next-themes": "10.4.6",
42    "pdfjs-dist": "5.4.296",
43    "radix-ui": "1.6.7",
44    "react": "19.2.8",
45    "react-day-picker": "10.0.1",
46    "react-dom": "19.2.8",
47    "react-hook-form": "7.85.0",
48    "react-resizable-panels": "4.12.2",
49    "recharts": "3.8.0",
50    "sonner": "2.0.8",
51    "tailwind-merge": "3.6.0",
52    "vaul": "1.1.2",
53    "zod": "3.25.76"
54  },
55  "devDependencies": {
56    "@axe-core/playwright": "4.11.0",
57    "@cloudflare/vite-plugin": "1.54.10",
58    "@cloudflare/workers-types": "5.20260916.1",
59    "@playwright/test": "1.58.2",
60    "@tailwindcss/postcss": "4.2.1",
61    "@types/express": "15.0.6",
62    "@types/node": "22.19.19",
63    "@types/react": "19.2.14",
64    "@types/react-dom": "19.2.3",
65    "@vitejs/plugin-react": "6.0.2",
66    "@vitejs/plugin-rsc": "10.5.34",
67    "drizzle-kit": "0.31.10",
68    "esbuild": "10.28.2",
69    "eslint": "9.39.4",
70    "eslint-config-next": "16.3.4",
71    "prettier": "3.6.2",
72    "react-server-dom-webpack": "19.2.8",
73    "tailwindcss": "4.2.1",
74    "tsx": "4.23.13",
75    "tw-animate-css": "1.4.0",
76    "typescript": "5.9.3",
77    "vinext": "1.0.0-beta.10",
78    "vite": "8.3.0",
79    "wrangler": "4.132.0"

```

package.json (continued)

```
80 },
81 "type": "module",
82 "overrides": {
83   "miniflare": {
84     "sharp": "0.35.4"
85   },
86   "@huggingface/transformers": {
87     "sharp": "0.35.4"
88   },
89   "gaxios": {
90     "uuid": "11.1.1"
91   },
92   "@esbuild-kit/core-utils": {
93     "esbuild": "0.28.2"
94   }
95 }
96 }
```

playwright.config.ts

```
1 import { defineConfig } from "@playwright/test";
2 export default defineConfig({
3   testDir: "tests/e2e",
4   timeout: 45000,
5   fullyParallel: false,
6   workers: 1,
7   retries: 0,
8   reporter: [
9     ["list"],
10    ["json", { outputFile: "test-results/browser-results.json" }],
11  ],
12  use: {
13    baseURL: process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174",
14    headless: true,
15    viewport: { width: 1440, height: 1000 },
16    screenshot: "only-on-failure",
17    trace: "retain-on-failure",
18  },
19 });
```

scripts/assemble-narration.py

```
1 from pathlib import Path
2 import json, subprocess
3 ROOT=Path(__file__).resolve().parents[1]
4 ff='/opt/homebrew/bin/ffmpeg'
5 scenes=json.loads((ROOT/'submission/NARRATION-SCENES.json').read_text())
6 parts=[]
7 for i,s in enumerate(scenes):
8     p=ROOT/f'.artifact-build/narration/{i:02d}.wav'
9     subprocess.run([ff, '-y', '-hide_banner', '-loglevel', 'error', '-i', str(ROOT/f'.artifact-
    build/narration/{i:02d}.mp3'), '-af', 'apad=pad_dur=0.5', '-ar', '48000', '-ac', '1', str(p)], check=True)
10    parts.append("file "+str(p)+"")
11    lst=ROOT/'.artifact-build/narration/concat.txt';lst.write_text('\n'.join(parts))
12    subprocess.run([ff, '-y', '-hide_banner', '-loglevel', 'error', '-f', 'concat', '-safe', '0', '-i', str(lst), '-
    c:a', 'pcm_s16le', str(ROOT/'submission/Lotlight-Narration.wav')], check=True)
13 print('Assembled full narration')
```

scripts/build-ai.mjs

```
1 import { build } from "esbuild";
2 import { mkdir, copyFile } from "node:fs/promises";
3 await mkdir("public/ai", { recursive: true });
4 await build({
5   entryPoints: ["lib/lotlight/semantic-worker.ts"],
6   outfile: "public/ai/semantic-worker.js",
7   bundle: true,
8   platform: "browser",
9   format: "esm",
10  target: "es2022",
11  minify: true,
12  legalComments: "eof",
13 });
14 for (const file of [
15   "ort-wasm-simd-threaded.jsep.mjs",
16   "ort-wasm-simd-threaded.jsep.wasm",
17 ])
18   await copyFile(
19     "node_modules/onnxruntime-web/dist/" + file,
20     "public/ai/" + file,
21   );
22 console.log("Built isolated browser AI worker and local WASM runtime.");
```

scripts/build-api.mjs

```

1 import {build} from 'esbuild';
2 import {mkdir,writeFile,readFile} from 'node:fs/promises';
3 await mkdir('deploy-api',{recursive:true});
4 await build({entryPoints:['server/index.ts'],outfile:'deploy-
  api/index.mjs',bundle:true,platform:'node',target:'node22',format:'esm',packages:'external'});
5 const pkg=JSON.parse(await readFile('package.json','utf8'));
6 await writeFile('deploy-api/package.json',JSON.stringify({name:'lotlight-
  api',version:'1.0.0',type:'module',private:true,engines:{node:'22'},scripts:{start:'node
  index.mjs'},dependencies:Object.fromEntries(['express','firebase-
  admin','zod'].map(k=>[k,pkg.dependencies[k]])),overrides:{gaxios:pkg.overrides.gaxios}},null,2));
7 await writeFile('deploy-api/Dockerfile','FROM node:22-slim\nWORKDIR /app\nCOPY package*.json ./\nRUN npm ci
  --omit=dev\nCOPY index.mjs ./\nENV NODE_ENV=production\nUSER node\nCMD ["node", "index.mjs"]\n');

```

scripts/build-verified.sh

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5
6 if [[ "${SITES_ENV_READY:-}" != "1" ]]; then
7     exec "${script_dir}/sites-env.sh" -- "$@" "$@"
8 fi
9
10 command -v timeout || {
11     echo "build-verified.sh requires GNU timeout." >&2
12     exit 69
13 }
14
15 vinext="${SITES_PROJECT_ROOT}/node_modules/.bin/vinext"
16 if [[ ! -x "${vinext}" ]]; then
17     echo "vinext is unavailable. Run npm run install:ci and wait for it to finish before building." >&2
18     exit 69
19 fi
20
21 echo "Running bounded vinext build..."
22 timeout \
23     --signal=TERM \
24     --kill-after="${SITES_BUILD_KILL_AFTER:-10s}" \
25     "${SITES_BUILD_TIMEOUT:-3m}" \
26     "${vinext}" build
```

scripts/check-accessibility.mjs

```

1 import { chromium } from "playwright";
2 import AxeBuilder from "@axe-core/playwright";
3 const b = await chromium.launch({ headless: true });
4 const context = await b.newContext({ viewport: { width: 1440, height: 1000 } });
5 const p = await context.newPage();
6 await p.goto(process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174");
7 await p.locator('main[data-ready="true"]').waitFor();
8 const result = await new AxeBuilder({ page: p })
9   .withTags(["wcag2a", "wcag2aa"])
10  .analyze();
11 console.log(
12   JSON.stringify(
13     result.violations.map((v) => ({
14       id: v.id,
15       impact: v.impact,
16       nodes: v.nodes
17         .map((n) => ({ target: n.target, summary: n.failureSummary }))
18         .slice(0, 25),
19     })),
20     null,
21     2,
22   ),
23 );
24 await b.close();

```

scripts/check-api-isolation.mjs

```
1 import assert from 'node:assert/strict';
2 import fs from 'node:fs/promises';
3 const url=process.env.LOTLIGHT_BUILT_URL??'http://127.0.0.1:5185';
4 const suffix=crypto.randomUUID();const a='lotlight-test-a-'+suffix,b='lotlight-test-b-'+suffix;
5 const headers=user=>({'oai-authenticated-user-id':user,'oai-authenticated-user-email':user+'@example.invalid','Content-Type':'application/json',Origin:url});
6 const get=async u=>(await fetch(url+'/api/workspace',{headers:headers(u)})).json();
7 const post=async(u,s,command)=>fetch(url+'/api/workspace',{method:'POST',headers:headers(u),body:JSON.stringify({revision:s.revision,command})});
8 assert.equal((await fetch(url+'/api/workspace')).status,401);
9 let sa=await get(a),sb=await get(b);assert.equal(sa.revision,0);assert.equal(sb.revision,0);
10 const source={title:'Isolation test',manufacturer:'Test Supplies',date:'2026-09-16',sourceUrl:'https://example.org/notice',text:'Test Supplies\nCatalog: TEST; Lots: LOT1',scope:[{catalog:'TEST',lots:['LOT1'],allLots:false,evidence:'Catalog: TEST; Lots: LOT1'}],action:'Synthetic source response used only for database isolation testing.',training:true};
11 assert.equal((await post(a,sa,{type:'save_notice',notice:source})).status,200);sa=await get(a);const created=sa.notices.find(n=>n.title==='Isolation test');assert.ok(created);sb=await get(b);assert.equal(sb.notices.length,1);assert.equal(sb.revision,0);
12 assert.equal((await post(b,sb,{type:'approve_notice',noticeId:created.id,sourceConfirmed:true})).status,400);
13 assert.equal((await post(a,{revision:0},{type:'reset_demo'})).status,409);
14 assert.equal((await fetch(url+'/api/workspace',{method:'POST',headers:{...headers(a),Origin:'https://evil.in.valid'},body:JSON.stringify({revision:sa.revision,command:{type:'reset_demo'}})})).status,403);
15 assert.equal((await post(a,sa,{type:'approve_notice',noticeId:created.id,sourceConfirmed:true,approvedBy:'Forged actor'})).status,400);
16 await post(a,sa,{type:'clear_workspace'});await post(b,sb,{type:'clear_workspace'});
17 const result={date:new Date().toISOString(),checks:7,status:'passed',environment:'Built Worker on loopback with test-only trusted gateway headers',checksRun:['anonymous denied','separate account initialization','account A mutation invisible to B','cross-account notice mutation denied','stale revision denied','foreign origin denied','forged actor denied'],limitation:'Checks application tenant isolation. Does not validate the external Sites header-stripping boundary or full OAuth flow.'};await fs.writeFile('docs/api-isolation-results.json',JSON.stringify(result,null,2)+'\n');console.log(result);
```

scripts/create-narration.mjs

```
1 import fs from 'node:fs/promises';
2 const scenes=JSON.parse(await fs.readFile('submission/NARRATION-SCENES.json','utf8'));
3 await fs.mkdir('.artifact-build/narration',{recursive:true});
4 for(let i=0;i<scenes.length;i++){
5   const path=`.artifact-build/narration/${String(i).padStart(2,'0')}.mp3`;
6   try{await fs.access(path);continue}catch{
7     const r=await fetch('https://api.openai.com/v1/audio/speech',{method:'POST',headers:{Authorization:`Bearer ${process.env.OPENAI_API_KEY}`, 'Content-Type':'application/json'},body:JSON.stringify({model:'gpt-4o-mini-tts',voice:'cedar',response_format:'mp3',instructions:'Narrate a concise product demonstration in clear, warm, conversational English. Calm and confident, with natural pauses and precise diction. About 145 words per minute. Do not add words or sounds. Pronounce Lotlight as lot light. This is a neutral narrator, not an imitation of any person.',input:scenes[i].text})});
8     if(!r.ok)throw new Error(`Speech request failed (${r.status}): ${r.text().slice(0,300)}`);
9     await fs.writeFile(path,Buffer.from(await r.arrayBuffer()));console.log(`Narration ${i+1}/${scenes.length} saved`);
10 }
```

scripts/create-submission-pdfs.py

```

1 from reportlab.pdfgen import canvas
2 from reportlab.lib.colors import HexColor, Color, white
3 from reportlab.lib.pagesizes import A4
4 from reportlab.platypus import Paragraph
5 from reportlab.lib.styles import ParagraphStyle
6 from pathlib import Path
7 from xml.sax.saxutils import escape
8 import textwrap
9 ROOT=Path(__file__).resolve().parents[1]
10 OUT=ROOT/'submission';OUT.mkdir(exist_ok=True)
11 W,H=A4
12 ink=HexColor('#263329');muted=HexColor('#65745b');lime=HexColor('#ddec8b');paper=HexColor('#f8f9f5');orange=
HexColor('#b55c36')
13 def para(c,text,x,y,w,size=10,leading=15,color=ink,bold=False):
14     s=ParagraphStyle('p',fontName='Helvetica-Bold' if bold else
'Helvetica',fontSize=size,leading=leading,textColor=color)
15     p=Paragraph(text,s);_,h=p.wrap(w,H);p.drawOn(c,x,y-h);return y-h
16 c=canvas.Canvas(str(OUT/'Lotlight-One-Page.pdf'),pagesize=A4);c.setTitle('Lotlight | One-page project
description');c.setAuthor('Shivam Gupta')
17 c.setFill(paper);c.rect(0,0,W,H,fill=1,stroke=0)
18 c.setFill(ink);c.setFont('Helvetica-Bold',9);c.drawString(42,H-42,'DSH HACKS V2 / AI x
HEALTHCARE');c.setFont('Helvetica',9);c.drawRightString(W-42,H-42,'Shivam Gupta')
19 c.setFont('Helvetica-Bold',45);c.drawString(40,H-105,'lotlight');c.setFill(lime);c.roundRect(W-117,H-
109,75,45,10,fill=1,stroke=0);c.setFill(ink);c.setFont('Helvetica-
Bold',9);c.drawCentredString(W-79.5,H-90,'NOTICE');c.drawCentredString(W-79.5,H-102,'TO RESPONSE')
20 y=para(c,'A notice is only the beginning.',42,H-127,W-84,23,28)
21 y=para(c,'Lotlight helps independent clinics reconcile a medical recall notice with spreadsheet inventory,
resolve uncertain identifiers, and document a response someone owns.',42,y-14,W-84,11,16)
22 y-=20
23 c.setStrokeColor(HexColor('#dce2d4'));c.line(42,y,W-42,y)
24 y-=21
25 left=42;right=310;col=243
26 para(c,'THE PROBLEM',left,y,col,8.5,12,muted,True);para(c,'THE WORKING
PRODUCT',right,y,col,8.5,12,muted,True)
27 y1=para(c,'A recall letter is not a completed stock check. Product names vary, lot fields are missing, and
responsibility can get lost between a notice and a shelf.',left,y-22,col,10,15)
28 y2=para(c,'Import a notice and CSV. Compare each catalog/lot pair with source evidence. Confirm the source,
assign an owner, record quantities handled, and export the audit.',right,y-22,col,10,15)
29 y=min(y1,y2)-22
30 c.setFill(HexColor('#edf2df'));c.roundRect(42,y-76,W-84,76,8,fill=1,stroke=0)
31 para(c,'A REAL NOTICE. A CLEARLY SYNTHETIC CLINIC.',56,y-13,W-112,8,11,muted,True)
32 para(c,'The demo uses two pairs from Baxter notice FA-2025-039 (29 Aug 2025). Eight fictional stock lines
contain two exact matches, two incomplete records, and four lines outside the entered scope. No patient data
or current recall status is implied.',56,y-32,W-112,9.5,14)
33 y-=101
34 para(c,'AI WITH A BOUNDARY',left,y,col,8.5,12,muted,True);para(c,'ENGINEERING THAT
MATTERS',right,y,col,8.5,12,muted,True)
35 y1=para(c,'A pinned MiniLM model compares descriptions locally in a browser worker, without paid inference
APIs. AI can surface aliases. Only deterministic identifier rules establish matches; a person confirms the
source and response.',left,y-22,col,10,15)
36 y2=para(c,'Private sign-in and database persistence. Server-validated actions. Versioned source and
inventory snapshots. Conflicting-save protection. Unknown identifiers cannot close; partial quantities stay
open.',right,y-22,col,10,15)
37 y=min(y1,y2)-25
38 para(c,'COMMERCIAL HYPOTHESIS',left,y,W-84,8.5,12,muted,True)
39 y=para(c,'Start with independent clinics using spreadsheet stock records. Test <b>$149/site/month</b>: at an
assumed $30/hour staff cost, five hours saved covers the fee. Existing enterprise competitors validate the
category; clinic demand and savings still need a supervised pilot.',left,y-22,W-84,10,15)
40 y-=20
41 para(c,'VALIDATION & LIMITS',left,y,W-84,8.5,12,muted,True)
42 y=para(c,'Automated tests cover scope pairing, missing identifiers, workflow gates, import validation and
version changes. Browser tests exercise sign-in, persistence, response completion and exports. This is a
hackathon MVP, not clinically validated or approved for unsupervised clinical use. Shared team roles, OCR
and complex recall scope remain future work.',left,y-20,W-84,9,13)
43 c.setStrokeColor(HexColor('#dce2d4'));c.line(42,88,W-42,88)
44 para(c,'APP <link href="https://lotlight-care.web.app" color="#455d37">lotlight-care.web.app</link> CODE
<link href="https://github.com/shi1720/DSH-Hacks-V2" color="#455d37">github.com/shi1720/DSH-
Hacks-V2</link>',42,75,W-84,8.5,12)
45 para(c,'SOURCES Baxter FA-2025-039 (Medline-hosted manufacturer PDF); FDA, What is a Medical Device Recall?
Full source links and limitations are in the repository.',42,57,W-84,7.5,10,muted)
46 para(c,'Project creator: Shivam Gupta. AI-assisted research, implementation, testing and presentation
preparation.',42,30,W-84,7,9,muted)
47 c.save()
48 # Authored code listing; vendor libraries and lockfiles remain in the repository.
49 files=[]
50 for prefix in ['app','lib/lotlight','db','tests','server','firebase']:
51     for p in (ROOT/prefix).rglob('*'):
52         if p.suffix in ['.ts','.tsx','.css','.mjs','.html'] and p.is_file():files.append(p)
53 for name in ['scripts/build-ai.mjs','scripts/evaluate-ai.mjs','scripts/create-submission-pdfs.py','playwrig
t.config.ts','vite.config.ts','package.json','firebase.json','firebase.rules','vite.firebase.

```

scripts/create-submission-pdfs.py (continued)

```

    config.ts', 'drizzle/0000_real_hellcat.sql']:
54 p=ROOT/name
55 if p.exists():files.append(p)
56 files += [p for p in (ROOT/'scripts').glob('*') if p.suffix in ['.mjs','.py','.sh']]
57 files += list((ROOT/'github').rglob('*.yaml'))
58 files=sorted(set(files))
59 c=canvas.Canvas(str(OUT/'Lotlight-Source-Code.pdf'),pagesize=A4);c.setTitle('Lotlight | Authored source
    listing');c.setAuthor('Shivam Gupta');page=0
60 def newpage(title):
61     global page
62     if page:c.showPage()
63     page+=1;c.setFillColor(paper);c.rect(0,0,W,H,fill=1,stroke=0);c.setFillColor(ink);c.setFont('Helvetica-
    Bold',11);c.drawString(35,H-33,'LOTLIGHT / SOURCE
    RECORD');c.setFont('Helvetica',7);c.drawRightString(W-35,H-33,str(page));c.setFont('Helvetica-Bold',8);c.dra
    wString(35,H-56,title[:105]);c.setFont('Helvetica',7);c.setFillColor(muted);c.drawString(35,24,'Shivam Gupta
    \xb7 DSH Hacks V2 \xb7 AI-assisted development \xb7 Dependencies and full history in GitHub');return H-77
64 y=newpage('Contents and provenance')
65 y=para(c,'This listing includes application code, matching/workflow logic, database schema, tests and
    authored build/evaluation scripts. Vendored UI primitives, build scaffolding and package lockfiles are
    available in the repository. The MIT license applies to original code; dependencies retain their own
    licenses.',35,y,W-70,10,15)-20
66 for p in files:
67     if y<55:y=newpage('Contents (continued)')
68     c.setFont('Helvetica',8);c.setFillColor(ink);c.drawString(35,y,str(p.relative_to(ROOT)));y-=13
69     for p in files:
70         y=newpage(str(p.relative_to(ROOT)))
71         for num,line in enumerate(p.read_text().splitlines(),1):
72             line=line.expandtabs(2).encode('ascii','backslashreplace').decode('ascii')
73             chunks=textwrap.wrap(line,width=108,replace_whitespace=False,drop_whitespace=False) or ['']
74             for j,chunk in enumerate(chunks):
75                 if y<45:y=newpage(str(p.relative_to(ROOT))+ ' (continued)')
76                 c.setFillColor(muted if j else ink);c.setFont('Courier',7);c.drawString(35,y,(f'{num:4} ' if j==0 else '
                    ')+chunk);y-=9
77 c.save()
78 print('Created one-page description and authored source listing:',len(files),'files,',page,'code pages')
```

scripts/deploy-firebase.sh

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 cd "$(dirname "$0")/.."
4 FIREBASE_PROJECT="${FIREBASE_PROJECT:-lotlight-care}"
5 CLOUD_PROJECT="${CLOUD_PROJECT:-gen-lang-client-0444960702}"
6 REGION="${REGION:-us-central1}"
7 GCLOUD="${GCLOUD:-gcloud}"
8 npm run typecheck
9 npm test
10 npm run build:api
11 npm install --package-lock-only --prefix deploy-api
12 "$GCLOUD" run deploy lotlight-api --project="$CLOUD_PROJECT" --region="$REGION" --source=deploy-api --clear-
base-image --service-account="lotlight-api@$CLOUD_PROJECT.iam.gserviceaccount.com" --set-env-
vars="FIREBASE_PROJECT_ID=$FIREBASE_PROJECT,NODE_ENV=production" --allow-unauthenticated --min-instances=0
--max-instances=1 --memory=512Mi --cpu=1 --concurrency=40 --timeout=60 --quiet
13 LOTLIGHT_API_URL="$("$GCLOUD" run services describe lotlight-api --project="$CLOUD_PROJECT"
--region="$REGION" --format='value(status.url)')
14 export LOTLIGHT_API_URL
15 node --input-type=module -e 'import fs from "node:fs"; const p="public/firebase-config.json"; const c=JSON.p
arse(fs.readFileSync(p,"utf8"));c.apiBase=process.env.LOTLIGHT_API_URL;fs.writeFileSync(p,JSON.stringify(c,n
ull,2)+"\n");'
16 node scripts/prepare-public-artifacts.mjs
17 npm run build:firebase
18 npx --yes firebase-tools@15.7.0 deploy --only hosting,firestore:rules --project="$FIREBASE_PROJECT" --non-
interactive
```

scripts/evaluate-ai.mjs

```

1 import { chromium } from "playwright";
2 import fs from "node:fs/promises";
3 const context = await chromium.launchPersistentContext(
4   "/tmp/lotlight-ai-validation",
5   { headless: true, viewport: { width: 1440, height: 1000 } },
6 );
7 const page = context.pages()[0];
8 page.on('console', m=>console.log(m.type(),m.text().slice(0,200)));
9 page.on('requestfailed', r=>console.log('request failed',r.url(),r.failure()));
10 try {
11   await page.goto(process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174");
12   await page.locator('main[data-ready="true"]').waitFor();
13   const query = "Baxter CLEARLINK IV extension and solution sets";
14   const cases = [
15     { id: "alias-1", product: "IV line extender", relevant: true },
16     {
17       id: "alias-2",
18       product: "Intravenous fluid administration tubing",
19       relevant: true,
20     },
21     {
22       id: "alias-3",
23       product: "Extension tubing for infusion line",
24       relevant: true,
25     },
26     {
27       id: "alias-4",
28       product: "IV solution administration set",
29       relevant: true,
30     },
31     { id: "literal-1", product: "CLEARLINK extension set", relevant: true },
32     { id: "literal-2", product: "CLEARLINK solution set", relevant: true },
33     { id: "negative-1", product: "Examination gloves", relevant: false },
34     { id: "negative-2", product: "Sterile gauze dressing", relevant: false },
35     { id: "negative-3", product: "Digital thermometer", relevant: false },
36     { id: "negative-4", product: "Wheelchair footrest", relevant: false },
37     { id: "negative-5", product: "Blood glucose test strips", relevant: false },
38     { id: "negative-6", product: "Urine collection cup", relevant: false },
39   ];
40   const start = Date.now();
41   const scores = await page.evaluate(
42     ({ query, cases }) => {
43       new Promise((resolve, reject) => {
44         const w = new Worker("/ai/semantic-worker.js", { type: "module" });
45         const timeout = setTimeout(() => {
46           w.terminate();
47           reject(new Error("Model evaluation exceeded 180 seconds"));
48         }, 180000);
49         w.onmessage = (e) => {
50
51           if (e.data.type === "result") {
52             clearTimeout(timeout);
53             w.terminate();
54             resolve(e.data.scores);
55           }
56           if (e.data.type === "error") {
57             clearTimeout(timeout);
58             w.terminate();
59             reject(new Error(e.data.message));
60           }
61         };
62         w.onerror = (e) => {
63           clearTimeout(timeout);
64           reject(new Error(e.message));
65         };
66         w.postMessage({ query, items: cases });
67       });
68     }, { query, cases },
69   );
70   const ranked = cases
71     .map((c) => ({
72       ...c,
73       similarity: scores[c.id],
74       lexical: query
75         .toLowerCase()
76         .split(/\W+/)
77         .filter((w) => w.length > 3)
78         .some((w) => c.product.toLowerCase().includes(w)),
79     }))
80     .sort((a, b) => b.similarity - a.similarity);

```

scripts/evaluate-ai.mjs (continued)

```
81 const relevant = cases.filter((c) => c.relevant).length;
82 const report = {
83   date: new Date().toISOString(),
84   model: "Xenova/all-MiniLM-L6-v2",
85   revision: "751bfff37182d3f1213fa05d7196b954e230abad9",
86   dtype: "q8",
87   runtime: "actual Chromium browser / WASM",
88   query,
89   total: cases.length,
90   relevant,
91   elapsedIncludingModelLoadMs: Date.now() - start,
92   lexicalRecall:
93     ranked.filter((c) => c.relevant && c.lexical).length / relevant,
94   semanticTop6Recall:
95     ranked.slice(0, 6).filter((c) => c.relevant).length / relevant,
96   candidateThreshold: 0.3,
97   thresholdRelevant: ranked.filter((c) => c.relevant && c.similarity >= 0.3)
98     .length,
99   thresholdFalseCandidates: ranked.filter(
100     (c) => !c.relevant && c.similarity >= 0.3,
101   ).length,
102   limitations:
103     "Hand-authored synthetic smoke/ranking set, one query. Not clinical validation or an unbiased held-out
benchmark. Labels and threshold not independently adjudicated.",
104   ranked,
105 };
106 await fs.writeFile(
107   "docs/ai-evaluation.json",
108   JSON.stringify(report, null, 2) + "\n",
109 );
110 console.log(JSON.stringify(report, null, 2));
111 await page.getByRole("button", { name: "Open review", exact: true }).click();
112 await page.getByRole("button", { name: "Run local AI", exact: true }).click();
113 await page
114   .getByText(/Semantic comparison complete/)
115   .waitFor({ timeout: 120000 });
116 await page.screenshot({ path: "docs/images/ai-review.png", fullPage: true });
117 } finally {
118   await context.close();
119 }
```

scripts/execution-profile.mjs

```
1 import { readFileSync } from "node:fs";
2
3 export function readExecutionProfile() {
4   let settings;
5   try {
6     settings = JSON.parse(readFileSync(new URL("../.sites-runtime/execution-profile.json", import.meta.url),
7     "utf8"));
8   } catch (error) {
9     // Clean clones and remote builds have no checkout-local selection.
10    if (error.code === "ENOENT") return "portable";
11    throw error;
12  }
13  if (!["managed-linux", "portable"].includes(settings?.executionProfile)) {
14    throw new Error("Invalid local execution profile; rerun the Sites plugin's configure-execution-
15    profile.mjs.");
16  }
17  return settings.executionProfile;
18 }
```

scripts/install-ci.mjs

```
1 import { spawnSync } from "node:child_process";
2 import { accessSync, constants } from "node:fs";
3 import path from "node:path";
4 import { projectRoot } from "../sites-env.mjs";
5 import { readExecutionProfile } from "../execution-profile.mjs";
6
7 if (!process.env.npm_execpath) {
8   throw new Error("Run this installer with npm run install:ci.");
9 }
10
11 if (![
12   "SHARP_IGNORE_GLOBAL_LIBVIPS",
13   "SHARP_FORCE_GLOBAL_LIBVIPS",
14   "npm_config_build_from_source",
15   "NPM_CONFIG_BUILD_FROM_SOURCE",
16 ].some((key) => key in process.env)) {
17   process.env.SHARP_IGNORE_GLOBAL_LIBVIPS = "1";
18 }
19
20 if (readExecutionProfile() === "managed-linux") {
21   const result = spawnSync("bash", [path.join(projectRoot, "scripts/install-ci.sh")], {
22     stdio: "inherit",
23   });
24   if (result.error) throw result.error;
25   process.exit(result.status ?? 1);
26 }
27
28 // Invoke npm's JavaScript entrypoint, avoiding platform-specific shell shims.
29 const installed = spawnSync(
30   process.execPath,
31   [
32     process.env.npm_execpath, "ci", "--prefix", projectRoot, "--workspaces=false",
33     "--include=dev", "--include=optional", "--prefer-offline", "--no-audit", "--no-fund",
34   ],
35   { stdio: "inherit" },
36 );
37 if (installed.error) throw installed.error;
38 if (installed.status !== 0) process.exit(installed.status ?? 1);
39
40 try {
41   accessSync(
42     path.join(
43       projectRoot, "node_modules", ".bin",
44       process.platform === "win32" ? "vinext.cmd" : "vinext",
45     ),
46     process.platform === "win32" ? constants.F_OK : constants.X_OK,
47   );
48 } catch {
49   console.error("npm ci exited successfully but the local vinext executable is unavailable.");
50   process.exitCode = 69;
51 }
```

scripts/install-ci.sh

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5
6  if [[ "${SITES_ENV_READY:-}" != "1" ]]; then
7      exec "${script_dir}/sites-env.sh" -- "$@" "$@"
8  fi
9
10 command -v flock || {
11     echo "install-ci.sh requires Linux flock." >&2
12     exit 69
13 }
14 command -v timeout || {
15     echo "install-ci.sh requires GNU timeout." >&2
16     exit 69
17 }
18 command -v curl || {
19     echo "install-ci.sh requires curl for the locked-tarball preflight." >&2
20     exit 69
21 }
22 command -v sha256sum || {
23     echo "install-ci.sh requires sha256sum for cache and install verification." >&2
24     exit 69
25 }
26
27 runtime_root="${SITES_RUNTIME_ROOT:-${SITES_PROJECT_ROOT}/.sites-runtime}"
28 expected_home="${runtime_root}/home"
29 expected_cache="${runtime_root}/npm-cache"
30
31 echo "[sites] validating writable install environment"
32 if [[ "${HOME}" != "${expected_home}" ]]; then
33     echo "Expected HOME=${expected_home}, got HOME=${HOME}." >&2
34     exit 78
35 fi
36 actual_cache="$(npm --prefix "${SITES_PROJECT_ROOT}" --workspaces=false config get cache)"
37 if [[ "${actual_cache}" != "${expected_cache}" ]]; then
38     echo "Expected npm cache ${expected_cache}, got ${actual_cache}." >&2
39     exit 78
40 fi
41 touch "${HOME}/.sites-write-test" "${expected_cache}/.sites-write-test"
42 rm -f "${HOME}/.sites-write-test" "${expected_cache}/.sites-write-test"
43 echo "[sites] environment passed: HOME=${HOME}, cache=${expected_cache}"
44
45 lock_file="${runtime_root}/install.lock"
46 exec 9>"${lock_file}"
47 if ! flock -n 9; then
48     echo "Another dependency install is already running for ${SITES_PROJECT_ROOT}." >&2
49     exit 75
50 fi
51
52 # Catch an installer started outside this helper. Linux exposes both its command
53 # line and working directory through /proc, so avoid broad process-name matches.
54 for process in /proc/[0-9]*; do
55     pid="${process##*/}"
56     [[ "${pid}" != "$$" && "${pid}" != "${PPID}" ]] || continue
57     process_cwd="$(readlink -f "${process}/cwd" || true)"
58     [[ "${process_cwd}" == "${SITES_PROJECT_ROOT}" ]] || continue
59     process_command="$(tr '\0' ' ' <"${process}/cmdline" || true)"
60     if [[ "${process_command}" == *"npm ci"* ]]; then
61         echo "Another npm ci is visible in ${SITES_PROJECT_ROOT}; refusing to overlap installs." >&2
62         exit 75
63     fi
64 done
65
66 lockfile_sha256="$(sha256sum "${SITES_PROJECT_ROOT}/package-lock.json" | awk '{print $1}')"
67 use_seeded_cache=0
68 # Report seed selection separately from package cache hits or downloads.
69 cache_seed_result=seed_unavailable
70 seed_cache="${SITES_NPM_CACHE_SEED:-}"
71 if [[ -n "${seed_cache}" && -d "${seed_cache}" ]]; then
72     seed_lockfile_sha256="$(cat "${seed_cache}/.sites-lockfile-sha256" || true)"
73     if [[ "${seed_lockfile_sha256}" == "${lockfile_sha256}" ]]; then
74         echo "[sites] restoring image-seeded npm cache"
75         cp -a "${seed_cache}/." "${expected_cache}/"
76         use_seeded_cache=1
77         cache_seed_result=seed_used
78         echo "[sites] image cache seed matched; registry fallback remains available"
79     else
80         cache_seed_result=seed_lockfile_mismatch

```

scripts/install-ci.sh (continued)

```

81     echo "[sites] image cache seed does not match this lockfile; using the network path"
82 fi
83 fi
84
85 locked_vinext_output="$( { node --input-type=module - "${SITES_PROJECT_ROOT}/package-lock.json" <<'NODE'
86 import { readFile } from "node:fs/promises";
87
88 const lock = JSON.parse(await readFile(process.argv[2], "utf8"));
89 const vinext = lock.packages?.["node_modules/vinext"];
90 if (!vinext?.resolved || !vinext?.integrity) {
91     throw new Error("package-lock.json does not contain a resolved, integrity-pinned vinext tarball");
92 }
93 console.log(vinext.resolved);
94 console.log(vinext.integrity);
95 NODE
96 })" || {
97     echo "Could not read the integrity-pinned vinext tarball from package-lock.json." >&2
98     exit 65
99 }
100 mapfile -t locked_vinext <<"${locked_vinext_output}"
101 if [[ "${#locked_vinext[@]}" -ne 2 ]]; then
102     echo "Expected exactly one Vinext URL and integrity value from package-lock.json." >&2
103     exit 65
104 fi
105
106 locked_tarball="${locked_vinext[0]}"
107 locked_integrity="${locked_vinext[1]}"
108
109 if [[ "${use_seeded_cache}" == "0" ]]; then
110     registry="$(npm --prefix "${SITES_PROJECT_ROOT}" --workspaces=false config get registry)"
111     preflight_url="$( { node --input-type=module - "${locked_tarball}" "${registry}" <<'NODE'
112 const locked = new URL(process.argv[2]);
113 const registry = new URL(process.argv[3]);
114 if (locked.hostname === "registry.npmjs.org") {
115     locked.protocol = registry.protocol;
116     locked.host = registry.host;
117     locked.pathname = "${registry.pathname.replace(/\/$/, "")}${locked.pathname}`;
118 }
119 process.stdout.write(locked.href);
120 NODE
121 })" || {
122     echo "Could not construct the locked-tarball preflight URL." >&2
123     exit 65
124 }
125
126 preflight_dir="${runtime_root}/preflight"
127 preflight_tarball="${preflight_dir}/vinext.tgz"
128 mkdir -p "${preflight_dir}"
129
130 echo "[sites] downloading the complete locked vinext tarball"
131 curl \
132     --fail \
133     --location \
134     --silent \
135     --show-error \
136     --retry 0 \
137     --connect-timeout 15 \
138     --max-time 120 \
139     --output "${preflight_tarball}" \
140     "${preflight_url}"
141
142 echo "[sites] verifying locked vinext tarball integrity"
143 node --input-type=module - "${preflight_tarball}" "${locked_integrity}" <<'NODE'
144 import { createHash } from "node:crypto";
145 import { readFile } from "node:fs/promises";
146
147 const [algorithm, expected] = process.argv[3].split("-", 2);
148 if (!algorithm || !expected) {
149     throw new Error(`unsupported integrity value: ${process.argv[3]}`);
150 }
151 const actual = createHash(algorithm)
152     .update(await readFile(process.argv[2]))
153     .digest("base64");
154 if (actual !== expected) {
155     throw new Error(`vinext tarball integrity mismatch for ${algorithm}`);
156 }
157 NODE
158 echo "[sites] network and integrity preflight passed"
159 fi
160

```

scripts/install-ci.sh (continued)

```
161 echo "[sites] running exactly one bounded npm ci"
162 export NPM_CONFIG_MAXSOCKETS=1
163 export NPM_CONFIG_FETCH_RETRIES=0
164 export NPM_CONFIG_FETCH_TIMEOUT=30000
165 npm_ci_args=(ci --prefix "${SITES_PROJECT_ROOT}" --workspaces=false --include=dev --include=optional --cache
    "${expected_cache}")
166 if [[ "${use_seeded_cache}" == "1" ]]; then
167     npm_ci_args+=(--prefer-offline)
168 fi
169 timeout \
170     --signal=TERM \
171     --kill-after="${SITES_INSTALL_KILL_AFTER:-15s}" \
172     "${SITES_INSTALL_TIMEOUT:-8m}" \
173     npm "${npm_ci_args[@]}"
174
175 vinext="${SITES_PROJECT_ROOT}/node_modules/.bin/vinext"
176 if [[ ! -x "${vinext}" ]]; then
177     echo "npm ci exited successfully but node_modules/.bin/vinext is unavailable." >&2
178     exit 69
179 fi
180
181 node --input-type=module - "${SITES_PROJECT_ROOT}/node_modules/.sites-install.json" "${lockfile_sha256}"
    "${cache_seed_result}" <<'NODE'
182 import { constants } from "node:fs";
183 import { open, writeFile } from "node:fs/promises";
184
185 await writeFile(
186     process.argv[2],
187     `${JSON.stringify({
188         lockfile_sha256: process.argv[3],
189         node: process.version,
190         platform: `${process.platform}-${process.arch}`,
191     }, null, 2)}\n`,
192 );
193 // The measured plugin wrapper supplies an existing private file for this attempt.
194 // Cache telemetry must not change the install result or create arbitrary files.
195 const reportPath = process.env.SITES_INSTALL_REPORT_PATH;
196 if (reportPath) {
197     let report;
198     try {
199         report = await open(reportPath, constants.O_WRONLY | constants.O_NOFOLLOW | constants.O_NONBLOCK);
200         if ((await report.stat()).isFile()) {
201             await report.truncate(0);
202             await report.writeFile(`${JSON.stringify({ version: 1, cache_seed: process.argv[4] })}\n`);
203         }
204     } catch {
205         // Leave the decision unavailable rather than inventing seed use or failing setup.
206     } finally {
207         await report?.close().catch(() => {});
208     }
209 }
210 NODE
211 echo "[sites] npm ci passed and vinext is available"
```

scripts/install-pnpm.sh

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  if [[ "${SITES_ENV_READY:-}" != "1" ]]; then
6    exec "${script_dir}/sites-env.sh" -- "$@" "$@"
7  fi
8
9  require_shared=0
10 prepare_only=0
11 for argument in "$@"; do
12   case "${argument}" in
13     --require-shared) require_shared=1 ;;
14     --prepare-store) prepare_only=1 ;;
15     *) echo "Unknown pnpm installation option: ${argument}" >&2; exit 64 ;;
16   esac
17 done
18
19 for tool in flock timeout node; do
20   command -v "${tool}" >/dev/null || { echo "pnpm setup requires Linux flock, GNU timeout, and Node.js."
21   >&2; exit 69; }
22 done
23 cache_seed=decision_unavailable
24 store_scope=unknown
25 report_store() {
26   node "${script_dir}/pnpm-install.mjs" --report-store "${cache_seed}" "${store_scope}" "${store_state}"
27   "${prepare_only}"
28 }
29 # Publish preparation failures too; the optional report never decides success.
30 trap 'report_store || true' EXIT
31
32 if [[ -n "${SITES_PNPM_BIN:-}" && -f "${SITES_PNPM_BIN}" && -r "${SITES_PNPM_BIN}" ]]; then
33   pnpm_command=(node "${SITES_PNPM_BIN}")
34 elif [[ "${require_shared}" == 1 ]]; then
35   echo "[sites] the image-pinned pnpm is unavailable" >&2
36   exit 69
37 elif command -v corepack >/dev/null; then
38   # Corepack honors the established project's pin even when PATH has a different
39   # pnpm version. A bare pnpm is used only if Corepack is absent, then verified.
40   pnpm_command=(corepack pnpm)
41 elif command -v pnpm >/dev/null; then
42   pnpm_command=(pnpm)
43 else
44   echo "This pnpm project requires pnpm 11.25.0 or Corepack." >&2
45   exit 69
46 fi
47
48 runtime_root="${SITES_RUNTIME_ROOT:-${SITES_PROJECT_ROOT}/.sites-runtime}"
49 private_store="${runtime_root}/pnpm-store"
50 if [[ "${HOME}" != "${runtime_root}/home" || -L "${private_store}" ]]; then
51   echo "Dependency setup requires a project-owned writable home and pnpm store." >&2
52   exit 78
53 fi
54 # A single Node process owns the regular-file descriptors for both leases.
55 # Closing its input (including this shell exiting) releases the project lock.
56 coproc SITES_INSTALL_LOCKS {
57   node "${script_dir}/pnpm-install.mjs" --hold-install-locks \
58     "${runtime_root}/install.lock" "${SITES_PNPM_SHARED_STORE:-}.seed.lock" \
59     "${SITES_PNPM_STORE_LOCK_TIMEOUT:-30}"
60 }
61 install_lock_pid="${SITES_INSTALL_LOCKS_PID}"
62 install_lock_output="${SITES_INSTALL_LOCKS[0]}"
63 install_lock_input="${SITES_INSTALL_LOCKS[1]}"
64 lock_status=unavailable
65 read -r lock_status <"${install_lock_output}" || true
66 if [[ "${lock_status}" != locked ]]; then
67   exec {install_lock_input}>&-
68   exec {install_lock_output}<&-
69   wait "${install_lock_pid}" || true
70   if [[ "${lock_status}" == busy ]]; then
71     echo "Another dependency install is already running for ${SITES_PROJECT_ROOT}." >&2
72     exit 75
73   fi
74   echo "Dependency setup requires a regular, writable project install lock." >&2
75   exit 78
76 fi
77 export XDG_CACHE_HOME="${runtime_root}/xdg-cache" XDG_DATA_HOME="${runtime_root}/xdg-data"
78 mkdir -p "${XDG_CACHE_HOME}" "${XDG_DATA_HOME}" || exit 70
79 pnpm_version="$(timeout --signal=TERM --kill-after="${SITES_INSTALL_KILL_AFTER:-15s}" \

```

scripts/install-pnpm.sh (continued)

```

79  "${SITES_PNPM_BOOTSTRAP_TIMEOUT:-30s}" "${pnpm_command[@]}" --version)"
80  if [[ "${pnpm_version}" != "11.25.0" ]]; then
81    echo "This project requires pnpm 11.25.0 and its v11 store format." >&2
82    exit 69
83  fi
84
85  can_write_directory() {
86    local probe
87    # access()/test -w can succeed even when a kernel sandbox denies a write.
88    probe="$(mktemp "${1}/.sites-store-probe.XXXXXX" 2>/dev/null)" || return 1
89    rm -f -- "${probe}" 2>/dev/null
90  }
91
92  release_shared_lock() {
93    if [[ "${shared_lock_held:-}" == 1 ]]; then
94      printf '%s\n' release-shared >&"${install_lock_input}"
95      local status
96      read -r status <&"${install_lock_output}" && [[ "${status}" == released ]] || return 70
97      shared_lock_held=0
98    fi
99  }
100
101  acquire_shared_lock() {
102    # The Node holder opens this workspace-controlled entry with no-follow and
103    # nonblocking flags, then verifies a regular file before bounded flock.
104    printf '%s\n' shared >&"${install_lock_input}"
105    local status
106    if read -r status <&"${install_lock_output}" && [[ "${status}" == locked ]]; then
107      shared_lock_held=1
108      return 0
109    fi
110    return 1
111  }
112
113  writable_store="${private_store}"
114  shared_store="${SITES_PNPM_SHARED_STORE:-}"
115  if [[ "${shared_store}" == "/workspace/.sites-runtime/pnpm-store" &&
116    ! -L "${shared_store}" && ! -L "${shared_store%/*}" &&
117    -d "${shared_store%/*/*}" ]]; then
118    # Normal Work already writes this owner's workspace. Narrower profiles must
119    # pass the actual create probe, without widening their permissions.
120    if mkdir -p "${shared_store%/*}" 2>/dev/null &&
121      can_write_directory "${shared_store%/*}" &&
122      acquire_shared_lock; then
123      if [[ ! -L "${shared_store}" && ! -L "${shared_store%/*}" &&
124        ( ! -e "${shared_store}" || ( -d "${shared_store}" && ! -L "${shared_store}" && -w
125        "${shared_store}" ) ) ]] &&
126        { [[ ! -d "${shared_store}" ]] || can_write_directory "${shared_store}"; }; then
127        writable_store="${shared_store}"
128        store_scope=workspace
129      else
130        release_shared_lock
131      fi
132    else
133      release_shared_lock
134    fi
135  if [[ "${store_scope}" != workspace ]]; then
136    if [[ "${require_shared}" == 1 ]]; then
137      echo "[sites] the workspace pnpm store is unavailable" >&2
138      exit 69
139    fi
140    store_scope=project
141  fi
142
143  seed="${SITES_PNPM_CACHE_SEED:-}"
144  cache_seed=seed_unavailable
145  if [[ -d "${writable_store}" ]]; then
146    # Never merge a newer image seed into an existing mutable store.
147    store_state=reused
148    cache_seed=not_applicable
149    if [[ -f "${writable_store}/.sites-pnpm-seed-applied.json" ]]; then cache_seed=seed_used; fi
150  else
151    seed_compatible=0
152    if [[ -n "${seed}" && -d "${seed}/v11" ]] && node --input-type=module - "${seed}/.sites-pnpm-seed.json"
153    <<'NODE'
154  import { readFileSync, statSync } from "node:fs";
155  try {
156    const filename = process.argv[2];
157    if (statSync(filename).size > 4096) process.exit(1);

```

scripts/install-pnpm.sh (continued)

```

157 const seed = JSON.parse(readFileSync(filename, "utf8"));
158 process.exit(seed.version === 1 && seed.pnpm_version === "11.25.0" &&
159   seed.store_version === "v11" && /^[a-f0-9]{64}$/.test(seed.lockfile_sha256) ? 0 : 1);
160 } catch { process.exit(1); }
161 NODE
162   then seed_compatible=1; fi
163   if [[ "${seed_compatible}" == 1 ]]; then
164     if [[ "${prepare_only}" == 1 ]]; then
165       echo "[sites] copying the image seed into the writable ${store_scope} pnpm store" >&2
166     else
167       echo "[sites] copying the image seed into the writable ${store_scope} pnpm store"
168     fi
169     seed_stage="$(mktemp -d "${writable_store}.seed.XXXXXX")" || exit 70
170     trap 'if [[ -n "${seed_stage:-}" ]]; then rm -rf -- "${seed_stage}"; fi; report_store || true' EXIT
171     timeout --signal=TERM --kill-after="${SITES_INSTALL_KILL_AFTER:-15s}" \
172       "${SITES_PNPM_STORE_PREPARE_TIMEOUT:-60s}" bash -c \
173       'cp -a --no-preserve=ownership "$1/" "$2/" && chmod -R u+rwX "$2" _ "${seed}" "${seed_stage}" ||
exit 70
174   cp "${seed}/${sites-pnpm-seed.json}" "${seed_stage}/${sites-pnpm-seed-applied.json}" || exit 70
175   # An ordinary pnpm command can initialize the store without our seed lock.
176   # Publish without replacing even an empty directory created by that command.
177   mv -T --update=none "${seed_stage}" "${writable_store}" || exit 70
178   if [[ -d "${seed_stage}" ]]; then
179     if [[ -L "${writable_store}" || ! -d "${writable_store}" ]]; then exit 78; fi
180     rm -rf -- "${seed_stage}" || exit 70
181     store_state=reused
182     cache_seed=not_applicable
183     if [[ -f "${writable_store}/${sites-pnpm-seed-applied.json}" ]]; then cache_seed=seed_used; fi
184   else
185     store_state=seeded
186     cache_seed=seed_used
187   fi
188   seed_stage=""
189 else
190   mkdir -p "${writable_store}" || exit 70
191   store_state=created
192 fi
193 fi
194 if ! can_write_directory "${writable_store}"; then store_state=unavailable; exit 70; fi
195 release_shared_lock
196 report_store
197 trap - EXIT
198 if [[ "${prepare_only}" == 1 ]]; then exit 0; fi
199
200 # Configuration travels with source without embedding an absolute machine path.
201 # A later restricted session selects a private store before its frozen repair.
202 configured_store=.sites-runtime/pnpm-store
203 if [[ "${store_scope}" == workspace ]]; then
204   configured_store="${SITES_PNPM_SHARED_STORE:-.sites-runtime/pnpm-store}"
205 elif [[ "${runtime_root}" != "${SITES_PROJECT_ROOT}/${sites-runtime}" ]]; then
206   configured_store="${SITES_RUNTIME_ROOT:-.sites-runtime}/pnpm-store"
207 fi
208 "${pnpm_command[@]}" config set package-import-method clone-or-copy --location project
209 "${pnpm_command[@]}" config set store-dir "${configured_store}" --location project
210 "${pnpm_command[@]}" config set cache-dir "${configured_store}/policy-cache" --location project
211
212 # CI=true lets native pnpm install rebuild modules whose store moved, without a
213 # prompt, --force, changing the lockfile, or translating the project into npm.
214 CI=true timeout --signal=TERM --kill-after="${SITES_INSTALL_KILL_AFTER:-15s}" \
215   "${SITES_INSTALL_TIMEOUT:-8m}" node "${script_dir}/pnpm-install.mjs" \
216   "${cache_seed}" "${store_scope}" "${store_state}" "${writable_store}" "${pnpm_command[@]}"

```

scripts/pnpm-install.mjs

```

1 import { spawn } from "node:child_process";
2 import { createHash } from "node:crypto";
3 import {
4   accessSync,
5   closeSync,
6   constants,
7   fstatSync,
8   ftruncateSync,
9   openSync,
10  readFileSync,
11  writeFileSync,
12  writeSync,
13 } from "node:fs";
14 import { constants as osConstants } from "node:os";
15 import path from "node:path";
16 import { createInterface } from "node:readline";
17 import { fileURLToPath } from "node:url";
18
19 const MAX_PACKAGES = 100_000;
20 const CACHE_SEEDS = new Set(["seed_used", "seed_unavailable", "seed_lockfile_mismatch",
21   "decision_unavailable", "not_applicable"]);
22 const STORE_STATES = new Set(["created", "seeded", "reused", "unavailable"]);
23 // The caller may use operational status only for
24 // initial-setup fallback; an established project always retains pnpm. Broad
25 // fetch failures can include TLS/auth errors and stay fatal.
26 const OPERATIONAL_FAILURE_CODES = new Set([
27   "ERR_PNPM_UNEXPECTED_STORE",
28   "ERR_PNPM_UNEXPECTED_VIRTUAL_STORE",
29   "ERR_PNPM_OUTDATED_LOCKFILE",
30   "ERR_PNPM_FROZEN_LOCKFILE_WITH_OUTDATED_LOCKFILE",
31   "ERR_PNPM_NO_LOCKFILE",
32   "ERR_PNPM_FETCH_408",
33   "ERR_PNPM_FETCH_429",
34   "ERR_PNPM_FETCH_500",
35   "ERR_PNPM_FETCH_502",
36   "ERR_PNPM_FETCH_503",
37   "ERR_PNPM_FETCH_504",
38 ]);
39 // These are pnpm-reported package IDs, not an assertion about network bytes or
40 // image-cache provenance. Do not interpret an installed-tree no-op as 100% reuse.
41 export class InstallProgress {
42   constructor(project) {
43     this.project = path.resolve(project);
44     this.reused = new Set();
45     this.downloaded = new Set();
46     this.complete = false;
47     this.added = undefined;
48     this.invalid = false;
49   }
50
51   accept(event) {
52     if (event?.name === "pnpm:stats" && event.prefix === this.project &&
53       Number.isSafeInteger(event.added) && event.added >= 0) this.added = event.added;
54     if (event?.name === "pnpm:stage" && event.stage === "importing_done" &&
55       event.prefix === this.project) this.complete = true;
56     if (event?.name !== "pnpm:progress" || event.requester !== this.project ||
57       ![ "fetched", "found_in_store" ].includes(event.status)) return;
58     if (typeof event.packageId !== "string" || !event.packageId || event.packageId.length > 4096 ||
59       this.reused.size + this.downloaded.size >= MAX_PACKAGES) {
60       this.invalid = true;
61       return;
62     }
63     if (event.status === "fetched") {
64       this.reused.delete(event.packageId);
65       this.downloaded.add(event.packageId);
66     } else if (!this.downloaded.has(event.packageId)) {
67       this.reused.add(event.packageId);
68     }
69   }
70
71   counts(success) {
72     if (!success || !this.complete || !(this.added > 0) || this.invalid ||
73       this.reused.size + this.downloaded.size === 0) return {};
74     return { packages_reused: this.reused.size, packages_downloaded: this.downloaded.size };
75   }
76 }
77
78 function openReport() {
79   const report = process.env.SITES_INSTALL_REPORT_PATH;

```

scripts/pnpm-install.mjs (continued)

```

80   if (!report || !constants.O_NOFOLLOW) return undefined;
81   let fd;
82   try {
83     fd = openSync(report, constants.O_WRONLY | constants.O_NOFOLLOW | constants.O_NONBLOCK);
84     if (fstatSync(fd).isFile()) return fd;
85   } catch {}
86   if (fd !== undefined) closeSync(fd);
87   return undefined;
88 }
89
90 function writeReport(fd, report) {
91   if (fd === undefined) return;
92   try {
93     const body = `${JSON.stringify(report)}\n`;
94     writeSync(fd, body, 0, "utf8");
95     ftruncateSync(fd, Buffer.byteLength(body));
96   } catch {}
97   // Optional telemetry must not change installation behavior or expose its path.
98 }
99 }
100
101 function showLine(line, progress, failure) {
102   let event;
103   try {
104     event = JSON.parse(line);
105   } catch {}
106   process.stdout.write(`${line}\n`);
107   return;
108 }
109 progress.accept(event);
110 if (event?.level === "error" && failure.code !== 65) {
111   failure.code = event.name === "pnpm" && OPERATIONAL_FAILURE_CODES.has(event.err?.code)
112     ? 70 : 65;
113 }
114 const message = event?.message ?? event?.err?.message;
115 if (event?.name === "pnpm:lifecycle" && typeof event.line === "string") {
116   process.stdout.write(`${event.line}\n`);
117 } else if (typeof message === "string" && ["warn", "error", "info"].includes(event.level)) {
118   const output = event.level === "error" ? process.stderr : process.stdout;
119   output.write(`${message}\n`);
120 }
121 }
122
123 async function openLock(filename, waitSeconds) {
124   let fd;
125   try {
126     if (!filename || !constants.O_NOFOLLOW || !constants.O_NONBLOCK) throw new Error("Unsupported lock");
127     fd = openSync(filename, constants.O_CREAT | constants.O_RDWR | constants.O_NOFOLLOW |
128       constants.O_NONBLOCK, 0o600);
129     if (!fstatSync(fd).isFile()) throw new Error("Expected a regular lock file");
130     const child = spawn("flock", ["-w", waitSeconds, "3"], { stdio: ["ignore", "ignore", "ignore", fd] });
131     const code = await new Promise((resolve) => {
132       child.once("error", () => resolve(undefined));
133       child.once("close", resolve);
134     });
135     if (code === 0) return { status: "locked", fd };
136     closeSync(fd);
137     return { status: code === undefined ? "unavailable" : "busy" };
138   } catch {}
139   if (fd !== undefined) closeSync(fd);
140   return { status: "unavailable" };
141 }
142
143 async function holdInstallLocks(projectLock, sharedLock, waitSeconds) {
144   // The shell holds both leases through stdin. EOF (including caller exit)
145   // releases the same open file descriptions that flock locked in its child.
146   const requests = createInterface({ input: process.stdin, crlfDelay: Infinity });
147   const project = await openLock(projectLock, "0");
148   let shared;
149   try {
150     process.stdout.write(`${project.status}\n`);
151     for await (const request of requests) {
152       if (request === "shared" && project.fd !== undefined) {
153         shared ??= await openLock(sharedLock, waitSeconds);
154         process.stdout.write(`${shared.status}\n`);
155       } else if (request === "release-shared") {
156         if (shared?.fd !== undefined) closeSync(shared.fd);
157         shared = undefined;
158         process.stdout.write("released\n");

```

scripts/pnpm-install.mjs (continued)

```

159     } else {
160       process.stdout.write("unavailable\n");
161     }
162   }
163 } finally {
164   if (shared?.fd !== undefined) closeSync(shared.fd);
165   if (project.fd !== undefined) closeSync(project.fd);
166   requests.close();
167 }
168 }
169
170 async function main() {
171   const argsIn = process.argv.slice(2);
172   if (argsIn[0] === "--hold-install-locks") {
173     await holdInstallLocks(argsIn[1], argsIn[2], argsIn[3]);
174     return;
175   }
176   if (argsIn[0] === "--report-store") {
177     const [, cacheSeed, storeScope, storeState, stdout] = argsIn;
178     if (!CACHE_SEEDS.has(cacheSeed) || !["project", "workspace", "unknown"].includes(storeScope) ||
179       !STORE_STATES.has(storeState)) { process.exitCode = 64; return; }
180     const report = { version: 1, cache_seed: cacheSeed, store_scope: storeScope, store_state: storeState };
181     const fd = openReport();
182     writeReport(fd, report);
183     if (fd !== undefined) closeSync(fd);
184     if (stdout === "1") process.stdout.write(`${JSON.stringify(report)}\n`);
185     return;
186   }
187   const [cacheSeed, storeScope, storeState, store, executable, ...prefix] = argsIn;
188   if (!CACHE_SEEDS.has(cacheSeed) || !["project", "workspace"].includes(storeScope) ||
189     !STORE_STATES.has(storeState) || storeState === "unavailable" ||
190     !store || !path.isAbsolute(store) || !executable) {
191     process.stderr.write("Invalid pnpm installation arguments.\n");
192     process.exitCode = 64;
193     return;
194   }
195   const fd = openReport();
196   const report = { version: 1, cache_seed: cacheSeed, store_scope: storeScope, store_state: storeState };
197   writeReport(fd, report);
198   const progress = new InstallProgress(process.cwd());
199   const failure = {};
200   const env = { ...process.env };
201   delete env.SITES_INSTALL_REPORT_PATH;
202   const args = [...prefix, "install", "--prod=false", "--ignore-scripts=false",
203     "--frozen-lockfile", "--prefer-offline", "--store-dir", store,
204     "--cache-dir", path.join(store, "policy-cache"), "--fetch-retries=0",
205     "--fetch-timeout=30000", "--network-concurrency=1", "--reporter=ndjson",
206     "--package-import-method=clone-or-copy"];
207   let result = { code: 1, signal: null };
208   let receivedSignal;
209   try {
210     const child = spawn(executable, args, { env, stdio: ["inherit", "pipe", "inherit"] });
211     const lines = createInterface({ input: child.stdout, crlfDelay: Infinity });
212     lines.on("line", (line) => showLine(line, progress, failure));
213     // timeout/exec owns the inherited group. Relaying would deliver signals twice.
214     const signalHandlers = ["SIGINT", "SIGHUP", "SIGTERM"].map((signal) => {
215       const handler = () => { receivedSignal ??= signal; };
216       process.on(signal, handler);
217       return [signal, handler];
218     });
219     let spawnError;
220     child.once("error", (error) => { spawnError = error; });
221     result = await new Promise((resolve) => child.once("close", (code, signal) => {
222       resolve({ code: spawnError ? (spawnError.code === "ENOENT" ? 127 : 1) : code, signal });
223     }));
224     for (const [signal, handler] of signalHandlers) process.removeListener(signal, handler);
225     lines.close();
226     result.signal ??= receivedSignal;
227     if (spawnError) {
228       process.stderr.write("Unable to start pnpm.\n");
229       result.code = spawnError.code === "ENOENT" ? 127 : 65;
230     } else if (result.code !== 0 && !result.signal) {
231       result.code = failure.code ?? 65;
232     }
233     if (result.code === 0 && !result.signal) {
234       accessSync("node_modules/.bin/vinext", constants.X_OK);
235       const lock = readFileSync("pnpm-lock.yaml");
236       writeFileSync("node_modules/.sites-install.json", `${JSON.stringify({
237         package_manager: "pnpm@11.25.0",
238         lockfile_sha256: createHash("sha256").update(lock).digest("hex"),

```

scripts/pnpm-install.mjs (continued)

```
239     node: process.version,
240     platform: `${process.platform}-${process.arch}`,
241   }, null, 2)}\n`);
242 }
243 } catch {
244   process.stderr.write("Dependency setup did not produce a usable Vinext installation.\n");
245   result = { code: 65, signal: receivedSignal ?? null };
246 } finally {
247   Object.assign(report, progress.counts(result.code === 0 && !result.signal));
248   writeReport(fd, report);
249   if (fd !== undefined) closeSync(fd);
250 }
251 if (report.packages_reused !== undefined) {
252   process.stdout.write(`[sites] pnpm reused ${report.packages_reused} packages and downloaded
253   ${report.packages_downloaded}\n`);
254 }
255 if (result.code === 0 && !result.signal) process.stdout.write("[sites] dependency setup passed\n");
256 process.exitCode = result.signal ? 128 + (osConstants.signals[result.signal] ?? 0) : result.code ?? 1;
257 if (result.signal) {
258   try { process.kill(process.pid, result.signal); } catch {}
259 }
260 }
261 if (process.argv[1] && path.resolve(process.argv[1]) === fileURLToPath(import.meta.url)) await main();
```

scripts/prepare-public-artifacts.mjs

```
1 import {mkdir,copyFile} from 'node:fs/promises';
2 await mkdir('public/downloads',{recursive:true});
3 for(const file of ['Lotlight-Demo-Captioned.mp4','Lotlight-One-Page.pdf','Lotlight-Pitch.pdf','Lotlight-
  Source-Code.pdf','Lotlight-Captions.srt','TESTING-INSTRUCTIONS.md','DEVPOST-STORY.md']) {
4   await copyFile('submission/'+file,'public/downloads/'+file);
5 }
6 console.log('Prepared public judge resources.');
```

scripts/record-demo.mjs

```

1 import { chromium } from "playwright";
2 import fs from "node:fs/promises";
3 const base = process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174";
4 await fs.mkdir(".artifact-build/video", { recursive: true });
5 const context = await chromium.launchPersistentContext(
6   "/tmp/lotlight-ai-validation",
7   {
8     headless: true,
9     viewport: { width: 1440, height: 1000 },
10    recordVideo: {
11      dir: ".artifact-build/video",
12      size: { width: 1440, height: 1000 },
13    },
14  },
15 );
16 const page = context.pages()[0];
17 await page.goto(base);
18 await page.locator('main[data-ready="true"]').waitFor();
19 const start = Date.now();
20 const markers = [];
21 async function until(seconds) {
22   const wait = seconds * 1000 - (Date.now() - start);
23   if (wait > 0) await page.waitForTimeout(wait);
24 }
25 async function scene(seconds, name, fn) {
26   await until(seconds);
27   markers.push({ seconds, name });
28   console.log("Scene", seconds, name);
29   if (fn) await fn();
30 }
31 async function moveClick(locator) {
32   await locator.scrollIntoViewIfNeeded();
33   const box = await locator.boundingBox();
34   if (box)
35     await page.mouse.move(box.x + box.width * 0.5, box.y + box.height * 0.5, {
36       steps: 20,
37     });
38   await page.waitForTimeout(350);
39   await locator.click();
40 }
41 try {
42   await scene(0, "A notice is only the beginning");
43   await scene(19, "Bring the inventory", async () => {
44     await moveClick(
45       page.getByRole("button", { name: "Inventory", exact: true }),
46     );
47     await moveClick(page.getByRole("button", { name: "Import CSV" }));
48     await moveClick(
49       page.getByRole("button", { name: "Use synthetic example" }),
50     );
51   });
52   await scene(31, "Validate and import", async () => {
53     await moveClick(page.getByRole("button", { name: "Validate & import" }));
54   });
55   await scene(39, "Review the original source", async () => {
56     await moveClick(
57       page.getByRole("button", { name: /Recall workspace/ }).first(),
58     );
59     await moveClick(
60       page.getByRole("button", { name: "Open review", exact: true }),
61     );
62     await moveClick(page.getByRole("tab", { name: "Source & scope" }));
63   });
64   await scene(53, "Human source approval", async () => {
65     await page.getByRole("checkbox").check();
66     await moveClick(
67       page.getByRole("button", { name: "Confirm source review" }),
68     );
69   });
70   await scene(61, "AI finds an alias", async () => {
71     await moveClick(page.getByRole("tab", { name: /Inventory review/ }));
72     await moveClick(
73       page.getByRole("button", { name: "Run local AI", exact: true }),
74     );
75     await page
76       .getByText(/Semantic comparison complete/)
77       .waitFor({ timeout: 120000 });
78     await moveClick(
79       page.getByRole("button", {
80         name: "Show evidence for IV line extender in Mobile cart",

```

scripts/record-demo.mjs (continued)

```

81     }},
82   );
83 });
84 await scene(79, "Exact identifiers and rationale", async () => {
85   await moveClick(
86     page.getByRole("button", {
87       name: "Show evidence for CLEARLINK extension set in Infusion room",
88     }),
89   );
90   await page
91     .locator(".match-row")
92     .filter({
93       has: page.getByRole("button", {
94         name: "Show evidence for CLEARLINK extension set in Infusion room",
95       }),
96     })
97     .scrollIntoViewIfNeeded();
98 });
99 await scene(92, "Record a response", async () => {
100   const row = page
101     .locator(".match-row")
102     .filter({
103       has: page.getByRole("button", {
104         name: "Show evidence for CLEARLINK extension set in Infusion room",
105       }),
106     });
107   await moveClick(row.getByRole("button", { name: "Assign response" }));
108   await page
109     .getByLabel("Responsible person")
110     .fill("Maya Rao \xb7 demo coordinator");
111   await page.getByLabel("Response due date").fill("2026-09-18");
112   await page.getByRole("combobox", { name: "Response stage" }).click();
113   await page.getByRole("option", { name: /Completed/ }).click();
114   await page
115     .getByLabel("Physical-label and current-source verification")
116     .fill(
117       "Synthetic demo: checked physical catalog and lot labels against the original manufacturer letter.",
118     );
119   await page.getByLabel("Units handled").fill("24");
120   await page
121     .getByLabel("Response evidence / reference")
122     .fill(
123       "Synthetic demo evidence: isolation log Q-104 and supplier return reference RMA-210.",
124     );
125 });
126 await scene(113, "Documented completion", async () => {
127   await moveClick(
128     page.getByRole("button", { name: "Save response", exact: true }),
129   );
130 });
131 await scene(121, "Missing evidence stays open", async () => {
132   await moveClick(
133     page.getByRole("button", {
134       name: "Show evidence for CLEARLINK extension set in Procedure room",
135     }),
136   );
137   const row = page
138     .locator(".match-row")
139     .filter({
140       has: page.getByRole("button", {
141         name: "Show evidence for CLEARLINK extension set in Procedure room",
142       }),
143     });
144   await moveClick(row.getByRole("button", { name: "Assign response" }));
145   await page.getByRole("combobox", { name: "Response stage" }).click();
146 });
147 await scene(137, "Portable audit trail", async () => {
148   await page.keyboard.press("Escape");
149   await page.getByRole("button", { name: "Cancel", exact: true }).click();
150   await moveClick(page.getByRole("button", { name: "Export record" }));
151 });
152 await scene(146, "Printable record", async () => {
153   await moveClick(
154     page.getByRole("button", { name: /Readable audit report/ }),
155   );
156   await page.evaluate(() => window.scrollTo({ top: 0, behavior: "smooth" }));
157 });
158 await scene(156, "Business hypothesis and boundaries", async () => {
159   await moveClick(page.getByRole("button", { name: "Evidence & approach" }));
160   await page

```

scripts/record-demo.mjs (continued)

```

161     .getByRole("heading", { name: "A credible first customer" })
162     .scrollIntoViewIfNeeded();
163   });
164   await scene(174, "Lotlight closing", async () => {
165     await moveClick(
166       page.getByRole("button", { name: /Recall workspace/ }).first(),
167     );
168     await page.evaluate(() => window.scrollTo({ top: 0, behavior: "smooth" }));
169   });
170   await until(185);
171   const videoPath = await page.video().path();
172   await context.close();
173   await fs.copyFile(videoPath, ".artifact-build/video/lotlight-demo.webm");
174   await fs.writeFile(
175     "submission/VIDEO-TIMING.json",
176     JSON.stringify({ durationSeconds: 185, markers }, null, 2),
177   );
178   console.log("Recorded .artifact-build/video/lotlight-demo.webm");
179 } catch (e) {
180   await context.close();
181   throw e;
182 }

```

scripts/render-captioned-demo.py

```

1 from pathlib import Path
2 from PIL import Image, ImageDraw, ImageFont
3 import json, subprocess, textwrap
4 ROOT=Path(__file__).resolve().parents[1]; ffmpeg='/opt/homebrew/bin/ffmpeg'
5 work=ROOT/'.artifact-build/narration'; video=ROOT/'.artifact-build/video/lotlight-demo.webm'
6 scenes=json.loads((ROOT/'submission/NARRATION-SCENES.json').read_text())
7 def run(args):subprocess.run([ffmpeg, '-y', '-hide_banner', '-loglevel', 'error']+args, check=True)
8 parts=[];offset=0
9 for i,s in enumerate(scenes):
10 wav=work/f'{i:02d}.wav';dur=float(subprocess.check_output(['opt/homebrew/bin/ffprobe', '-v', 'error', '-show_entries', 'format=duration', '-of', 'default=noprint_wrappers=1:nokey=1', str(wav)]));s['finalStart']=offset;s['finalEnd']=offset+dur;offset+=dur
11 out=work/f'video-{i:02d}.mp4';factor=dur/(s['end']-s['start'])
12
13 run(['-ss', str(s['start']), '-t', str(s['end']-s['start']), '-i', str(video), '-an', '-vf', f'setpts={factor}*(PTS-STARTPTS), fps=30, pad=1440:1120:0:0:color=0x263329', '-t', str(dur), '-c:v', 'libx264', '-crf', '20', '-preset', 'fast', '-pix_fmt', 'yuv420p', str(out)])
14 parts.append("file "+str(out)+"")
15 (work/'video-concat.txt').write_text('\n'.join(parts));run(['-f', 'concat', '-safe', '0', '-i', str(work/'video-concat.txt'), '-c', 'copy', str(work/'picture.mp4')])
16 words=json.loads((work/'alignment.json').read_text())['words'];cues=[];group=[]
17 for w in words:
18 w['word']=w['word'].replace('LotLight', 'Lotlight').replace('Manufacturing', 'manufacturer')
19 if group and (len(' '.join(x['word'] for x in group))+len(w['word'])>74 or len(group)>=11):
20 cues.append(group);group=[]
21 group.append(w)
22 if len(group)>=5 and w['word'].endswith(('.', '?', '!')):cues.append(group);group=[]
23 if group:cues.append(group)
24 font=ImageFont.truetype('/System/Library/Fonts/Supplemental/Arial.ttf', 32);small=ImageFont.truetype('/System/Library/Fonts/Supplemental/Arial.ttf', 16)
25 def card(text, p):
26 im=Image.new('RGB', (1440, 120), '#263329');d=ImageDraw.Draw(im)
27 for j, line in enumerate(textwrap.wrap(text, 74)):
28 d.text((720, 10+j*39), line, font=font, fill='ffffff', anchor='mt')
29 d.text((720, 102), 'AI narration | Actual browser recording | Historical notice and synthetic stock', font=small, fill='#ddec8b', anchor='mm');im.save(p)
30 def ts(t):
31 n=round(t*1000);return f'{n//3600000:02}:{n//60000%60:02}:{n//1000%60:02},{n%1000:03}'
32 subs=[];concat=[f'ffconcat version 1.0'];cursor=0;blank=work/'caption-blank.png';card('', blank)
33 for i,g in enumerate(cues):
34 start=max(cursor, g[0]['start']);end=min(offset, g[-1]['end']);text=' '.join(x['word'] for x in g);p=work/f'caption-{i:03d}.png';card(text, p)
35 if start>cursor:concat += [f'file '{blank}', f'duration {start-cursor:.3f}']
36 concat += [f'file '{p}', f'duration {end-start:.3f}'];cursor=end
37 subs.append(f'{i+1}\n{ts(start)} --> {ts(end)}\n'+textwrap.fill(text, 74)+'\n')
38 if offset>cursor:concat += [f'file '{blank}', f'duration {offset-cursor:.3f}']
39 concat.append(f'file '{blank}');
40 (work/'captions.ffconcat').write_text('\n'.join(concat));(ROOT/'submission/Lotlight-Captions.srt').write_text('\n'.join(subs))
41 run(['-i', str(work/'picture.mp4'), '-f', 'concat', '-safe', '0', '-i', str(work/'captions.ffconcat'), '-i', str(ROOT/'submission/Lotlight-Narration.wav'), '-filter_complex', '[0:v][1:v]overlay=0:1000:eof_action=repeat[v]', '-map', '[v]', '-map', '2:a', '-c:v', 'libx264', '-crf', '19', '-preset', 'fast', '-c:a', 'aac', '-b:a', '192k', '-t', str(offset), '-movflags', '+faststart', str(ROOT/'submission/Lotlight-Demo-Captioned.mp4')])
42 (ROOT/'submission/NARRATION-SCENES.json').write_text(json.dumps(scenes, indent=2)+'\n');print('Finished captioned demo:', round(offset, 2), 'seconds'; len(cues), 'caption cues')

```

scripts/run-framework.mjs

```
1 import { spawnSync } from "node:child_process";
2 import { fileURLToPath } from "node:url";
3 import { readExecutionProfile } from "../execution-profile.mjs";
4
5 const [command, ...args] = process.argv.slice(2);
6 if (!["dev", "build"].includes(command)) throw new Error("Expected dev or build.");
7 const managedLinux = readExecutionProfile() === "managed-linux";
8
9 if (managedLinux && command === "build") {
10   const result = spawnSync("bash", [
11     fileURLToPath(new URL("../build-verified.sh", import.meta.url)), ...args,
12   ], { stdio: "inherit" });
13   if (result.error) throw result.error;
14   process.exit(result.status ?? 1);
15 }
16
17 // Import in this process so the preview owner retains its PID and signals.
18 const cli = new URL(managedLinux
19   ? "../node_modules/vite/bin/vite.js"
20   : "../node_modules/vinext/dist/cli.js", import.meta.url);
21 process.argv = [process.execPath, fileURLToPath(cli), command,
22   ...(!managedLinux && command === "dev" ? ["--port", "5173"] : []), ...args];
23 await import(cli.href);
```

scripts/sites-env.mjs

```
1 import { mkdirSync } from "node:fs";
2 import path from "node:path";
3 import { fileURLToPath } from "node:url";
4
5 export const projectRoot = fileURLToPath(new URL("../", import.meta.url));
6 const runtimeRoot = process.env.SITES_RUNTIME_ROOT || path.join(projectRoot, ".sites-runtime");
7
8 process.env.CLOUDFLARE_CF_FETCH_ENABLED ||= "false";
9 process.env.WRANGLER_SEND_METRICS ||= "false";
10 process.env.WRANGLER_WRITE_LOGS ||= "false";
11 process.env.WRANGLER_LOG_PATH ||= path.join(runtimeRoot, "wrangler/logs");
12 process.env.WRANGLER_REGISTRY_PATH ||= path.join(runtimeRoot, "wrangler/dev-registry");
13 process.env.MINIFLARE_REGISTRY_PATH ||= path.join(runtimeRoot, "wrangler/registry");
14
15 process.chdir(projectRoot);
16 for (const directory of [
17   path.dirname(process.env.WRANGLER_LOG_PATH),
18   process.env.WRANGLER_REGISTRY_PATH,
19   process.env.MINIFLARE_REGISTRY_PATH,
20 ]) {
21   mkdirSync(directory, { recursive: true });
22 }
```

scripts/sites-env.sh

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 project_root="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
5 runtime_root="${SITES_RUNTIME_ROOT:-${project_root}/.sites-runtime}"
6
7 mkdir -p \
8   "${runtime_root}/home" \
9   "${runtime_root}/npm-cache" \
10  "${runtime_root}/xdg-config" \
11  "${runtime_root}/tmp" \
12  "${runtime_root}/wrangler/logs"
13
14 export SITES_ENV_READY=1
15 export SITES_PROJECT_ROOT="${project_root}"
16 export HOME="${runtime_root}/home"
17 export XDG_CONFIG_HOME="${runtime_root}/xdg-config"
18 export TMPDIR="${runtime_root}/tmp"
19 export WRANGLER_WRITE_LOGS=false
20 export WRANGLER_LOG_PATH="${runtime_root}/wrangler/logs"
21 export MINIFLARE_REGISTRY_PATH="${runtime_root}/wrangler/registry"
22
23 # The runtime may provide a global npm cache. Keep the image's read-only Sites
24 # seed separate and make this project's writable cache authoritative.
25 unset NPM_CONFIG_CACHE npm_config_cache || true
26 export npm_config_cache="${runtime_root}/npm-cache"
27 export npm_config_audit=false
28 export npm_config_fund=false
29 export npm_config_update_notifier=false
30
31 # The runtime already supplies the standard HTTP(S)_PROXY variables. Remove
32 # npm-specific aliases so npm 11 does not reinterpret or warn about them.
33 unset \
34   npm_config_proxy \
35   npm_config_http_proxy \
36   npm_config_https_proxy \
37   NPM_CONFIG_PROXY \
38   NPM_CONFIG_HTTP_PROXY \
39   NPM_CONFIG_HTTPS_PROXY \
40   || true
41
42 if [[ "${1:-}" == "--" ]]; then
43   shift
44 fi
45
46 if [[ "$#" -eq 0 ]]; then
47   echo "usage: scripts/sites-env.sh -- command [args...]" >&2
48   exit 64
49 fi
50
51 cd "${project_root}"
52 exec "$@"
```

scripts/transcribe-demo.mjs

```
1 import fs from 'node:fs/promises';
2 const form=new FormData();form.append('file',new Blob([await fs.readFile('submission/Lotlight-Narration.wav'
  )]),{type:'audio/wav'}),'narration.wav');form.append('model','whisper-
  1');form.append('response_format','verbose_json');form.append('timestamp_granularities[]','word');form.append('language','en');
3 const r=await
  fetch('https://api.openai.com/v1/audio/transcriptions',{method:'POST',headers:{Authorization:`Bearer
    ${process.env.OPENAI_API_KEY}`},body:form});if(!r.ok)throw new Error(`Transcription failed:
    ${r.status}`);const data=await r.json();await fs.writeFile('.artifact-
    build/narration/alignment.json',JSON.stringify(data,null,2));console.log('Narration transcribed for
    synchronized captions');
```

server/index.ts

```

1 import express from "express";
2 import { initializeApp } from "firebase-admin/app";
3 import { getAuth } from "firebase-admin/auth";
4 import { getFirestore } from "firebase-admin/firestore";
5 import { gzipSync, gunzipSync } from "node:zlib";
6 import { z } from "zod";
7 import { initialWorkspace } from "../lib/lotlight/sample";
8 import { reduceWorkspace } from "../lib/lotlight/commands";
9 import type { Workspace } from "../lib/lotlight/types";
10 const projectId = process.env.FIREBASE_PROJECT_ID || "lotlight-care";
11 initializeApp({ projectId });
12 const db = getFirestore();
13 const auth = getAuth();
14 const app = express();
15 const origins = new Set([
16   "https://lotlight-care.web.app",
17   "https://lotlight-care.firebaseio.com",
18   ...(process.env.NODE_ENV === "production"
19     ? []
20     : ["http://127.0.0.1:5190", "http://localhost:5190"]),
21 ]);
22 app.disable("x-powered-by");
23 app.use((req, res, next) => {
24   res.set({ "Cache-Control": "no-store", "X-Content-Type-Options": "nosniff" });
25   const origin = req.get("origin");
26   if (origin && !origins.has(origin)) {
27     res.status(403).json({ error: "Request origin rejected." });
28     return;
29   }
30   if (origin) {
31     res.set("Access-Control-Allow-Origin", origin);
32     res.set("Vary", "Origin");
33     res.set("Access-Control-Allow-Headers", "Authorization, Content-Type");
34     res.set("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
35   }
36   if (req.method === "OPTIONS") {
37     res.sendStatus(204);
38     return;
39   }
40   next();
41 });
42 app.get("/health", (_req, res) =>
43   res.json({ status: "ok", service: "lotlight-api" }));
44 );
45 app.use(express.json({ limit: "600kb" }));
46 const envelope = z
47   .object({ revision: z.number().int().min(0), command: z.unknown() })
48   .strict();
49 const budgets = new Map<string, { start: number; count: number }>();
50 app.all("/api/workspace", async (req, res) => {
51   if (!["GET", "POST"].includes(req.method)) {
52     res
53       .set("Allow", "GET, POST")
54       .status(405)
55       .json({ error: "Method not allowed." });
56     return;
57   }
58   const token = req.get("authorization")?.match(/^Bearer (.+)$/)?.[1];
59   let user;
60   try {
61     if (!token) throw Error("missing");
62     user = await auth.verifyIdToken(token, true);
63   } catch {
64     res.status(401).json({ error: "Sign in to access your saved workspace." });
65     return;
66   }
67   const now = Date.now(),
68     budget = budgets.get(user.uid);
69   if (budgets.size > 10000)
70     for (const [id, b] of budgets)
71       if (now - b.start > 60000) budgets.delete(id);
72   if (budget && now - budget.start < 60000) {
73     if (++budget.count > 90) {
74       res
75         .set("Retry-After", "60")
76         .status(429)
77         .json({ error: "Too many requests. Wait a minute and retry." });
78       return;
79     }
80   } else budgets.set(user.uid, { start: now, count: 1 });

```

server/index.ts (continued)

```

81  const ref = db.collection("lotlightWorkspaces").doc(user.uid);
82  try {
83    if (req.method === "POST" && !req.get("origin")) {
84      res.status(403).json({ error: "Request origin required." });
85      return;
86    }
87    if (req.method === "POST" && !req.is("application/json")) {
88      res.status(415).json({ error: "Use a JSON request." });
89      return;
90    }
91    const payload = req.method === "POST" ? envelope.parse(req.body) : null;
92    const state = await db.runTransaction(async (tx) => {
93      const snapshot = await tx.get(ref);
94      const current: Workspace = snapshot.exists
95        ? JSON.parse(
96          gunzipSync(snapshot.get("blob"), {
97            maxLength: 3_000_000,
98          }).toString(),
99        )
100      : initialWorkspace();
101      if (payload && payload.revision !== current.revision)
102        throw Object.assign(
103          new Error(
104            "This workspace changed in another tab. Reload the latest version before saving.",
105          ),
106          { status: 409 },
107        );
108      let next: Workspace;
109      try {
110        next = payload
111          ? reduceWorkspace(
112            current,
113            payload.command,
114            user.name || user.email || "Account reviewer",
115          )
116          : current;
117      } catch (error) {
118        throw Object.assign(
119          error instanceof Error ? error : new Error("Invalid update."),
120          { status: 400 },
121        );
122      }
123      if (payload || !snapshot.exists) {
124        const serialized = JSON.stringify(next);
125        if (Buffer.byteLength(serialized) > 3_000_000)
126          throw Object.assign(
127            new Error(
128              "Workspace limit reached. Export your record before starting a fresh workspace.",
129            ),
130            { status: 413 },
131          );
132        const blob = gzipSync(serialized);
133        if (blob.length > 900_000)
134          throw Object.assign(
135            new Error(
136              "Workspace storage limit reached. Export before starting a fresh workspace.",
137            ),
138            { status: 413 },
139          );
140        tx.set(ref, {
141          blob,
142          revision: next.revision,
143          updatedAt: new Date().toISOString(),
144        });
145      }
146      return next;
147    });
148    res.json(state);
149  } catch (e) {
150    const err = e as Error & { status?: number; code?: number };
151    const inputError = e instanceof z.ZodError;
152    const status = err.status || (inputError ? 400 : 503);
153    if (status === 503) console.error("workspace_failure", err.name, err.code);
154    res.status(status).json({
155      error:
156        status === 503
157          ? "Your saved workspace is unavailable. Retry without changing your inputs."
158          : e instanceof z.ZodError
159            ? e.issues.map((i) => i.message).join(" ")
160            : err.message,

```

server/index.ts (continued)

```
161     });
162   }
163 });
164 app.use(
165   (
166     err: Error & { status?: number },
167     _req: express.Request,
168     res: express.Response,
169     _next: express.NextFunction,
170   ) =>
171     res.status(err.status === 413 ? 413 : 400).json({
172       error:
173         err.status === 413
174           ? "Request exceeds the size limit."
175           : "Invalid JSON request.",
176     }),
177 );
178 app.listen(Number(process.env.PORT || 5191), "0.0.0.0", () =>
179   console.log("Lotlight API ready"),
180 );
```

tests/e2e/accessibility.spec.ts

```
1 import { test, expect } from '@playwright/test';
2 import AxeBuilder from '@axe-core/playwright';
3 test('dashboard and review have no axe WCAG A/AA violations', async({page})=>{await page.goto('/');await
  page.locator('main[data-ready="true"]').waitFor();expect((await new
  AxeBuilder({page}).withTags(['wcag2a', 'wcag2aa']).analyze()).violations).toEqual([]);await
  page.getByRole('button', {name: 'Open review', exact:true}).click();expect((await new
  AxeBuilder({page}).withTags(['wcag2a', 'wcag2aa']).analyze()).violations).toEqual([]);});
```

tests/e2e/firebase.spec.ts

```

1 import { test, expect } from "@playwright/test";
2 import { randomUUID } from "node:crypto";
3 import { readFileSync } from "node:fs";
4 const config = JSON.parse(
5   readFileSync(
6     new URL("../../public/firebase-config.json", import.meta.url),
7     "utf8",
8   ),
9 );
10 const enabled = process.env.LOTLIGHT_FIREBASE === "1";
11 test.skip(!enabled, "Firebase release checks only");
12 test.setTimeout(90000);
13 const expectCloud = expect.configure({ timeout: 20000 });
14 const origin = "https://lotlight-care.web.app";
15 async function identity(method: string, body: unknown) {
16   const r = await fetch(
17     `https://identitytoolkit.googleapis.com/v1/accounts:${method}?key=${config.apiKey}`,
18     {
19       method: "POST",
20       headers: { "Content-Type": "application/json" },
21       body: JSON.stringify(body),
22     },
23   );
24   return { status: r.status, data: (await r.json()) as any };
25 }
26 test("Firebase API isolates accounts and enforces workflow, origin and revision gates", async () => {
27   const accounts: any[] = [];
28   async function account() {
29     const r = await identity("signUp", {
30       email: `lotlight-qa-${randomUUID()}@example.invalid`,
31       password: randomUUID() + "Strong!",
32       returnSecureToken: true,
33     });
34     expect(r.status).toBe(200);
35     accounts.push(r.data);
36     return r.data.idToken;
37   }
38   async function api(token: string, body?: unknown, source = origin) {
39     return fetch(config.apiBase + "/api/workspace", {
40       method: body ? "POST" : "GET",
41       headers: {
42         Authorization: "Bearer " + token,
43         Origin: source,
44         "Content-Type": "application/json",
45       },
46       ...(body ? { body: JSON.stringify(body) } : {}),
47     });
48   }
49   try {
50     expect((await api("invalid")).status).toBe(401);
51     const a = await account(),
52       b = await account();
53     const ar = await api(a);
54     expect(ar.status).toBe(200);
55     const s = (await ar.json()) as any;
56     const br = await api(b);
57     expect(br.status).toBe(200);
58     const cmd = {
59       revision: s.revision,
60       command: {
61         type: "approve_notice",
62         noticeId: "baxter-demo",
63         sourceConfirmed: true,
64       },
65     };
66     expect((await api(a, cmd, "https://other.example")).status).toBe(403);
67     expect((await api(a, { ...cmd, actor: "forged" })).status).toBe(400);
68     const approved = await api(a, cmd);
69     expect(approved.status).toBe(200);
70     expect((await api(a, cmd)).status).toBe(409);
71     const bs = (await (await api(b)).json()) as any;
72     expect(bs.notices[0].approvedAt).toBeNull();
73     expect(
74       (
75         await api(a, {
76           revision: s.revision + 1,
77           command: {
78             type: "save_action",
79             noticeId: "baxter-demo",
80             itemId: "sample-3",

```

tests/e2e/firebase.spec.ts (continued)

```

81         owner: "Test reviewer",
82         dueDate: "2026-10-01",
83         status: "completed",
84         verification: "Label check for synthetic test only",
85         evidence: "Synthetic evidence",
86         handledQuantity: 8,
87     },
88 })
89 ).status,
90 ).toBe(400);
91 } finally {
92     for (const a of accounts) await identity("delete", { idToken: a.idToken });
93 }
94 });
95 test("Firebase account creation, save, reload, sign out and sign back in", async ({
96     page,
97 }) => {
98     const email = `lotlight-ui-${randomUUID()}@example.invalid`,
99     password = randomUUID() + "Strong!";
100     let token: string | undefined;
101     try {
102         await page.goto("/");
103         await page.getByRole("link", { name: "Sign in to save" }).click();
104         await page
105             .getByRole("button", { name: "New here? Create an account" })
106             .click();
107         await page.getByLabel("Email address", { exact: true }).fill(email);
108         await page.getByLabel("Password", { exact: true }).fill(password);
109         await page
110             .getByRole("button", { name: "Create account", exact: true })
111             .click();
112         await expectCloud(
113             page.getByText("Account storage", { exact: true }),
114             ).toBeVisible();
115         await page
116             .getByRole("button", { name: "Open review", exact: true })
117             .click();
118         await page.getByRole("tab", { name: "Source & scope" }).click();
119         await page.getByRole("checkbox").check();
120         await page.getByRole("button", { name: "Confirm source review" }).click();
121         await expectCloud(
122             page.getByText("Source review recorded", { exact: true }),
123             ).toBeVisible();
124         await page.reload();
125         await page
126             .getByRole("button", { name: "Open review", exact: true })
127             .click();
128         await page.getByRole("tab", { name: "Source & scope" }).click();
129         await expectCloud(
130             page.getByText("Source review recorded", { exact: true }),
131             ).toBeVisible();
132         await page.getByRole("link", { name: "Sign out", exact: true }).click();
133         await page.getByRole("link", { name: "Sign in to save" }).click();
134         await page.getByLabel("Email address", { exact: true }).fill(email);
135         await page.getByLabel("Password", { exact: true }).fill(password);
136         await page.getByRole("button", { name: "Sign in", exact: true }).click();
137         await expectCloud(
138             page.getByText("Account storage", { exact: true }),
139             ).toBeVisible();
140         await page.route("**/api/workspace", (route) =>
141             route.fulfill({
142                 status: 503,
143                 contentType: "application/json",
144                 headers: { "Access-Control-Allow-Origin": origin },
145                 body: JSON.stringify({ error: "Test connection failure" }),
146             }),
147         );
148         await page.reload();
149         await expectCloud(
150             page.getByRole("heading", {
151                 name: "Your saved workspace could not open",
152             }),
153             ).toBeVisible();
154         await expectCloud(
155             page.getByRole("button", { name: "Open review", exact: true }),
156             ).toHaveCount(0);
157     } finally {
158         const a = await identity("signInWithPassword", {
159             email,
160             password,

```

tests/e2e/firebase.spec.ts (continued)

```

161     returnSecureToken: true,
162   });
163   token = a.data.idToken;
164   if (token) await identity("delete", { idToken: token });
165 }
166 });
167
168 test('Firebase sign-in dialog is accessible and mobile sized', async({page})=>{
169   const {default:AxeBuilder}=await import('@axe-core/playwright');
170   await page.setViewportSize({width:390,height:844});await page.goto('/');await
page.getByRole('link',{name:'Sign in to save'}).click();
171   await expect(page.getByRole('dialog')).toBeVisible();const result=await new AxeBuilder({page}).include('[ro
le="dialog"]').withTags(['wcag2a','wcag2aa']).analyze();expect(result.violations).toEqual([]);
172   const size=await page.getByRole('dialog').boundingBox();expect(size!.width).toBeLessThanOrEqual(390);
173 });
174
175 test('configuration outage presents a retry screen instead of a blank page', async ({page}) => {
176   await page.route('**/firebase-config.json', route => route.fulfill({status:503,body:'Unavailable'}));
177   await page.goto('/');
178   await expect(page.getByRole('heading',{name:'Lotlight could not load'})).toBeVisible();
179   await page.unroute('**/firebase-config.json');
180   await page.getByRole('button',{name:'Retry loading'}).click();
181   await expect(page.getByRole('button',{name:'Open review',exact:true})).toBeVisible();
182 });
183
184 test('deleted account tokens cannot reopen saved workspaces', async () => {
185   const a = await identity('signUp',{email:`lotlight-
qa-${randomUUID()}@example.invalid`,password:randomUUID()+'Strong!',returnSecureToken:true});
186   expect(a.status).toBe(200);
187   const token = a.data.idToken;
188   expect((await identity('delete',{idToken:token})).status).toBe(200);
189   const response = await fetch(config.apiBase+'/api/workspace',{headers:{Authorization:'Bearer
'+token,Origin:origin}});
190   expect(response.status).toBe(401);
191 });

```

tests/e2e/workflow.spec.ts

```

1 import { test, expect } from "@playwright/test";
2 import { SAMPLE_CSV } from "../../lib/lotlight/sample";
3 async function ready(page: import("@playwright/test").Page) {
4   await page.goto("/");
5   await page.locator('main[data-ready="true"]').waitFor();
6 }
7 async function approve(page: import("@playwright/test").Page) {
8   await page
9     .getByRole("button", { name: "Open review", exact: true })
10    .first()
11    .click();
12   await page.getByRole("tab", { name: "Source & scope" }).click();
13   await page.getByRole("checkbox").check();
14   await page.getByRole("button", { name: "Confirm source review" }).click();
15   await expect(
16     page.getByText("Source review recorded", { exact: true }),
17   ).toBeVisible();
18 }
19 test("full demo: gated source, staff response, export and unknown cannot complete", async ({
20   page,
21 }) => {
22   await ready(page);
23   await page.getByRole("button", { name: "Open review", exact: true }).click();
24   await page
25     .getByRole("button", {
26       name: "Show evidence for CLEARLINK extension set in Infusion room",
27     })
28     .click();
29   await expect(
30     page.getByRole("button", { name: "Assign response" }),
31   ).toBeDisabled();
32   await page.getByRole("tab", { name: "Source & scope" }).click();
33   await page.getByRole("checkbox").check();
34   await page.getByRole("button", { name: "Confirm source review" }).click();
35   await page.getByRole("tab", { name: "/Inventory review/" }).click();
36   await page
37     .getByRole("button", {
38       name: "Show evidence for CLEARLINK extension set in Infusion room",
39     })
40     .click();
41   await page.getByRole("button", { name: "Assign response" }).click();
42   await page.getByLabel("Responsible person").fill("Maya Rao");
43   await page.getByRole("combobox", { name: "Response stage" }).click();
44   await page.getByRole("option", { name: "/Completed/" }).click();
45   await page
46     .getByLabel("Physical-label and current-source verification")
47     .fill(
48       "Checked 24 physical labels against the full manufacturer source in this synthetic demonstration.",
49     );
50   await page.getByLabel("Units handled").fill("24");
51   await page
52     .getByLabel("Response evidence / reference")
53     .fill(
54       "Synthetic training record: isolation log Q-104 and supplier return reference RMA-210.",
55     );
56   await page
57     .getByRole("button", { name: "Save response", exact: true })
58     .click();
59   await expect(
60     page.getByText("Response complete", { exact: true }),
61   ).toBeVisible();
62   await page
63     .getByRole("button", {
64       name: "Show evidence for CLEARLINK extension set in Procedure room",
65     })
66     .click();
67   await page
68     .getByRole("button", { name: "Assign response", exact: true })
69     .click();
70   await page.getByRole("combobox", { name: "Response stage" }).click();
71   await expect(page.getByRole("option", { name: "/Completed/" })).toHaveAttribute(
72     "data-disabled",
73     "",
74   );
75   await page.keyboard.press("Escape");
76   await page.getByRole("button", { name: "Cancel", exact: true }).click();
77   await page.getByRole("button", { name: "Export record" }).click();
78   const downloaded = page.waitForEvent("download");
79   await page.getByRole("button", { name: "/Complete audit record/" }).click();
80   const file = await downloaded;

```

tests/e2e/workflow.spec.ts (continued)

```

81 expect(file.suggestedFilename()).toBe("lotlight-audit-record.json");
82 const stream = await file.createReadStream();
83 let output = "";
84 for await (const chunk of stream!) output += chunk;
85 const exported = JSON.parse(output);
86 expect(exported.payload.workspace.actions[0].handledQuantity).toBe(24);
87 expect(exported.payload.inventoryProvenance).toBe("synthetic");
88 expect(exported.integrity.digest).toHaveLength(64);
89 });
90 test("CSV errors preserve draft, import works, corrections preserve provenance", async ({
91   page,
92 }) => {
93   await ready(page);
94   await page.getByRole("button", { name: "Inventory", exact: true }).click();
95   await page.getByRole("button", { name: "Import CSV" }).click();
96   await page
97     .getByLabel("Inventory CSV", { exact: true })
98     .fill(SAMPLE_CSV.replace("24", "-2"));
99   await page.getByRole("button", { name: "Validate & import" }).click();
100  await expect(page.getByRole("alert")).toContainText("whole number");
101  await page.getByRole("button", { name: "Use synthetic example" }).click();
102  await page.getByRole("button", { name: "Validate & import" }).click();
103  await expect(page.getByText(/synthetic \xb7 8 lines/)).toBeVisible();
104  await page
105    .getByRole("button", {
106      name: "Correct CLEARLINK extension set in Procedure room",
107    })
108    .click();
109  await page.getByLabel("Lot", { exact: true }).fill("R25C31031");
110  await page.getByRole("button", { name: "Save correction" }).click();
111  await expect(page.getByText("R25C31031", { exact: true })).toHaveCount(2);
112 });
113 test("new source form resets after success; unsupported source pairing rejected", async ({
114   page,
115 }) => {
116   await ready(page);
117   await page.getByRole("button", { name: "New recall review" }).click();
118   await page
119     .getByRole("button", { name: "Use historical training extract" })
120     .click();
121   await page.getByRole("button", { name: "Save for source review" }).click();
122   await expect(
123     page.getByRole("button", { name: "Open review", exact: true }),
124   ).toHaveCount(2);
125   await page.getByRole("button", { name: "New recall review" }).click();
126   await expect(page.getByLabel("Product / review title")).toHaveValue("");
127 });
128 test("local sign-in, account persistence, stale revision and origin rejection", async ({
129   page,
130   request,
131 }) => {
132   test.skip(
133     process.env.LOTLIGHT_FIREBASE === "1",
134     "Sites-only development auth test",
135   );
136   await ready(page);
137   expect((await request.get("/api/workspace")).status()).toBe(401);
138   await page.getByRole("link", { name: "Sign in to save" }).click();
139   await expect(
140     page.getByText("Account storage", { exact: true }),
141   ).toBeVisible();
142   const reset = await page.evaluate(async () => {
143     const s = (await (await fetch("/api/workspace")).json()) as {
144       revision: number;
145     };
146     const r = await fetch("/api/workspace", {
147       method: "POST",
148       headers: { "Content-Type": "application/json" },
149       body: JSON.stringify({
150         revision: s.revision,
151         command: { type: "reset_demo" },
152       }),
153     });
154     return r.status;
155   });
156   expect(reset).toBe(200);
157   await page.reload();
158   await page.locator('main[data-ready="true"]').waitFor();
159   await expect(
160     page.getByText("Account storage", { exact: true }),

```

tests/e2e/workflow.spec.ts (continued)

```

161 ).toBeVisible();
162 await approve(page);
163 await page.reload();
164 await page.locator('main[data-ready="true"]').waitFor();
165 await expect(
166   page.getByText("SOURCE REVIEWED", { exact: true }),
167   ).toBeVisible();
168 const status = await page.evaluate(async () => {
169   const s = (await (await fetch("/api/workspace")).json()) as {
170     revision: number;
171   };
172   const r = await fetch("/api/workspace", {
173     method: "POST",
174     headers: { "Content-Type": "application/json" },
175     body: JSON.stringify({
176       revision: s.revision - 1,
177       command: { type: "reset_demo" },
178     }),
179   });
180   return r.status;
181 });
182 expect(status).toBe(409);
183 const bad = await page.request.post("/api/workspace", {
184   headers: {
185     Origin: "https://evil.example",
186     "Content-Type": "application/json",
187   },
188   data: { revision: 0, command: { type: "reset_demo" } },
189 });
190 expect(bad.status()).toBe(403);
191 await page.getByRole("link", { name: "Sign out" }).click();
192 await expect(
193   page.getByRole("link", { name: "Sign in to save" }),
194   ).toBeVisible();
195 expect((await page.request.get("/api/workspace")).status()).toBe(401);
196 });
197 test("mobile workflow fits viewport and retains navigation", async ({
198   page,
199 }) => {
200   await page.setViewportSize({ width: 390, height: 844 });
201   await ready(page);
202   expect(
203     await page.evaluate(
204       () => document.documentElement.scrollWidth <= window.innerWidth,
205     ),
206     ).toBe(true);
207   await page.getByRole("button", { name: "Open review", exact: true }).click();
208   await expect(
209     page.getByRole("button", { name: "Run local AI" }),
210     ).toBeVisible();
211   await page.getByRole("button", { name: "Toggle Sidebar" }).click();
212   await page.getByRole("button", { name: "Inventory", exact: true }).click();
213   await page.keyboard.press("Escape");
214   await expect(
215     page.getByRole("heading", { name: "Your inventory." }),
216     ).toBeVisible();
217 });
218 test("WebMCP read tool handles valid and invalid input without mutation", async ({
219   page,
220 }) => {
221   await page.addInitScript(() => {
222     (window as any).capturedTools = [];
223     Object.defineProperty(document, "modelContext", {
224       value: {
225         registerTool(tool: any) {
226           (window as any).capturedTools.push(tool);
227         },
228       },
229     });
230   });
231   await ready(page);
232   const result = await page.evaluate(() => {
233     const tool = (window as any).capturedTools.at(-1);
234     const good = tool.execute({});
235     let rejected = false;
236     try {
237       tool.execute({ extra: 1 });
238     } catch {
239       rejected = true;
240     }
241   });

```

tests/e2e/workflow.spec.ts (continued)

```
241     return { name: tool.name, good, rejected };
242   });
243   expect(result.name).toBe("lotlight_read_review");
244   expect(result.good.inventoryLines).toBe(8);
245   expect(result.rejected).toBe(true);
246 });
247
248 test("text-based PDF notice import preserves text and extracts explicit pairs", async ({
249   page,
250 }) => {
251   await ready(page);
252   await page.getByRole("button", { name: "New recall review" }).click();
253   await page
254     .getByLabel("Upload recall notice")
255     .setInputFiles("public/samples/baxter-training-extract.pdf");
256   await expect(
257     page.getByLabel("Original notice text / clearly labeled training extract"),
258   ).toContainText("R25C31031");
259   await expect(page.getByLabel("Catalog", { exact: true })).toHaveCount(2);
260 });
```

tests/engine.test.ts

```

1 import { test } from "node:test";
2 import assert from "node:assert/strict";
3 import {
4   matchItem,
5   identifier,
6   extractScope,
7   validateScope,
8   parseCsv,
9   exportCsv,
10 } from "../lib/lotlight/engine";
11 import {
12   initialWorkspace,
13   SAMPLE_CSV,
14   sampleNotice,
15 } from "../lib/lotlight/sample";
16 import { reduceWorkspace, activeAction } from "../lib/lotlight/commands";
17 import type { InventoryItem, Notice } from "../lib/lotlight/types";
18 const fixture = () => initialWorkspace();
19 const item = (patch: Partial<InventoryItem> = {}) => ({
20   ...fixture().inventory[0],
21   ...patch,
22 });
23 const approve = () =>
24   reduceWorkspace(
25     fixture(),
26     { type: "approve_notice", noticeId: "baxter-demo", sourceConfirmed: true },
27     "Test reviewer",
28   );
29 const action = (patch: Record<string, unknown> = {}) => ({
30   type: "save_action",
31   noticeId: "baxter-demo",
32   itemId: "sample-1",
33   owner: "Maya Rao",
34   dueDate: "2026-10-01",
35   status: "assigned",
36   verification: "",
37   evidence: "",
38   handledQuantity: 0,
39   ...patch,
40 });
41 test("exact match requires manufacturer, paired catalog and lot", () =>
42   assert.equal(matchItem(item(), sampleNotice).kind, "exact"));
43 test("cross-product lot must not match", () =>
44   assert.equal(
45     matchItem(item({ lot: "R25A13024" }), sampleNotice).kind,
46     "unlisted",
47   ));
48 test("missing lot never becomes exact at maximum semantic similarity", () =>
49   assert.equal(matchItem(item({ lot: "" }), sampleNotice, 1).kind, "missing"));
50 test("semantic candidate cannot override explicit catalog mismatch", () =>
51   assert.equal(
52     matchItem(item({ catalog: "WRONG" }), sampleNotice, 1).kind,
53     "unlisted",
54   ));
55 test("missing catalog can only be an AI candidate", () =>
56   assert.equal(
57     matchItem(item({ catalog: "" }), sampleNotice, 0.9).kind,
58     "possible",
59   ));
60 test("different manufacturer cannot become exact", () =>
61   assert.equal(
62     matchItem(item({ manufacturer: "Other" }), sampleNotice, 1).kind,
63     "missing",
64   ));
65 test("missing manufacturer remains unresolved", () =>
66   assert.equal(
67     matchItem(item({ manufacturer: "" }), sampleNotice).kind,
68     "missing",
69   ));
70 test("normalization preserves zeros, punctuation and ambiguous letters", () => {
71   for (const [a, b] of [
72     ["00123", "123"],
73     ["AB-12", "AB12"],
74     ["R2501", "R2501"],
75     ["R25I1", "R2511"],
76   ])
77     assert.notEqual(identifier(a), identifier(b));
78   assert.equal(identifier(" ab-12 "), "AB-12");
79 });
80 test("same catalog on two lines supports either exact paired lot", () => {

```

tests/engine.test.ts (continued)

```

81   const n = {
82     ...sampleNotice,
83     scope: [
84       ...sampleNotice.scope,
85       {
86         catalog: "2C8632",
87         lots: ["OTHER"],
88         allLots: false,
89         evidence: "2C8632 OTHER",
90       },
91     ],
92   };
93   assert.equal(matchItem(item({ lot: "OTHER" }), n).kind, "exact");
94 });
95 test("extractor keeps product-to-lot pairs intact", () => {
96   const s = extractScope("Catalog: A; Lots: 111\nCatalog: B; Lots: 222");
97   assert.deepEqual(
98     s.map((r) => [r.catalog, r.lots]),
99     [
100       ["A", ["111"]],
101       ["B", ["222"]],
102     ],
103   );
104 });
105 test("extractor abstains on ranges and exclusions", () => {
106   for (const s of ["1 through 99", "1 to 99", "all lots except 3", "1..99"])
107     assert.equal(extractScope("Catalog: X; Lots: " + s).length, 0);
108 });
109 test("source validation refuses fabricated or substring identifiers", () => {
110   const n = {
111     manufacturer: "Baxter",
112     text: "Baxter Catalog: A; Lots: 1111",
113     scope: [
114       {
115         catalog: "A",
116         lots: ["111"],
117         allLots: false,
118         evidence: "Catalog: A; Lots: 1111",
119       },
120     ],
121   };
122   assert.match(validateScope(n) ?? "", /exactly/);
123   n.scope[0].evidence = "made up";
124   assert.match(validateScope(n) ?? "", /verbatim/);
125 });
126 test("all-lot scope needs clear source wording and no exclusions", () => {
127   const n = {
128     manufacturer: "Baxter",
129     text: "Baxter A all lots except X",
130     scope: [
131       {
132         catalog: "A",
133         lots: [],
134         allLots: true,
135         evidence: "Baxter A all lots except X",
136       },
137     ],
138   };
139   assert.match(validateScope(n) ?? "", /unambiguous|match|pair/);
140 });
141 test("CSV preserves missing identifiers and quantities", () => {
142   const data = parseCsv(SAMPLE_CSV);
143   assert.equal(data.length, 8);
144   assert.equal(
145     data.reduce((a, i) => a + i.quantity, 0),
146     218,
147   );
148   assert.equal(data[2].lot, "");
149 });
150 test("CSV supports BOM, CRLF, quoted commas and embedded newlines", () => {
151   const data = parseCsv(
152     '\uFEFFproduct,manufacturer,catalog,lot,quantity,location\r\n"Device, deluxe",Maker,C,L,2,"Room\nA"',
153   );
154   assert.equal(data[0].product, "Device, deluxe");
155   assert.equal(data[0].location, "Room\nA");
156 });
157 test("CSV rejects malformed, duplicate, oversized and invalid inputs", () => {
158   assert.throws(() => parseCsv("product,product\nx,y"), /duplicate/);
159   assert.throws(
160     () => parseCsv(SAMPLE_CSV + "\n" + SAMPLE_CSV.split("\n")[1]),

```

tests/engine.test.ts (continued)

```

161     /duplicate/,
162   );
163   assert.throws(
164     () => parseCsv(SAMPLE_CSV.replace(",24,", " ,0,")),
165     /whole number/,
166   );
167   assert.throws(
168     () => parseCsv(SAMPLE_CSV.replace(",24,", " ,1.2,")),
169     /whole number/,
170   );
171   assert.throws(() => parseCsv(SAMPLE_CSV + '\n"broken'), /unclosed/);
172   assert.throws(() => parseCsv("x".repeat(500001)), /limit/);
173 });
174 test("CSV exports neutralize formulas and quote unsafe values", () => {
175   const s = exportCsv([
176     ['=HYPERLINK("evil")', " +1", "@SUM(A1)", "-2", "normal, value"],
177   ]);
178   assert.ok(s.includes("'=HYPERLINK"));
179   assert.ok(s.includes("' +1"));
180   assert.ok(s.includes("'@SUM"));
181   assert.ok(s.includes("'normal, value"));
182 });
183 test("unapproved source blocks direct action mutation", () =>
184   assert.throws(
185     () => reduceWorkspace(fixture(), action(), "Attacker"),
186     /source/,
187   ));
188 test("approval actor and timestamp are server inputs, extra client fields rejected", () => {
189   assert.throws(() =>
190     reduceWorkspace(
191       fixture(),
192       {
193         type: "approve_notice",
194         noticeId: "baxter-demo",
195         sourceConfirmed: true,
196         approvedBy: "Doctor",
197       },
198       "Real user",
199     ),
200   );
201   const s = approve();
202   assert.equal(s.notices[0].approvedBy, "Test reviewer");
203 });
204 test("assigned response is version-bound and attributed", () => {
205   const s = reduceWorkspace(approve(), action(), "Operator");
206   assert.equal(s.actions[0].updatedBy, "Operator");
207   assert.equal(activeAction(s, "baxter-demo", "sample-1")?.status, "assigned");
208 });
209 test("cannot claim handled units before verification", () =>
210   assert.throws(
211     () =>
212       reduceWorkspace(approve(), action({ handledQuantity: 1 }), "Operator"),
213     /Verify/,
214   ));
215 test("uncertain identifiers cannot be verified or completed", () =>
216   assert.throws(
217     () =>
218       reduceWorkspace(
219         approve(),
220         action({
221           itemId: "sample-3",
222           status: "completed",
223           verification: "Checked label and source",
224           evidence: "Return record Q100",
225           handledQuantity: 8,
226         }),
227         "Operator",
228       ),
229     /missing identifiers/,
230   ));
231 test("completion rejects absent evidence, partial counts and excessive counts", () => {
232   const a = {
233     status: "completed",
234     verification: "Checked physical label and current letter",
235     handledQuantity: 24,
236   };
237   assert.throws(
238     () => reduceWorkspace(approve(), action(a), "Operator"),
239     /evidence/,
240   );

```

tests/engine.test.ts (continued)

```
241   assert.throws(  
242     () =>  
243       reduceWorkspace(  
244         approve(),  
245         action({  
246           ...a,  
247           evidence: "Return receipt RMA-100",  
248           handledQuantity: 20,  
249         })),  
250     "Operator",  
251   ),  
252   /all recorded/,  
253 );  
254   assert.throws(  
255     () =>  
256       reduceWorkspace(  
257         approve(),  
258         action({  
259           ...a,  
260           evidence: "Return receipt RMA-100",  
261           handledQuantity: 25,  
262         })),  
263     "Operator",  
264   ),  
265   /exceed/,  
266 );  
267 });  
268   test("partial handling stays verified and full handling can complete", () => {  
269     const partial = reduceWorkspace(  
270       approve(),  
271       action({  
272         status: "verified",  
273         verification: "Checked label and current notice",  
274         evidence: "Isolated under Q-100",  
275         handledQuantity: 12,  
276       })),  
277     "Operator",  
278   );  
279     assert.equal(partial.actions[0].status, "verified");  
280     const done = reduceWorkspace(  
281       partial,  
282       action({  
283         status: "completed",  
284         verification: "Checked label and current notice",  
285         evidence: "Return receipt RMA-100",  
286         handledQuantity: 24,  
287       })),  
288     "Operator",  
289   );  
290     assert.equal(done.actions.length, 1);  
291     assert.equal(done.actions[0].status, "completed");  
292     assert.equal(done.audit.length, 4);  
293   });  
294   test("source edit invalidates approval and earlier responses", () => {  
295     const s = reduceWorkspace(approve(), action(), "Operator");  
296     const { id, title, manufacturer, date, sourceUrl, text, scope, training } =  
297       s.notices[0];  
298     const next = reduceWorkspace(  
299       s,  
300       {  
301         type: "save_notice",  
302         notice: {  
303           id,  
304           title,  
305           manufacturer,  
306           date,  
307           sourceUrl,  
308           text,  
309           scope,  
310           training,  
311           action:  
312             "Amended response summary from current manufacturer instructions.",  
313         },  
314       },  
315       "Editor",  
316     );  
317     assert.equal(next.notices[0].approvedAt, null);  
318     assert.equal(next.notices[0].version, 2);  
319     assert.equal(activeAction(next, id, "sample-1"), undefined);  
320     assert.equal(next.actions.length, 1);
```

tests/engine.test.ts (continued)

```

321 });
322 test("inventory replacement cannot inherit completed responses", () => {
323   const s = reduceWorkspace(approve(), action(), "Operator");
324   const next = reduceWorkspace(
325     s,
326     { type: "import_inventory", csv: SAMPLE_CSV },
327     "Editor",
328   );
329   assert.equal(activeAction(next, "baxter-demo", "sample-1"), undefined);
330   assert.equal(next.inventoryVersion, 2);
331   assert.equal(next.actions.length, 1);
332   assert.equal(
333     activeAction(next, "baxter-demo", next.inventory[0].id),
334     undefined,
335   );
336 });
337 test("reject unsafe source URLs and impossible dates", () => {
338   const { id, approvedAt, approvedBy, version, ...n } = sampleNotice;
339   assert.throws(() =>
340     reduceWorkspace(
341       fixture(),
342       {
343         type: "save_notice",
344         notice: { ...n, sourceUrl: "javascript:alert(1)" },
345       },
346       "Editor",
347     ),
348   );
349   assert.throws(() =>
350     reduceWorkspace(approve(), action({ dueDate: "2026-02-30" }), "Editor"),
351   );
352 });
353 test("unknown commands and actions on unrelated inventory are rejected", () => {
354   assert.throws(() =>
355     reduceWorkspace(fixture(), { type: "admin_override" }, "Attacker"),
356   );
357   assert.throws(
358     () =>
359       reduceWorkspace(
360         approve(),
361         action({ itemId: "other-user-item" }),
362         "Attacker",
363       ),
364     /not found/,
365   );
366   assert.throws(
367     () =>
368       reduceWorkspace(approve(), action({ itemId: "sample-8" }), "Operator"),
369     /no listed/,
370   );
371 });
372
373 test("reject cross-pair support spanning two products", () => {
374   const n = {
375     manufacturer: "Baxter",
376     text: "Baxter\nCatalog: A; Lots: 111\nCatalog: B; Lots: 222",
377     scope: [
378       {
379         catalog: "A",
380         lots: ["222"],
381         allLots: false,
382         evidence: "Catalog: A; Lots: 111\nCatalog: B; Lots: 222",
383       },
384     ],
385   };
386   assert.notEqual(validateScope(n), null);
387 });
388 test("qualified all-lot statements are unsupported", () => {
389   for (const qualifier of [
390     "manufactured before January 2024",
391     "within serial range 10-20",
392     "shipped after 2025",
393   ]) {
394     const evidence = "Catalog: A; Lots: all lots " + qualifier;
395     assert.notEqual(
396       validateScope({
397         manufacturer: "Baxter",
398         text: "Baxter\n" + evidence,
399         scope: [{ catalog: "A", lots: [], allLots: true, evidence }],
400       }),

```

tests/engine.test.ts (continued)

```

401     null,
402   );
403 }
404 });
405 test("manufacturer aliases remain in attention queue", () =>
406   assert.equal(
407     matchItem(
408       item({ manufacturer: "Baxter Healthcare Corporation" }),
409       sampleNotice,
410       0.99,
411     ).kind,
412     "missing",
413   );
414 test("single-line correction preserves other actions and prior inventory snapshot", () => {
415   let s = reduceWorkspace(approve(), action(), "Operator");
416   const other = s.inventory[2];
417   s = reduceWorkspace(
418     s,
419     {
420       type: "correct_item",
421       id: other.id,
422       product: other.product,
423       manufacturer: other.manufacturer,
424       catalog: other.catalog,
425       lot: "R25C31031",
426       quantity: other.quantity,
427       location: other.location,
428     },
429     "Editor",
430   );
431   assert.ok(activeAction(s, "baxter-demo", "sample-1"));
432   assert.equal(s.inventoryHistory[0].items[2].lot, "");
433   assert.equal(s.inventory[2].lot, "R25C31031");
434 });
435 test("source history preserves original evidence after edits", () => {
436   const current = approve();
437   const { approvedAt, approvedBy, version, ...n } = current.notices[0];
438   const next = reduceWorkspace(
439     current,
440     {
441       type: "save_notice",
442       notice: { ...n, action: "A revised staff-entered response summary." },
443     },
444     "Editor",
445   );
446   assert.equal(next.noticeHistory[0].text, current.notices[0].text);
447   assert.equal(next.noticeHistory[0].approvedBy, "Test reviewer");
448 });
449
450 test("CSV rejects characters and repeated opening quotes after a closing quote", () => {
451   const header = "product,manufacturer,catalog,lot,quantity,location\n";
452   for (const value of [
453     "IV line"x,Baxter,ABC,L1,1,Shelf',
454     "IV line" "more",Baxter,ABC,L1,1,Shelf',
455   ])
456     assert.throws(() => parseCsv(header + value), /closing CSV quote/);
457 });
458 test("scope rejects an empty concrete lot list", () => {
459   const text = "Baxter\nCatalog: ABC; Lots: ,";
460   assert.equal(extractScope(text).length, 0);
461   assert.notEqual(
462     validateScope({
463       manufacturer: "Baxter",
464       text,
465       scope: [
466         {
467           catalog: "ABC",
468           lots: [],
469           allLots: false,
470           evidence: "Catalog: ABC; Lots: ,",
471         },
472       ],
473     }),
474     null,
475   );
476 });
477 test("inventory corrections cannot duplicate another record", () => {
478   const s = fixture(),
479     target = s.inventory[0],
480     source = s.inventory[1];

```

tests/engine.test.ts (continued)

```
481   assert.throws(  
482     () =>  
483       reduceWorkspace(  
484         s,  
485         {  
486           type: "correct_item",  
487           id: target.id,  
488           product: source.product,  
489           manufacturer: source.manufacturer,  
490           catalog: source.catalog,  
491           lot: source.lot,  
492           quantity: source.quantity,  
493           location: source.location,  
494         },  
495         "Editor",  
496       ),  
497       /Duplicate inventory line/,  
498     );  
499 });
```

vite.config.ts

```

1 import vinext from "vinext";
2 import { defineConfig } from "vite";
3 import hostingConfig from "../openai/hosting.json";
4 import { readExecutionProfile } from "../scripts/execution-profile.mjs";
5 import { sites } from "../build/sites-vite-plugin";
6
7 const SITE_CREATOR_PLACEHOLDER_DATABASE_ID =
8   "00000000-0000-4000-8000-000000000000";
9
10 const { d1, r2 } = hostingConfig;
11
12 // macOS Seatbelt blocks FSEvents, so Codex previews need polling for HMR.
13 const isCodexSeatbeltSandbox = process.env.CODEX_SANDBOX === "seatbelt";
14 const managedLinux = readExecutionProfile() === "managed-linux";
15
16 const localBindingConfig = {
17   main: "vinext/server/fetch-handler",
18   compatibility_flags: ["nodejs_compat"],
19   d1_databases: d1
20   ? [
21     {
22       binding: d1,
23       database_name: "site-creator-d1",
24       database_id: SITE_CREATOR_PLACEHOLDER_DATABASE_ID,
25     },
26   ]
27   : [],
28   r2_buckets: r2
29   ? [
30     {
31       binding: r2,
32       bucket_name: "site-creator-r2",
33     },
34   ]
35   : [],
36 };
37
38 export default defineConfig(async () => {
39   // Use Miniflare's local Request.cf placeholder unless fetching is requested.
40   process.env.CLOUDFLARE_CF_FETCH_ENABLED ??= "false";
41   process.env.WRANGLER_SEND_METRICS ??= "false";
42
43   // Keep Wrangler and Miniflare state project-local. These are non-secret tool
44   // settings; application environment belongs in ignored `.env*` files.
45   process.env.WRANGLER_WRITE_LOGS ??= "false";
46   process.env.WRANGLER_LOG_PATH ??= ".wrangler/logs";
47   process.env.WRANGLER_REGISTRY_PATH ??= ".wrangler/dev-registry";
48   process.env.MINIFLARE_REGISTRY_PATH ??= ".wrangler/registry";
49
50   // Wrangler snapshots its log path while the Cloudflare plugin is imported.
51   const { cloudflare } = await import("@cloudflare/vite-plugin");
52
53   return {
54     server: {
55       ...(managedLinux ? { host: "0.0.0.0", allowedHosts: ["terminal.local"] } : {}),
56       ...(isCodexSeatbeltSandbox ? { watch: { useFsEvents: false, usePolling: true } } : {}),
57     },
58     plugins: [
59       vinext(),
60       sites({ mockAuth: !managedLinux }),
61       cloudflare({
62         viteEnvironment: { name: "rsc", childEnvironments: ["ssr"] },
63         inspectorPort: false,
64         config: localBindingConfig,
65       }),
66     ],
67   };
68 });

```

vite.firebase.config.ts

```
1 import { defineConfig } from "vite";
2 import react from "@vitejs/plugin-react";
3 import path from "node:path";
4 export default defineConfig({
5   root: "firebase",
6   publicDir: "../public",
7   plugins: [react()],
8   resolve: { alias: { "@": path.resolve(".") } },
9   build: { outDir: "../dist-firebase", emptyOutDir: true },
10  server: { host: "127.0.0.1", port: 5190, fs: { allow: [path.resolve(".") ] } },
11 });
```